

0 Abstract

📖 단락 0-1

Enabling LLMs to improve their outputs by using more test-time compute is a critical step towards building self-improving agents that can operate on open-ended natural language. In this paper, we scale up inference-time computation in LLMs, with a focus on answering: if an LLM is allowed to use a fixed but non-trivial amount of inference-time compute, how much can it improve its performance on a challenging prompt?

✅ 문장 0-1-1

Enabling LLMs to improve their outputs by using more test-time compute is a critical step towards building self-improving agents that can operate on open-ended natural language.

🔍 단위 해석

LLM이 테스트 시점(inference time)에 더 많은 연산 자원을 사용하여 출력을 개선하도록 하는 것은, 개방형 자연어 작업을 수행할 수 있는 자가 발전형(self-improving) 에이전트를 구축하는 데 있어 필수적인 단계입니다.

💡 보조 설명

- **[Test-time compute]:** 모델 학습이 끝난 후, 실제 사용(추론) 단계에서 추가적인 연산 시간을 들여 답을 정제하거나 탐색하는 과정을 의미합니다. (예: 생각 할 시간을 더 주는 것)

✅ 문장 0-1-2

In this paper, we scale up inference-time computation in LLMs, with a focus on answering: if an LLM is allowed to use a fixed but non-trivial amount of inference-time compute, how much can it improve its performance on a challenging prompt?

🔍 단위 해석

본 논문에서는 LLM의 추론 시점 연산량을 확장하며 다음 질문에 답하는 데 초점을 맞춥니다: 만약 LLM에게 고정되어 있지만 결코 적지 않은 양의 추론 연산 자원을 허용한다면, 어려운 프롬프트에 대한 성능을 얼마나 향상시킬 수 있을까?

📖 단락 0-2

Answering this question has implications not only on performance, but also on the future of LLM pretraining and how to tradeoff inference-time and pre-training compute. Little research has attempted to understand the scaling behaviors of test-time inference methods, with current work largely providing negative results for a number of these strategies.

✅ 문장 0-2-1

Answering this question has implications not only on performance, but also on the future of LLM pretraining and how to tradeoff inference-time and pre-training compute.

🔍 단위 해석

이 질문에 대한 답은 단순히 성능뿐만 아니라, LLM 사전 학습(pretraining)의 미래와 ‘추론 시점 연산’ 대 ‘사전 학습 연산’ 간의 트레이드오프(tradeoff)를 어떻게 조절할지에 대한 시사점을 제공합니다.

✅ 문장 0-2-2

Little research has attempted to understand the scaling behaviors of test-time inference methods, with current work largely providing negative results for a number of these strategies.

🔍 단위 해석

테스트 시점 추론 방법론들의 스케일링 동작(scaling behaviors)을 이해하려는 연구는 거의 없었으며, 기존 연구들은 대부분 이러한 전략들에 대해 부정적인 결과를 보여주었습니다.

📖 단락 0-3

In this work, we analyze two primary mechanisms to scale test-time computation: (1) searching against dense, process-based verifier reward models (PRMs); and (2) updating the model's distribution over a response adaptively, given the prompt at test time.

✅ 문장 0-3-1

In this work, we analyze two primary mechanisms to scale test-time computation: (1) searching against dense, process-based verifier reward models (PRMs); and (2) updating the model's distribution over a response adaptively, given the prompt at test time.

🔍 단위 해석

본 연구에서는 테스트 시점 연산을 확장하기 위한 두 가지 주요 메커니즘을 분석합니다: (1) 조밀한 과정 중심 검증 보상 모델(Process-based Verifier Reward Models, PRMs)을 활용한 탐색, 그리고 (2) 테스트 시점에 주어진 프롬프트에 맞춰 모델의 응답 분포를 적응적으로 업데이트하는 방법입니다.

💡 보조 설명

- **[PRM (Process Reward Model)]**: 최종 결과만 채점하는 것이 아니라, 풀이 과정의 각 단계(Step)마다 점수를 매겨 더 정교한 탐색(Search)을 가능하게 하는 모델입니다.
- **[Updating distribution]**: 단순히 생성하는 것이 아니라, 모델이 스스로 답을 수정(Revision)하게 하여 더 나은 답을 낼 확률(분포)을 높이는 방식을 의미합니다.

📖 단락 0-4

We find that in both cases, the effectiveness of different approaches to scaling test-time compute critically varies depending on the difficulty of the prompt. This observation motivates applying a “compute-optimal” scaling strategy, which acts to, as effectively as possible, allocate test-time compute per prompt in an adaptive manner.

✅ 문장 0-4-1

We find that in both cases, the effectiveness of different approaches to scaling test-time compute critically varies depending on the difficulty of the prompt.

🔍 단위 해석

우리는 두 경우 모두, 테스트 시점 연산 확장 접근법의 유효성이 프롬프트(문제)의 난이도에 따라 결정적으로 달라진다는 사실을 발견했습니다.

✅ 문장 0-4-2

This observation motivates applying a “compute-optimal” scaling strategy, which acts to, as effectively as possible, allocate test-time compute per prompt in an adaptive manner.

🔍 단위 해석

이러한 관찰은 각 프롬프트에 대해 테스트 시점 연산 자원을 최대한 효과적이고 적응적으로 할당하는 “연산 최적화(compute-optimal)” 스케일링 전략을 적용하도록 동기를 부여합니다.

💡 보조 설명

- **[Compute-optimal]**: 모든 문제에 똑같은 전략(예: 무조건 100번 다시 풀기)을 쓰는 게 아니라, 쉬운 문제는 적게, 어려운 문제는 많이, 혹은 문제 유형에 따라 다른 전략을 써서 자원 효율을 극대화하는 방식입니다.

📖 단락 0-5

Using this compute-optimal strategy, we can improve the efficiency of test-time compute scaling for math reasoning problems by more than 4x compared to a best-of-N baseline. Additionally, in a FLOPs-matched evaluation, we find that on problems where a smaller base model attains somewhat non-trivial success rates, test-time compute can be used to outperform a 14x larger model.

✅ 문장 0-5-1

Using this compute-optimal strategy, we can improve the efficiency of test-time compute scaling for math reasoning problems by more than 4x compared to a best-of-N baseline.

🔍 단위 해석

이 연산 최적화 전략을 사용함으로써, 수학 추론 문제에서 ‘Best-of-N’ 베이스라인과 비교해 테스트 시점 연산 스케일링의 효율성을 4배 이상 향상시킬 수 있습니다.

💡 보조 설명

- **[Best-of-N]**: N개의 답을 생성한 뒤 그중 가장 좋은 것을 고르는 가장 단순한 형태의 테스트 시점 연산 방법입니다.

✅ 문장 0-5-2

Additionally, in a FLOPs-matched evaluation, we find that on problems where a smaller base model attains somewhat non-trivial success rates, test-time compute can be used to outperform a 14x larger model.

단위 해석

또한, 연산량(FLOPs)을 동일하게 맞춘 평가에서, 작은 베이스 모델이 어느 정도 유의미한 성공률을 보이는 문제들의 경우, 테스트 시점 연산을 활용하면 14배 더 큰 모델보다 더 나은 성능을 낼 수 있음을 확인했습니다.

- 보조 설명
- [FLOPs-matched]: 공정한 비교를 위해 '작은 모델 + 추가 연산'의 총비용과 '큰 모델'의 비용을 같게 맞춘 상태를 뜻합니다.
 - [PDF 참조]: **Figure 1 (Right)*에서 작은 모델(PaLM 2-S)이 추가 연산을 통해 14배 큰 모델을 능가하는 구간(Star 마크 아래에 선이 위치하는 경우)을 시각적으로 확인할 수 있습니다.

Introduction

단락 1-1

Given a challenging input, can we enable LLMs to most effectively make use of additional computation at test-time to improve their responses? In theory, additional test-time compute should enable an LLM to do better than what it was trained to do in zero-shot.

문장 1-1-1

Given a challenging input, can we enable LLMs to most effectively make use of additional computation at test-time to improve their responses?

단위 해석

어려운 입력이 주어졌을 때, LLM이 테스트 시점에 추가적인 연산을 가장 효과적으로 활용하여 응답을 개선하도록 만들 수 있을까요?

문장 1-1-2

In theory, additional test-time compute should enable an LLM to do better than what it was trained to do in zero-shot.

단위 해석

이론적으로, 추가적인 테스트 시점 연산은 LLM이 제로샷(zero-shot, 추가 연산 없이 즉시 대답하는 것)으로 훈련된 수준보다 더 나은 성능을 발휘하게 해 줄 것입니다.

단락 1-2

Such a capability at test-time bears the potential to unlock agentic and reasoning abilities (Shinn et al., 2023; Qu et al., 2024b). Additionally, if pre-trained model size can be traded off for additional inference compute, this would enable the deployment of smaller on-device models in place of datacenter scale LLMs.

문장 1-2-1

Such a capability at test-time bears the potential to unlock agentic and reasoning abilities (Shinn et al., 2023; Qu et al., 2024b).

단위 해석

테스트 시점의 이러한 능력은 에이전트적(agentic) 능력과 추론 능력을 잠금 해제할 잠재력을 지니고 있습니다.

문장 1-2-2

Additionally, if pre-trained model size can be traded off for additional inference compute, this would enable the deployment of smaller on-device models in place of datacenter scale LLMs.

단위 해석

게다가, 사전 학습된 모델의 크기를 추가적인 추론 연산과 트레이드오프(교환)할 수 있다면, 이는 데이터센터 규모의 거대 LLM 대신 소형 온디바이스(on-device) 모델을 배포하는 것을 가능하게 할 것입니다.

단락 1-3

Automating the inference-time improvement of model outputs also provides a path towards a general self-improvement algorithm that can function with reduced human supervision. Prior work studying inference-time computation provides mixed results.

✔ 문장 1-3-1

Automating the inference-time improvement of model outputs also provides a path towards a general self-improvement algorithm that can function with reduced human supervision.

🔍 단위 해석

추론 시점에 모델 출력을 개선하는 과정을 자동화하는 것은, 인간의 감독을 줄이면서도 작동할 수 있는 범용적인 자가 발전(self-improvement) 알고리즘으로 가는 길을 열어줍니다.

✔ 문장 1-3-2

Prior work studying inference-time computation provides mixed results.

🔍 단위 해석

추론 시점 연산을 연구한 이전 연구들은 엇갈린 결과들을 보여줍니다.

📖 단락 1-4

On the one hand, some prior work shows that current LLMs can use test-time computation to improve their outputs [...], on the other hand, several other works show that the effectiveness of these methods on complex tasks such as math reasoning remains limited [...].

✔ 문장 1-4-1

On the one hand, some prior work shows that current LLMs can use test-time computation to improve their outputs (Bai et al., 2022; Madaan et al., 2023; Du et al., 2023; Saunders et al., 2022; Yao et al., 2023), on the other hand, several other works show that the effectiveness of these methods on complex tasks such as math reasoning remains limited (Huang et al., 2023; Stechly et al., 2023; Valmeekam et al., 2023; Wang et al., 2024a; Olausson et al., 2024).

🔍 단위 해석

한편으로는 현재의 LLM들이 테스트 시점 연산을 통해 출력을 개선할 수 있다는 선행 연구들이 있는 반면, 다른 한편으로는 수학 추론과 같은 복잡한 작업에서는 이러한 방법들의 효과가 여전히 제한적이라는 연구 결과들도 다수 존재합니다.

📖 단락 1-5

However, reasoning is a domain we should expect to benefit from test-time compute, since reasoning involves drawing inferences from existing knowledge as opposed to acquiring new knowledge. Therefore, the disagreement in these prior findings motivates the need for a systematic analysis of different approaches for scaling test-time compute.

✔ 문장 1-5-1

However, reasoning is a domain we should expect to benefit from test-time compute, since reasoning involves drawing inferences from existing knowledge as opposed to acquiring new knowledge.

🔍 단위 해석

하지만 추론(reasoning)은 새로운 지식을 습득하는 것이 아니라 기존 지식으로부터 논리적 결론을 도출하는 것이므로, 테스트 시점 연산의 혜택을 볼 것으로 기대해야 하는 영역입니다.

✔ 문장 1-5-2

Therefore, the disagreement in these prior findings motivates the need for a systematic analysis of different approaches for scaling test-time compute.

🔍 단위 해석

따라서, 이러한 선행 연구 결과들의 불일치는 테스트 시점 연산을 확장하는 다양한 접근법에 대한 체계적인 분석의 필요성을 제기합니다.

📖 단락 1-6

In this paper we understand the pros and cons of scaling up test-time compute, and how it compares with scaling up pre-training compute. While the simplest approach for scaling test-time compute is best-of-N sampling - sampling N outputs in “parallel” from a base LLM and selecting the one that scores the highest per a learned verifier or a reward model [...].

✔ 문장 1-6-1

In this paper we understand the pros and cons of scaling up test-time compute, and how it compares with scaling up pre-training compute.

🔍 단위 해석

본 논문에서 우리는 테스트 시점 연산 확장의 장단점을 이해하고, 이것이 사전 학습(pre-training) 연산 확장과 비교했을 때 어떠한지를 파악합니다.

✔ 문장 1-6-2

While the simplest approach for scaling test-time compute is best-of-N sampling - sampling N outputs in “parallel” from a base LLM and selecting the one that scores the highest per a learned verifier or a reward model (Cobbe et al., 2021; Lightman et al., 2023) - there are many other ways we could conceivably scale up test-time compute.

🔍 단위 해석

테스트 시점 연산을 확장하는 가장 간단한 방법은 'Best-of-N 샘플링'(기본 LLM에서 N개의 출력을 '병렬'로 샘플링하고 학습된 검증기나 보상 모델을 통해 가장 높은 점수를 받은 것을 선택하는 방식)이지만, 그 외에도 테스트 시점 연산을 확장할 수 있는 방법은 많습니다.

📖 단락 1-7

We unify methods into those that modify either the proposal distribution from which responses are sampled (for e.g., by asking the base model to revise its responses (Qu et al., 2024b)) or those that alter how the verifier is used for searching directly in the output space (e.g. by training a PRM (Lightman et al., 2023)).

✔ 문장 1-7-1

We unify methods into those that modify either the proposal distribution from which responses are sampled (for e.g., by asking the base model to revise its responses (Qu et al., 2024b)) or those that alter how the verifier is used for searching directly in the output space (e.g. by training a PRM (Lightman et al., 2023)).

🔍 단위 해석

우리는 방법론들을 크게 두 가지로 통합합니다: (1) 응답이 샘플링되는 '제안 분포(proposal distribution)'를 수정하는 방법(예: 기본 모델에게 응답을 수정하도록 요청)과 (2) 출력 공간에서 직접 탐색(search)하기 위해 검증기(verifier)를 사용하는 방식을 변경하는 방법(예: PRM 학습)입니다.

💡 보조 설명

- **[Proposal distribution]:** 모델이 답을 내놓을 때의 확률 분포입니다. 모델이 스스로 답을 고치게(Revision) 하면 정답이 나올 확률 분포가 바뀌게 되므로, 이를 '제안 분포 수정'이라고 표현합니다.
- **[Output space search]:** 생성된 답안들 사이를 탐색하는 것입니다. PRM(과정 보상 모델)을 쓰면 단순히 답만 보는 게 아니라 풀이 과정(트리 구조)을 탐색할 수 있습니다.

🖼️ 시각 자료 [Figure 1]

(원문 캡션: **Figure 1:** Summary of results. Left: Compute-optimal scaling for revisions and search. We compare the compute-optimal scaling policy with PaLM 2-S* against baselines... Right: Comparing test-time compute and parameter scaling...)

🔍 캡션 해석

결과 요약. 왼쪽: 수정(revision)과 탐색(search)을 위한 연산 최적화(compute-optimal) 스케일링. PaLM 2-S* 모델을 사용해 연산 최적화 정책과 베이스라인을 비교합니다. 오른쪽: 테스트 시점 연산과 파라미터 스케일링의 비교. 테스트 시점 연산을 사용한 모델과 이를 사용하지 않은 약 14배 더 큰 모델을 비교합니다.

📄 PDF 시각 분석

PDF 2페이지의 **Figure 1**을 분석한 결과입니다:

- 좌측 상단 (**Revisions**): 'Compute Optimal' 곡선이 'Parallel' (단순 Best-of-N)보다 가파르게 상승하며 더 적은 예산으로 높은 정확도에 도달합니다.
- 좌측 하단 (**PRM Search**): 역시 'Compute Optimal Search'가 'Majority'나 일반 'Best-of-N'보다 초기 구간에서 훨씬 효율적으로 성능을 높입니다.
- 우측 상단/하단 (**FLOPs Matched Evaluation**):
 - **Easy/Medium Questions (녹색/파란색):** 막대가 양수(+)입니다. 즉, 작은 모델에 테스트 타임 연산을 추가하는 것이 큰 모델을 쓰는 것보다 낫습니다.
 - **Hard Questions (주황색):** 막대가 음수(-)입니다. 즉, 어려운 문제에서는 테스트 타임 연산보다 모델 크기(사전 학습)를 키우는 것이 더 유리합니다.

2 UNIFIED PERSPECTIVE ON TEST-TIME COMPUTE: PROPOSER & VERIFIER

📖 단락 2-1

We first provide an unified abstraction of test-time compute to situate contemporary approaches. We view the use of test-time compute through the lens of modifying the model’s distribution on a given prompt, adaptively at test-time. Ideally, test-time compute should allow for the ability to express more complex distributions than naïvely sampling from the LLM. In general, there are two knobs to modify an LLM’s distribution: (1) at the input level: by augmenting the given prompt with an additional set of tokens that the LLM conditions on to obtain a modified proposal distribution, or (2) at the output level: by sampling multiple candidates from the standard LLM and performing surgery on them, using some post-hoc verifiers or scorers. This process is reminiscent of Markov chain Monte Carlo (MCMC) (Andrieu et al., 2003) sampling from a complex distribution by combining a simple proposal distribution and a score function. Modifying the proposal distribution by altering inputs tokens and using a verifier form the two independent axes of our study.

✅ 문장 2-1-1

We first provide an unified abstraction of test-time compute to situate contemporary approaches.

🔍 단위 해석

우리는 먼저 최신 접근 방식들의 위치를 정립하기 위해, 테스트 시점 연산에 대한 통합된 추상화(unified abstraction)를 제공합니다.

✅ 문장 2-1-2

We view the use of test-time compute through the lens of modifying the model’s distribution on a given prompt, adaptively at test-time.

🔍 단위 해석

우리는 테스트 시점 연산의 사용을, 주어진 프롬프트에 대한 모델의 분포를 테스트 시점에 적응적으로(adaptively) 수정하는 관점에서 바라봅니다.

✅ 문장 2-1-3

Ideally, test-time compute should allow for the ability to express more complex distributions than naïvely sampling from the LLM.

🔍 단위 해석

이상적으로, 테스트 시점 연산은 단순히 LLM에서 샘플링하는 것보다 더 복잡한 분포를 표현할 수 있는 능력을 허용해야 합니다.

✅ 문장 2-1-4

In general, there are two knobs to modify an LLM’s distribution: (1) at the input level: by augmenting the given prompt with an additional set of tokens that the LLM conditions on to obtain a modified proposal distribution, or (2) at the output level: by sampling multiple candidates from the standard LLM and performing surgery on them, using some post-hoc verifiers or scorers.

🔍 단위 해석

일반적으로 LLM의 분포를 수정하는 데는 두 가지 조절 장치(knobs)가 있습니다: (1) 입력 레벨: LLM이 조건부로 삼을 추가 토큰 세트로 프롬프트를 증강하여 수정된 제안 분포(proposal distribution)를 얻는 방법, 혹은 (2) 출력 레벨: 표준 LLM에서 여러 후보를 샘플링한 뒤 사후 검증기(post-hoc verifiers)나 채점기(scorers)를 사용하여 이들을 수술(조작/선별)하는 방법입니다.

✅ 문장 2-1-5

This process is reminiscent of Markov chain Monte Carlo (MCMC) (Andrieu et al., 2003) sampling from a complex distribution by combining a simple proposal distribution and a score function.

🔍 단위 해석

이 과정은 단순한 제안 분포와 점수 함수(score function)를 결합하여 복잡한 분포에서 샘플링하는 마르코프 체인 몬테카를로(MCMC) 방식을 연상시킵니다.

📖 단락 2-2

(1) Modifying the proposal distribution. One way to improve the proposal distribution is to directly optimize the model for a given reasoning task via RL-inspired finetuning methods such as STaR or $ReST^{EM}$ (Zelikman et al., 2022; Singh et al., 2024). These techniques specifically finetune the model to directly improve the proposal distribution, rather than generating additional tokens at test-time. Instead, techniques such as self-critique (Bai et al., 2022; Madaan et al., 2023; Du et al., 2023; Saunders et al., 2022) enable the model to improve its own proposals at test time by instructing it to critique and revise its outputs iteratively. Since prompting off-the-shelf models is not effective at enabling effective revisions at test time, we specifically finetune models to iteratively revise their answers for complex reasoning, using Best-of-N guidance (Qu et al., 2024b; Kumar et al., 2024).

✔ 문장 2-2-1

(1) Modifying the proposal distribution.

🔍 단위 해석

(1) 제안 분포(Proposal Distribution) 수정하기.

✔ 문장 2-2-2

One way to improve the proposal distribution is to directly optimize the model for a given reasoning task via RL-inspired finetuning methods such as STaR or $ReST^{EM}$ (Zelikman et al., 2022; Singh et al., 2024).

🔍 단위 해석

제안 분포를 개선하는 한 가지 방법은 STaR나 $ReST^{EM}$ 과 같은 강화학습(RL) 영감을 받은 미세 조정(finetuning) 방법을 통해 주어진 추론 작업에 대해 모델을 직접 최적화하는 것입니다.

✔ 문장 2-2-3

These techniques specifically finetune the model to directly improve the proposal distribution, rather than generating additional tokens at test-time.

🔍 단위 해석

이러한 기법들은 테스트 시점에 추가 토큰을 생성하기보다는, 모델 자체를 미세 조정하여 제안 분포를 직접적으로 개선하는 데 특화되어 있습니다.

✔ 문장 2-2-4

Instead, techniques such as self-critique (Bai et al., 2022; Madaan et al., 2023; Du et al., 2023; Saunders et al., 2022) enable the model to improve its own proposals at test time by instructing it to critique and revise its outputs iteratively.

🔍 단위 해석

반면, 자기 비평(self-critique)과 같은 기법들은 모델에게 출력을 반복적으로 비평하고 수정하도록 지시함으로써, 테스트 시점에 자신의 제안을 스스로 개선할 수 있게 합니다.

✔ 문장 2-2-5

Since prompting off-the-shelf models is not effective at enabling effective revisions at test time, we specifically finetune models to iteratively revise their answers for complex reasoning, using Best-of-N guidance (Qu et al., 2024b; Kumar et al., 2024).

🔍 단위 해석

상용(off-the-shelf) 모델에 단순히 프롬프트만 입력하는 것은 테스트 시점의 효과적인 수정(revision)을 이끌어내기에 부족하므로, 우리는 Best-of-N 가이드를 사용하여 복잡한 추론을 위해 답을 반복적으로 수정하도록 모델을 특별히 미세 조정합니다.

📖 단락 2-3

(2) Optimizing the verifier. The verifier selects the best answer from the proposal distribution. The most canonical way to use such a verifier is by applying best-of-N sampling, wherein we sample N solutions and then select the best one with a verifier (Cobbe et al., 2021). This approach can be further improved by training a process-based reward model (PRM) (Lightman et al., 2023), which produces a prediction of the correctness of each intermediate step in a solution. We can then utilize these per-step predictions to perform tree search over the solution space, enabling a more effective modification of the proposal distribution (Yao et al., 2023; Feng et al., 2024; Chen et al., 2024).

✔ 문장 2-3-1

(2) Optimizing the verifier.

🔍 단위 해석

(2) 검증기(Verifier) 최적화하기.

✔ 문장 2-3-2

The verifier selects the best answer from the proposal distribution.

🔍 단위 해석

검증기는 제안 분포에서 나온 답안 중 가장 좋은 것을 선택합니다.

✔ 문장 2-3-3

The most canonical way to use such a verifier is by applying best-of-N sampling, wherein we sample N solutions and then select the best one with a verifier (Cobbe et al., 2021).

단위 해석

이러한 검증기를 사용하는 가장 전형적인(canonical) 방법은 Best-of-N 샘플링을 적용하는 것인데, 이는 N개의 해답을 샘플링한 후 검증기를 이용해 그중 최고를 선택하는 방식입니다.

문장 2-3-4

This approach can be further improved by training a process-based reward model (PRM) (Lightman et al., 2023), which produces a prediction of the correctness of each intermediate step in a solution.

단위 해석

이 접근법은 과정 중심 보상 모델(PRM)을 훈련함으로써 더욱 개선될 수 있는데, PRM은 풀이 과정의 각 중간 단계(intermediate step)에 대해 정답 여부(correctness)를 예측해 줍니다.

문장 2-3-5

We can then utilize these per-step predictions to perform tree search over the solution space, enabling a more effective modification of the proposal distribution (Yao et al., 2023; Feng et al., 2024; Chen et al., 2024).

단위 해석

우리는 이 단계별 예측값을 활용하여 해답 공간(solution space)에서 트리 탐색(tree search)을 수행할 수 있으며, 이를 통해 제안 분포를 보다 효과적으로 수정할 수 있게 됩니다.

3 HOW TO SCALE TEST-TIME COMPUTATION OPTIMALLY

단락 3-1

Using this unified view of different methods, we would like to understand and characterize how to most effectively use test-time computation to improve performance on a given prompt by answering the question below. When either refining the proposal distribution or searching against a verifier, there are numerous choices on how to allocate test-time compute. For example, when using a model finetuned for revisions as the proposal distribution and an ORM verifier, we could either spend the full test-time budget on generating N independent samples in parallel from the model and then apply best-of-N, or we could sample N revisions in sequence using a revision model and then select the best answer in the sequence with an ORM, or strike a balance between these extremes. Intuitively, we might expect that problems where the initial samples are more likely to be on the right track to benefit more from revisions. On the other hand, problems that require exploration over high-level problem solving strategies might benefit from sampling more independent answers in parallel. Finally, in the case of verifiers, we also can choose between different search algorithms (e.g. beam-search, lookahead-search, best-of-N), each of which may exhibit different properties depending on the quality of the verifier and proposal distribution at hand. More sophisticated search procedures might be more useful in harder problems compared to a much simpler best-of-N or majority baseline.

문장 3-1-1

Using this unified view of different methods, we would like to understand and characterize how to most effectively use test-time computation to improve performance on a given prompt by answering the question below.

단위 해석

다양한 방법론에 대한 이 통합된 관점을 바탕으로, 우리는 아래 질문에 답함으로써 주어진 프롬프트의 성능을 높이기 위해 테스트 시점 연산을 가장 효과적으로 사용하는 방법을 이해하고 규명하고자 합니다.

문장 3-1-2

When either refining the proposal distribution or searching against a verifier, there are numerous choices on how to allocate test-time compute.

단위 해석

제안 분포를 정제하든 검증기를 통해 탐색하든, 테스트 시점 연산 자원을 어떻게 할당할지에 대해서는 수많은 선택지가 존재합니다.

문장 3-1-3

For example, when using a model finetuned for revisions as the proposal distribution and an ORM verifier, we could either spend the full test-time budget on generating N independent samples in parallel from the model and then apply best-of-N, or we could sample N revisions in sequence using a revision model and then select the best answer in the sequence with an ORM, or strike a balance between these extremes.

단위 해석

예를 들어, 수정(revision)을 위해 미세 조정된 모델을 제안 분포로 사용하고 ORM(결과 중심 보상 모델)을 검증기로 사용할 때, 우리는 전체 연산 예산을 (1) 모델에서 N개의 독립적인 샘플을 병렬로 생성한 뒤 Best-of-N을 적용하는 데 쓸 수도 있고, (2) 수정 모델을 사용해 순차적으로 N번의 수정을 거친 뒤 ORM으로 시퀀스 내 최고의 답을 고르는 데 쓸 수도 있으며, (3) 이 두 극단 사이에서 균형을 맞출 수도 있습니다.

- [ORM (Outcome Reward Model)]: 과정(Process)이 아닌 최종 결과(Outcome)만 보고 점수를 매기는 검증 모델입니다.

✅ 문장 3-1-4

Intuitively, we might expect that problems where the initial samples are more likely to be on the right track to benefit more from revisions.

🔍 단위 해석

직관적으로, 초기 샘플이 정답에 가까울 가능성이 높은 문제들은 수정(revision)을 통해 더 많은 이득을 볼 것이라 예상할 수 있습니다.

✅ 문장 3-1-5

On the other hand, problems that require exploration over high-level problem solving strategies might benefit from sampling more independent answers in parallel.

🔍 단위 해석

반면에, 고차원적인 문제 해결 전략을 탐색해야 하는 문제들은 독립적인 답안을 병렬로 더 많이 샘플링하는 것이 유리할 수 있습니다.

📖 단락 3-2

We are given a prompt and a test-time compute budget within which to solve the problem. Under the abstraction above, there are different knobs we can tune when utilizing test-time computation. How can we determine the most effective way to utilize test-time compute for a given prompt? And how well would this do against simply utilizing a much bigger pretrained model?

✅ 문장 3-2-1

We are given a prompt and a test-time compute budget within which to solve the problem.

🔍 단위 해석

우리에게는 문제를 해결하기 위한 프롬프트와 테스트 시점 연산 예산(budget)이 주어집니다.

✅ 문장 3-2-2

How can we determine the most effective way to utilize test-time compute for a given prompt?

🔍 단위 해석

주어진 프롬프트에 대해 테스트 시점 연산을 활용하는 가장 효과적인 방법을 어떻게 결정할 수 있을까요?

✅ 문장 3-2-3

And how well would this do against simply utilizing a much bigger pretrained model?

🔍 단위 해석

그리고 이것은 단순히 훨씬 더 큰 사전 학습 모델을 사용하는 것과 비교했을 때 얼마나 좋은 성능을 낼까요?

3.1 COMPUTE-OPTIMAL TEST-TIME SCALING STRATEGY

📖 단락 3.1-1

Per the discussion above, we would like to prescribe the optimal allocation of our test-time compute budget onto a given problem. To this end, for any given approach of utilizing test-time compute (e.g., revisions and search against a verifier in this paper, some combination or other methods in general), we define the “test-time compute-optimal scaling strategy” as the strategy that chooses hyperparameters appearing in a given approach for maximal performance benefits on a given prompt at test time.

✅ 문장 3.1-1-1

Per the discussion above, we would like to prescribe the optimal allocation of our test-time compute budget onto a given problem.

🔍 단위 해석

앞서의 논의에 따라, 우리는 주어진 문제에 대해 테스트 시점 연산 예산을 최적으로 할당하는 방안을 제시하고자 합니다.

✅ 문장 3.1-1-2

To this end, for any given approach of utilizing test-time compute (e.g., revisions and search against a verifier in this paper, some combination or other methods in general), we define the “test-time compute-optimal scaling strategy” as the strategy that chooses hyperparameters appearing in a given approach for maximal performance benefits on a given prompt at test time.

단위 해석

이를 위해, 테스트 시점 연산을 활용하는 모든 접근 방식(본 논문의 수정 및 검증기 탐색, 혹은 일반적인 다른 방법들의 조합 등)에 대해, 우리는 "테스트 시점 연산 최적화 스케일링 전략(test-time compute-optimal scaling strategy)"을 다음과 같이 정의합니다: 주어진 프롬프트에 대해 테스트 시점에 최대의 성능 이득을 얻을 수 있도록 해당 접근 방식의 하이퍼파라미터를 선택하는 전략.

단락 3.1-2

Formally, define $Target(\theta, N, q)$ as the distribution over natural language output tokens induced by the model for a given prompt q , using test-time compute hyper-parameters θ , and a compute budget of N . We would like to select the hyper-parameters θ which maximize the accuracy of the target distribution for a given problem. We express this formally as:

$$\theta_{q,y^*(q)}^*(N) = \operatorname{argmax}_{\theta}(\mathbb{E}_{y \sim Target(\theta, N, q)}[\mathbb{I}_{y=y^*(q)}]),$$

(1)

where $y^*(q)$ denotes the ground-truth correct response for input query q , and $\theta_{q,y^*(q)}^*(N)$ represents the test-time compute-optimal scaling strategy for the problem q with compute budget N . We note that our definition of test-time compute-optimal scaling differs slightly from that of concurrent work (Wu et al., 2024) in that our notion of scaling is question dependent.

문장 3.1-2-1

Formally, define $Target(\theta, N, q)$ as the distribution over natural language output tokens induced by the model for a given prompt q , using test-time compute hyper-parameters θ , and a compute budget of N .

단위 해석

공식적으로, 주어진 프롬프트 q , 테스트 시점 연산 하이퍼파라미터 θ , 그리고 연산 예산 N 을 사용하여 모델이 유도해 낸 자연어 출력 토큰 분포를 $Target(\theta, N, q)$ 라고 정의합니다.

문장 3.1-2-2

We would like to select the hyper-parameters θ which maximize the accuracy of the target distribution for a given problem.

단위 해석

우리는 주어진 문제에 대해 타겟 분포의 정확도를 최대화하는 하이퍼파라미터 θ 를 선택하고자 합니다.

문장 3.1-2-3

We express this formally as:

$$\theta_{q,y^*(q)}^*(N) = \operatorname{argmax}_{\theta}(\mathbb{E}_{y \sim Target(\theta, N, q)}[\mathbb{I}_{y=y^*(q)}])$$

단위 해석

이를 수식으로 표현하면 위와 같습니다. (즉, 정답 y 가 나올 기댓값을 최대화하는 θ 를 찾는 것)

문장 3.1-2-4

where $y^*(q)$ denotes the ground-truth correct response for input query q , and $\theta_{q,y^*(q)}^*(N)$ represents the test-time compute-optimal scaling strategy for the problem q with compute budget N .

단위 해석

여기서 $y^*(q)$ 는 입력 쿼리 q 에 대한 실제 정답(ground-truth)을 의미하며, $\theta_{q,y^*(q)}^*(N)$ 는 연산 예산 N 하에서 문제 q 에 대한 테스트 시점 연산 최적화 스케일링 전략을 나타냅니다.

문장 3.1-2-5

We note that our definition of test-time compute-optimal scaling differs slightly from that of concurrent work (Wu et al., 2024) in that our notion of scaling is question dependent.

단위 해석

우리의 테스트 시점 연산 최적화 스케일링 정의는 동시대 연구(Wu et al., 2024)와 약간 다른데, 우리의 스케일링 개념은 '질문(문제)에 의존적(question dependent)'이라는 점이 특징입니다.

3.2 QUESTION DIFFICULTY IS A GOOD APPROXIMATION FOR THE OPTIMAL STRATEGY

📖 단락 3.2-1

In order to effectively analyze the test-time scaling properties of the different mechanisms discussed in Section 2 (e.g. proposal distribution and verifier), we will prescribe an approximation to this optimal strategy $\theta_{q,y^*(q)}^*(N)$ as a function of a statistic of a given prompt. Our approximation estimates a notion of difficulty for a given prompt. The compute-optimal strategy is then defined as a function of the difficulty of a prompt. Despite being only an heuristic approach to solve Equation 1, we find that it can still induce substantial improvements in performance over a baseline strategy of allocating this inference-time compute in an ad-hoc manner.

✅ 문장 3.2-1-1

In order to effectively analyze the test-time scaling properties of the different mechanisms discussed in Section 2 (e.g. proposal distribution and verifier), we will prescribe an approximation to this optimal strategy $\theta_{q,y^*(q)}^*(N)$ as a function of a statistic of a given prompt.

🔍 단위 해석

섹션 2에서 논의된 다양한 메커니즘(예: 제안 분포 및 검증기)의 테스트 시점 스케일링 특성을 효과적으로 분석하기 위해, 우리는 이 최적 전략 $\theta_{q,y^*(q)}^*(N)$ 에 대한 근사치를 주어진 프롬프트의 통계량에 대한 함수로 규정할 것입니다.

✅ 문장 3.2-1-2

Our approximation estimates a notion of difficulty for a given prompt.

🔍 단위 해석

우리의 근사법은 주어진 프롬프트에 대한 ‘난이도(difficulty)’ 개념을 추정합니다.

✅ 문장 3.2-1-3

The compute-optimal strategy is then defined as a function of the difficulty of a prompt.

🔍 단위 해석

그다음 연산 최적화 전략은 이 프롬프트 난이도의 함수로 정의됩니다.

✅ 문장 3.2-1-4

Despite being only an heuristic approach to solve Equation 1, we find that it can still induce substantial improvements in performance over a baseline strategy of allocating this inference-time compute in an ad-hoc manner.

🔍 단위 해석

이것이 식(1)을 풀기 위한 휴리스틱(어림짐작) 접근법에 불과함에도 불구하고, 우리는 이것이 추론 시점 연산을 임의로(ad-hoc) 할당하는 베이스라인 전략에 비해 상당한 성능 향상을 이끌어낼 수 있음을 발견했습니다.

📖 단락 3.2-2

Our estimate of question difficulty assigns a given question to one of five discrete difficulty levels. We then use these bins to estimate $\theta_{q,y^*(q)}^*(N)$ on a validation set (given a compute budget), and apply the optimal strategy on the test set. Thus, question difficulty acts as a sufficient statistic for designing the compute-optimal strategy. For example, to optimally allocate test-time compute between parallel best-of-N and sequential sampling, we first pre-compute the accuracy of both techniques within each difficulty bin using a held-out set. Given a new test question, we then determine the difficulty bin it belongs to and select the best performing strategy within that bin.

✅ 문장 3.2-2-1

Our estimate of question difficulty assigns a given question to one of five discrete difficulty levels.

🔍 단위 해석

우리의 문제 난이도 추정 방식은 주어진 문제를 5개의 이산적인(discrete) 난이도 레벨 중 하나로 할당합니다.

✅ 문장 3.2-2-2

We then use these bins to estimate $\theta_{q,y^*(q)}^*(N)$ on a validation set (given a compute budget), and apply the optimal strategy on the test set.

🔍 단위 해석

그런 다음 우리는 검증 세트(validation set)에서 (주어진 연산 예산하에) 이 난이도 구간(bins)들을 사용하여 $\theta_{q,y^*(q)}^*(N)$ 를 추정하고, 그 최적 전략을 테스트 세트

에 적용합니다.

✅ **문장 3.2-2-3**

Thus, question difficulty acts as a sufficient statistic for designing the compute-optimal strategy.

🔍 **단위 해석**

따라서 문제 난이도는 연산 최적화 전략을 설계하기 위한 충분통계량(sufficient statistic) 역할을 합니다.

✅ **문장 3.2-2-4**

For example, to optimally allocate test-time compute between parallel best-of-N and sequential sampling, we first pre-compute the accuracy of both techniques within each difficulty bin using a held-out set.

🔍 **단위 해석**

예를 들어, 병렬 Best-of-N과 순차적 샘플링 간에 테스트 시점 연산을 최적으로 할당하기 위해, 우리는 먼저 보류된(held-out) 세트를 사용하여 각 난이도 구간 내에서 두 기법의 정확도를 미리 계산합니다.

✅ **문장 3.2-2-5**

Given a new test question, we then determine the difficulty bin it belongs to and select the best performing strategy within that bin.

🔍 **단위 해석**

새로운 테스트 문제가 주어지면, 우리는 그것이 속한 난이도 구간을 결정하고 해당 구간에서 가장 성능이 좋았던 전략을 선택합니다.

📖 **단락 3.2-3**

Defining question difficulty. Following Lightman et al. (2023), we define question difficulty as a function of the given base LLM. Specifically, we bin the model’s pass@1 rate estimated from 2048 samples on each test question into five quantiles, each corresponding to increasing difficulty levels. We find this notion of model-specific difficulty bins to be more predictive of the efficacy of using test-time compute compared to the hand-labeled difficulty bins in the MATH dataset. That said, we note that assessing difficulty as described assumes oracle access to a correctness checker, which is unavailable at deployment. To enable a realistic estimate of difficulty, we approximate difficulty via a model-predicted notion of difficulty, which constructs the bins by averaging the score of a learned verifier on the same 2048 samples per problem. We refer to this setting as model-predicted difficulty and the setting which relies on ground-truth correctness as oracle difficulty. Predicted difficulty removes the reliance on ground truth labels, but still incurs computational cost. Our experiments do not account for this cost largely for simplicity, since our goal is to present some of the first results of what is in fact possible by effectively allocating test-time compute.

✅ **문장 3.2-3-1**

Defining question difficulty. Following Lightman et al. (2023), we define question difficulty as a function of the given base LLM.

🔍 **단위 해석**

문제 난이도의 정의. Lightman et al. (2023)을 따르며, 우리는 문제 난이도를 주어진 기본 LLM에 대한 함수로 정의합니다. (즉, 모델이 어려워하면 어려운 문제임)

✅ **문장 3.2-3-2**

Specifically, we bin the model’s pass@1 rate estimated from 2048 samples on each test question into five quantiles, each corresponding to increasing difficulty levels.

🔍 **단위 해석**

구체적으로, 각 테스트 문제에 대해 2048개의 샘플로부터 추정된 모델의 pass@1(한 번에 맞출 확률) 비율을 5개의 분위수(quantiles)로 나누어 구간화(binning)하며, 각 구간은 증가하는 난이도 레벨에 해당합니다.

✅ **문장 3.2-3-3**

We find this notion of model-specific difficulty bins to be more predictive of the efficacy of using test-time compute compared to the hand-labeled difficulty bins in the MATH dataset.

🔍 **단위 해석**

우리는 이러한 ‘모델 맞춤형 난이도 구간’ 개념이 MATH 데이터셋에 있는 사람이 라벨링한 난이도 구간보다 테스트 시점 연산의 효용성을 더 잘 예측한다는 것을 발견했습니다.

✅ **문장 3.2-3-4**

That said, we note that assessing difficulty as described assumes oracle access to a correctness checker, which is unavailable at deployment.

단위 해석

다만, 위와 같이 난이도를 평가하는 것은 배포 시점에는 사용할 수 없는 정답 판별기(correctness checker)에 대한 오라클(oracle, 정답을 아는 존재) 접근 권한을 가정한다는 점을 언급합니다.

문장 3.2-3-5

To enable a realistic estimate of difficulty, we approximate difficulty via a model-predicted notion of difficulty, which constructs the bins by averaging the score of a learned verifier on the same 2048 samples per problem.

단위 해석

현실적인 난이도 추정을 가능하게 하기 위해, 우리는 '모델 예측 난이도(model-predicted difficulty)'라는 개념을 통해 난이도를 근사하는데, 이는 문제당 동일한 2048개 샘플에 대해 학습된 검증기(learned verifier)의 점수 평균을 내어 구간을 구성하는 방식입니다.

문장 3.2-3-6

We refer to this setting as model-predicted difficulty and the setting which relies on ground-truth correctness as oracle difficulty.

단위 해석

우리는 이 설정을 '모델 예측 난이도'라고 부르며, 실제 정답(ground-truth)에 의존하는 설정을 '오라클 난이도'라고 부릅니다.

- 보조 설명
- [Oracle Difficulty]:** 실제 정답을 알고 있는 상태에서 "이 문제는 우리 모델이 20% 확률로 맞추더라"라고 정확히 난이도를 아는 경우 (이상적인 상황).
 - [Predicted Difficulty]:** 정답은 모르지만, 검증기(Verifier)가 "이 문제는 점수가 낮게 나오는 걸 보니 어려운 문제 같다"라고 추측하는 경우 (현실적인 상황).

문장 3.2-3-7

Predicted difficulty removes the reliance on ground truth labels, but still incurs computational cost.

단위 해석

예측된 난이도는 실제 정답 라벨에 대한 의존성을 제거하지만, 여전히 연산 비용이 발생합니다.

문장 3.2-3-8

Our experiments do not account for this cost largely for simplicity, since our goal is to present some of the first results of what is in fact possible by effectively allocating test-time compute.

단위 해석

우리의 실험에서는 주로 단순함을 위해 이 비용(난이도 예측 비용)을 고려하지 않았는데, 이는 우리의 목표가 테스트 시점 연산을 효과적으로 할당함으로써 무엇이 가능한지에 대한 초기 결과를 제시하는 것이기 때문입니다.

4 EXPERIMENTAL SETUP

단락 4-1

We first outline our experimental setup for conducting this analysis with multiple verifier design choices and proposal distributions, followed by the analysis results in the subsequent sections.

문장 4-1-1

We first outline our experimental setup for conducting this analysis with multiple verifier design choices and proposal distributions, followed by the analysis results in the subsequent sections.

단위 해석

우리는 먼저 다양한 검증기(verifier) 설계 방식과 제안 분포(proposal distribution)를 사용한 본 분석의 실험 설정을 개괄하고, 이어서 후속 섹션에서 분석 결과를 제시합니다.

단락 4-2

Datasets. We expect test-time compute to be most helpful when models already have all the basic “knowledge” needed to answer a query, and instead the primary challenge is about drawing (complex) inferences from this knowledge. To this end, we focus on the MATH (Hendrycks et al., 2021) benchmark, which consists of high-school competition level math problems with a range of difficulty levels. For all experiments, we use the dataset split consisting of 12k train and 500 test questions.

✔ 문장 4-2-1

Datasets. We expect test-time compute to be most helpful when models already have all the basic “knowledge” needed to answer a query, and instead the primary challenge is about drawing (complex) inferences from this knowledge.

🔍 단위 해석

데이터셋. 우리는 모델이 질문에 답하는 데 필요한 기본 "지식"은 이미 갖추고 있으나, 그 지식으로부터 (복잡한) 추론을 이끌어내는 것이 주된 과제일 때 테스트 시점 연산이 가장 도움이 될 것이라 예상합니다.

✔ 문장 4-2-2

To this end, we focus on the MATH (Hendrycks et al., 2021) benchmark, which consists of high-school competition level math problems with a range of difficulty levels.

🔍 단위 해석

이를 위해, 우리는 다양한 난이도의 고등학교 경시대회 수준 수학 문제들로 구성된 MATH 벤치마크에 집중합니다.

✔ 문장 4-2-3

For all experiments, we use the dataset split consisting of 12k train and 500 test questions.

🔍 단위 해석

모든 실험에서 우리는 12,000개의 훈련 문항과 500개의 테스트 문항으로 구성된 데이터 분할(split)을 사용합니다.

📖 단락 4-3

Models. We use the PaLM 2-S* (Anil et al., 2023) (Codey) model. We chose this model, as it is representative of the capabilities of many contemporary LLMs, and is small enough to efficiently run many experiments on. Most importantly, this model attains a non-trivial performance on MATH (but not saturated). For these reasons, we expect this model to provide a good test-bed.

✔ 문장 4-3-1

Models. We use the PaLM 2-S* (Anil et al., 2023) (Codey) model.

🔍 단위 해석

모델. 우리는 PaLM 2-S* (Codey) 모델을 사용합니다.

✔ 문장 4-3-2

We chose this model, as it is representative of the capabilities of many contemporary LLMs, and is small enough to efficiently run many experiments on.

🔍 단위 해석

우리는 이 모델이 최신 LLM들의 능력을 대표하면서도, 효율적으로 다수의 실험을 수행하기에 충분히 작기 때문에 선택했습니다.

✔ 문장 4-3-3

Most importantly, this model attains a non-trivial performance on MATH (but not saturated).

🔍 단위 해석

가장 중요한 점은, 이 모델이 MATH 벤치마크에서 (성능이 포화 상태는 아니지만) 무시할 수 없는(non-trivial) 수준의 성능을 달성한다는 것입니다.

💡 보조 설명

- **[Non-trivial but not saturated]:** 너무 못해서 실험이 불가능하지도 않고, 이미 다 맞혀서 개선의 여지가 없지도 않은, 실험하기 딱 적절한 성능 구간을 의미합니다.

✔ 문장 4-3-4

For these reasons, we expect this model to provide a good test-bed.

🔍 단위 해석

이러한 이유들로 인해, 우리는 이 모델이 좋은 테스트베드(실험 환경)를 제공할 것이라 기대합니다.

5 SCALING TEST-TIME COMPUTE VIA VERIFIERS

단락 5-1

In this section, we study how test-time compute can be most effectively scaled by searching against a verifier and keeping the proposal distribution fixed to the base LM. Specifically, we study different search approaches with PRMs and analyze their test-time compute scaling properties, but first we provide a brief overview of how a PRM can be trained.

문장 5-1-1

In this section, we study how test-time compute can be most effectively scaled by searching against a verifier and keeping the proposal distribution fixed to the base LM.

단위 해석

이 섹션에서는 제안 분포를 기본 언어 모델(base LM)로 고정한 상태에서, 검증기를 활용한 탐색(search against a verifier)을 통해 테스트 시점 연산을 가장 효과적으로 확장하는 방법을 연구합니다.

문장 5-1-2

Specifically, we study different search approaches with PRMs and analyze their test-time compute scaling properties, but first we provide a brief overview of how a PRM can be trained.

단위 해석

구체적으로, PRM(과정 보상 모델)을 활용한 다양한 탐색 접근법을 연구하고 그들의 테스트 시점 연산 스케일링 특성을 분석하며, 이에 앞서 PRM이 어떻게 훈련되는 지에 대한 간략한 개요를 제공합니다.

5.1 TRAINING VERIFIERS AMENABLE TO SEARCH

단락 5.1-1

We follow the approach of Wang et al. (2023), which supervises the PRM using estimates of per-step correctness obtained from running Monte Carlo rollouts from each step in the solution. Our PRM’s per-step predictions therefore correspond to value estimates of reward-to-go for the base model’s sampling policy, similar to recent work (Wang et al., 2023; Setlur et al., 2024). We also compared to an ORM baseline (Appendix H) but found that our PRM consistently outperforms the ORM. Hence, all of the search experiments in this section use a PRM model. Additional details are in Appendix F.

문장 5.1-1-1

We follow the approach of Wang et al. (2023), which supervises the PRM using estimates of per-step correctness obtained from running Monte Carlo rollouts from each step in the solution.

단위 해석

우리는 Wang et al. (2023)의 접근 방식을 따르는데, 이는 해답의 각 단계(step)에서 몬테카를로 롤아웃(Monte Carlo rollouts)을 실행하여 얻은 단계별 정답 여부 추정치를 사용하여 PRM을 지도 학습시키는 방식입니다.

보조 설명

- **[Monte Carlo rollout]:** 현재 단계에서 끝까지 여러 번 답을 생성해 보고, 최종적으로 정답에 도달하는 비율을 계산하여 현재 단계의 가치를 평가하는 방법입니다.

문장 5.1-1-2

Our PRM’s per-step predictions therefore correspond to value estimates of reward-to-go for the base model’s sampling policy, similar to recent work (Wang et al., 2023; Setlur et al., 2024).

단위 해석

따라서 우리 PRM의 단계별 예측값은 최근 연구들과 유사하게, 기본 모델의 샘플링 정책에 대한 '보상 잔여값(reward-to-go)'의 가치 추정치에 해당합니다.

문장 5.1-1-3

We also compared to an ORM baseline (Appendix H) but found that our PRM consistently outperforms the ORM.

단위 해석

우리는 또한 ORM 베이스라인과도 비교했으나(부록 H), PRM이 일관되게 ORM보다 성능이 우수함을 확인했습니다.

✔ 문장 5.1-1-4

Hence, all of the search experiments in this section use a PRM model.

🔍 단위 해석

따라서, 이 섹션의 모든 탐색 실험은 PRM 모델을 사용합니다.

📖 단락 5.1-2

Answer aggregation. At test time, PRMs can be used to score each individual step appearing in a set of solutions sampled from the base model. To pick out the best answer from N samples with the PRM, we need a function that can aggregate across all the per-step scores for each answer to determine the best candidate for the correct answer. To do this, we take the PRM’s prediction at the last step as representative of the full-answer score and then follow Li et al. (2023) by applying “best-of-N weighted” selection across answers. We include more detail on these decisions in Appendix G.

✔ 문장 5.1-2-1

Answer aggregation. At test time, PRMs can be used to score each individual step appearing in a set of solutions sampled from the base model.

🔍 단위 해석

답안 집계. 테스트 시점에, PRM은 기본 모델에서 샘플링된 해답 세트에 포함된 각 개별 단계에 점수를 매기는 데 사용될 수 있습니다.

✔ 문장 5.1-2-2

To pick out the best answer from N samples with the PRM, we need a function that can aggregate across all the per-step scores for each answer to determine the best candidate for the correct answer.

🔍 단위 해석

PRM을 사용하여 N개의 샘플 중 최고의 답을 골라내기 위해서는, 각 답안의 모든 단계별 점수를 종합(aggregate)하여 정답이 될 최고의 후보를 결정하는 함수가 필요합니다.

✔ 문장 5.1-2-3

To do this, we take the PRM’s prediction at the last step as representative of the full-answer score and then follow Li et al. (2023) by applying “best-of-N weighted” selection across answers.

🔍 단위 해석

이를 위해, 우리는 마지막 단계에 대한 PRM의 예측값을 전체 답안 점수의 대표값으로 삼고, Li et al. (2023)을 따라 답안들 전반에 “가중(weighted) Best-of-N” 선택 방식을 적용합니다.

5.2 SEARCH METHODS AGAINST A PRM

📖 단락 5.2-1

We optimize the PRM at test time via tree search methods. We study three search approaches that sample outputs from a few-shot prompted base LLM (see Appendix J). An illustration is shown in Figure 2. We note that for all search algorithms, we use the same PRM verifier, enabling an even comparison. We include additional details about our different search methods in Appendix C.

✔ 문장 5.2-1-1

We optimize the PRM at test time via tree search methods.

🔍 단위 해석

우리는 테스트 시점에 트리 탐색 방법을 통해 PRM을 최적화(활용)합니다.

✔ 문장 5.2-1-2

We study three search approaches that sample outputs from a few-shot prompted base LLM (see Appendix J).

🔍 단위 해석

우리는 퓨샷(few-shot) 프롬프팅된 기본 LLM으로부터 출력을 샘플링하는 세 가지 탐색 접근법을 연구합니다.

✔ 문장 5.2-1-3

An illustration is shown in Figure 2.

🔍 단위 해석

이에 대한 예시가 Figure 2에 나와 있습니다.

🖼️ 시각 자료 [Figure 2]

(원문 캡션: **Figure 2:** Comparing different PRM search methods. Left: Best-of-N samples N full answers and then selects the best answer according to the PRM final score. Center: Beam search samples N candidates at each step, and selects the top M according to the PRM to continue the search from. Right: lookahead-search extends each step in beam-search to utilize a k-step lookahead while assessing which steps to retain and continue the search from. Thus lookahead-search needs more compute.)

🔍 캡션 해석

다양한 PRM 탐색 방법 비교. 왼쪽: Best-of-N은 N개의 전체 답안을 샘플링한 후 PRM 최종 점수에 따라 최고의 답을 선택합니다. 가운데: 빔 서치(Beam search)는 각 단계마다 N개의 후보를 샘플링하고, PRM 점수에 따라 상위 M개를 선택하여 탐색을 이어갑니다. 오른쪽: 룩어헤드 서치(Lookahead-search)는 빔 서치의 각 단계를 확장하여, 어떤 단계를 유지하고 탐색을 이어갈지 평가할 때 k-단계 앞을 내다보는(lookahead) 방식을 활용합니다. 따라서 룩어헤드 서치는 더 많은 연산을 필요로 합니다.

📄 PDF 시각 분석

PDF 5페이지의 **Figure 2**를 분석한 결과입니다:

- Best-of-N (좌측):** 파란색 질문 노드에서 여러 개의 빨간 선(답안)이 끝까지 뻗어나가고, 마지막에 검증기(점선 사각형)가 가장 좋은 것 하나(초록색 점)를 고르는 단순한 구조입니다.
- Beam Search (가운데):** 단계별로 진행됩니다. 각 단계마다 여러 후보(빨간 원)를 생성하고, 검증기가 이 중 유망한 것들(초록 원)만 골라 다음 단계로 확장합니다. 가지치기(pruning)가 일어나는 트리 구조를 보여줍니다.
- Lookahead Search (우측):** 빔 서치와 비슷하지만, 검증기가 현재 단계의 점수만 보는 게 아니라, 점선 화살표로 표시된 미래의 경로(Lookahead rollout)를 미리 가보고(시뮬레이션) 그 결과를 바탕으로 현재 단계를 선택합니다. 훨씬 더 깊고 복잡한 연산 과정을 시각적으로 보여줍니다.

✔ 문장 5.2-1-4

We note that for all search algorithms, we use the same PRM verifier, enabling an even comparison.

🔍 단위 해석

모든 탐색 알고리즘에 대해 동일한 PRM 검증기를 사용하여 공정한 비교가 가능하도록 했다는 점을 밝힙니다.

📖 단락 5.2-2

Best-of-N weighted. We sample N answers independently from the base LLM and then select the best answer according to the PRM’s final answer judgment.

✔ 문장 5.2-2-1

Best-of-N weighted. We sample N answers independently from the base LLM and then select the best answer according to the PRM’s final answer judgment.

🔍 단위 해석

가중 **Best-of-N.** 기본 LLM에서 N개의 답안을 독립적으로 샘플링한 다음, PRM의 최종 답안 판단(점수)에 따라 가장 좋은 답을 선택합니다.

📖 단락 5.2-3

Beam search. Beam search optimizes the PRM by searching over its per-step predictions. Our implementation is similar to BFS-V (Yao et al., 2023; Feng et al., 2024). Concretely, we consider a fixed number of beams N and a beam width M. At the end of the search we have N final answer candidates, to which we apply best-of-N weighted selection to make our final answer prediction.

✔ 문장 5.2-3-1

Beam search. Beam search optimizes the PRM by searching over its per-step predictions.

🔍 단위 해석

빔 서치. 빔 서치는 PRM의 단계별 예측값을 탐색하여 PRM 활용을 최적화합니다.

✔ 문장 5.2-3-2

Our implementation is similar to BFS-V (Yao et al., 2023; Feng et al., 2024).

🔍 단위 해석

우리의 구현은 BFS-V 방식과 유사합니다.

✔ 문장 5.2-3-3

Concretely, we consider a fixed number of beams N and a beam width M .

🔍 단위 해석

구체적으로, 우리는 고정된 빔 개수 N 과 빔 폭(width) M 을 고려합니다.

✔ 문장 5.2-3-4

At the end of the search we have N final answer candidates, to which we apply best-of- N weighted selection to make our final answer prediction.

🔍 단위 해석

탐색이 끝나면 N 개의 최종 답안 후보를 얻게 되며, 여기에 가중 Best-of- N 선택을 적용하여 최종 정답 예측을 수행합니다.

📖 단락 5.2-4

Lookahead search. Lookahead search modifies how beam search evaluates each step. At each step in the search, rather than using the PRM score at the current step to select the top options, lookahead search performs a simulation, rolling out k steps. We stop early if the end of a solution is reached.

✔ 문장 5.2-4-1

Lookahead search. Lookahead search modifies how beam search evaluates each step.

🔍 단위 해석

룩어헤드 서치. 룩어헤드 서치는 빔 서치가 각 단계를 평가하는 방식을 수정합니다.

✔ 문장 5.2-4-2

At each step in the search, rather than using the PRM score at the current step to select the top options, lookahead search performs a simulation, rolling out k steps.

🔍 단위 해석

탐색의 각 단계에서, 상위 옵션을 선택하기 위해 현재 단계의 PRM 점수를 사용하는 대신, 룩어헤드 서치는 k 단계를 미리 굴러보는(rolling out) 시뮬레이션을 수행합니다.

5.3 ANALYSIS RESULTS: TEST-TIME SCALING FOR SEARCH WITH VERIFIERS

📖 단락 5.3-1

We now present our results comparing various search algorithms and identify a prompt difficulty dependent compute-optimal scaling strategy for search methods.

✔ 문장 5.3-1-1

We now present our results comparing various search algorithms and identify a prompt difficulty dependent compute-optimal scaling strategy for search methods.

🔍 단위 해석

이제 다양한 탐색 알고리즘을 비교한 결과를 제시하고, 탐색 방법을 위한 ‘프롬프트 난이도 의존적’ 연산 최적화 스케일링 전략을 규명합니다.

단락 5.3-2

Comparing search algorithms. We first conduct a sweep over different search settings. In addition to the standard best-of-N approach, we sweep over the two main parameters that distinguish these methods: beam-width M and number of lookahead steps k. While we are not able to exhaustively sweep all configurations, we sweep over the following settings with a maximum budget of 256: 1) Beam search with the beam width set to $\sqrt{N}$, where N is the generation budget; 2) Beam search with a fixed beam width of 4; 3) Lookahead search with $k = 3$ applied to both beam-search settings 1) and 2); 4) Lookahead search with $k = 1$ applied to beam-search setting 1).

문장 5.3-2-1

Comparing search algorithms. We first conduct a sweep over different search settings.

 단위 해석

탐색 알고리즘 비교. 우리는 먼저 다양한 탐색 설정에 대해 스위프(sweep, 전수 조사 또는 광범위한 실험)을 수행합니다.

문장 5.3-2-2

In addition to the standard best-of-N approach, we sweep over the two main parameters that distinguish these methods: beam-width M and number of lookahead steps k.

 단위 해석

표준 Best-of-N 접근법 외에도, 우리는 이 방법들을 구분 짓는 두 가지 주요 변수인 빔 폭(M)과 룩어헤드 단계 수(k)에 대해 실험합니다.

문장 5.3-2-3

While we are not able to exhaustively sweep all configurations, we sweep over the following settings with a maximum budget of 256: 1) Beam search with the beam width set to $\sqrt{N}$, where N is the generation budget; 2) Beam search with a fixed beam width of 4; 3) Lookahead search with $k = 3$ applied to both beam-search settings 1) and 2); 4) Lookahead search with $k = 1$ applied to beam-search setting 1).

 단위 해석

모든 설정을 완벽하게 다 실험할 수는 없었지만, 최대 예산 256 내에서 다음 설정들을 실험했습니다:

1. 빔 폭을 $\sqrt{N}$ 으로 설정한 빔 서치 (N은 생성 예산)
2. 빔 폭을 4로 고정한 빔 서치
3. 1번과 2번 설정에 $k = 3$ 룩어헤드를 적용한 경우
4. 1번 설정에 $k = 1$ 룩어헤드를 적용한 경우

단락 5.3-3

To compare search methods as a function of generation budget fairly, we estimate the inference-time cost of each method. For beam search and best-of-N the generation budget corresponds to the number of beams and N respectively. Lookahead search utilizes additional compute: at each step, we sample k additional steps ahead. Therefore, the cost of lookahead-search is $N \times (k + 1)$ samples. Querying the verifier also adds a 2x overhead for all methods but we account for this in our analysis.

문장 5.3-3-1

To compare search methods as a function of generation budget fairly, we estimate the inference-time cost of each method.

 단위 해석

생성 예산에 따른 탐색 방법들을 공정하게 비교하기 위해, 우리는 각 방법의 추론 비용을 추산합니다.

문장 5.3-3-2

For beam search and best-of-N the generation budget corresponds to the number of beams and N respectively.

 단위 해석

빔 서치와 Best-of-N의 경우 생성 예산은 각각 빔의 개수와 N에 해당합니다.

문장 5.3-3-3

Lookahead search utilizes additional compute: at each step, we sample k additional steps ahead.

 단위 해석

룩어헤드 서치는 추가적인 연산을 사용합니다: 각 단계에서 k개의 단계를 추가로 앞서 샘플링합니다.

✔ 문장 5.3-3-4

Therefore, the cost of lookahead-search is $N \times (k + 1)$ samples.

🔍 단위 해석

따라서 룩어헤드 서치의 비용은 $N \times (k + 1)$ 샘플입니다.

📖 단락 5.3-4

Results. As shown in Figure 3 (left), with small budgets, beam search outperforms best-of-N. However, at high budgets, these improvements diminish, with beam search underperforming. Additionally, lookahead-search underperforms other methods, likely due to the additional computation induced by looking-ahead. It is possible that with further test-time scaling or with an online MCST trained value function, lookahead search may perform better; we leave further exploration of this to future work. The diminishing returns from search are likely due to exploitation of the PRM’s predictions. For example, we see instances (such as in Figure 32), where search causes the model to generate repetitive low-information steps. In other cases, we find that over-optimizing search can result in overly short solutions, of just 1-2 steps. We include several of these examples in Appendix R.

✔ 문장 5.3-4-1

Results. As shown in Figure 3 (left), with small budgets, beam search outperforms best-of-N.

🔍 단위 해석

결과. Figure 3(왼쪽)에서 볼 수 있듯이, 적은 예산에서는 빔 서치가 Best-of-N보다 성능이 뛰어납니다.

✔ 문장 5.3-4-2

However, at high budgets, these improvements diminish, with beam search underperforming.

🔍 단위 해석

하지만 예산이 커지면 이러한 개선 효과는 줄어들며, 결국 빔 서치의 성능이 더 낮아집니다.

✔ 문장 5.3-4-3

Additionally, lookahead-search underperforms other methods, likely due to the additional computation induced by looking-ahead.

🔍 단위 해석

게다가 룩어헤드 서치는 다른 방법들보다 성능이 떨어지는데, 이는 아마도 룩어헤드(미리보기)로 인한 추가 연산 비용 때문일 것입니다.

🖼️ 시각 자료 [Figure 3]

(원문 캡션: **Figure 3:** Left: Comparing different methods for conducting search against PRM verifiers. We see that at low generation budgets, beam search performs best, but as we scale the budget further the improvements diminish, falling below the best-of-N baseline. Lookahead-search generally underperforms other methods at the same generation budget. Right: Comparing beam search and best-of-N binned by difficulty level...)

🔍 캡션 해석

왼쪽: PRM 검증기를 이용한 다양한 탐색 방법 비교. 낮은 생성 예산에서는 빔 서치가 가장 우수하지만, 예산을 확장할수록 개선폭이 줄어들어 Best-of-N 베이스라 인보다 떨어짐을 알 수 있습니다. 룩어헤드 서치는 일반적으로 같은 예산에서 다른 방법들보다 성능이 낮습니다. 오른쪽: 난이도별로 구분한 빔 서치와 Best-of-N 비교...

📄 PDF 시각 분석

PDF 6페이지의 Figure 3 분석:

- Left (알고리즘 비교):** X축(예산)이 증가함에 따라 대부분의 곡선이 우상향하지만, ‘Beam Search’ (파란색/보라색 점선) 계열은 초반($2^2 \sim 2^5$)에 급격히 상승하다가 후반(2^6 이상)에는 꺾이거나 평탄해집니다. 반면 ‘Best-of-N Weighted’ (주황색)는 꾸준히 상승하여 결국 가장 높은 정확도에 도달합니다.
- Right (난이도별 비교):**
 - 난이도 1~2 (쉬움): 빔 서치(파란 막대)가 예산을 늘릴수록(막대가 4개) 오히려 성능이 떨어지거나 정체되는 경향이 뚜렷합니다. (과적합/오남용)
 - 난이도 3~4 (중간/어려움): 빔 서치가 Best-of-N(주황 막대)보다 성능이 좋습니다. 어려운 문제일수록 정교한 탐색이 효과적임을 보여줍니다.
 - 난이도 5 (매우 어려움): 두 방법 모두 점수가 매우 낮아 큰 차이가 없습니다.

✔ 문장 5.3-4-4

The diminishing returns from search are likely due to exploitation of the PRM’s predictions.

🔍 단위 해석

탐색의 수확 체감(diminishing returns) 현상은 아마도 PRM 예측값에 대한 과도한 착취(exploitation, 모델의 허점을 파고드는 현상) 때문일 가능성이 큼니다.

✔ 문장 5.3-4-5

For example, we see instances (such as in Figure 32), where search causes the model to generate repetitive low-information steps.

🔍 단위 해석

예를 들어, 우리는 탐색이 모델로 하여금 정보가 거의 없는 반복적인 단계를 생성하게 만드는 사례들(Figure 32 등)을 목격했습니다.

📖 단락 5.3-5

Which problems does search improve? To understand how to scale search adaptively per problem, we conduct a difficulty bin analysis. Specifically, we compare beam-search ($M = 4$) against best-of-N. In Figure 3 (right), we find that, despite performing similarly in aggregate, the two methods exhibit very different behavior across difficulty levels. For example, on easy questions (levels 1/2), the stronger optimizer of the two, beam search, degrades in performance as the budget increases, suggesting possible exploitation of the PRM signal. In contrast, on the harder questions (levels 3/4), beam search outperforms best-of-N. Finally, on the most difficult questions (level 5), no method makes meaningful progress. These findings match intuition: we might expect that on the easy or medium difficulty questions, the verifier will make mostly correct assessments of correctness. Therefore, by optimizing further, we may be only further amplifying any spurious features learned by the verifier, causing performance degradation. On more difficult questions, the base model is less likely to sample the correct answer, so using search can help steer the model.

✔ 문장 5.3-5-1

Which problems does search improve? To understand how to scale search adaptively per problem, we conduct a difficulty bin analysis.

🔍 단위 해석

탐색은 어떤 문제들을 개선하는가? 문제마다 적응적으로 탐색을 확장하는 방법을 이해하기 위해, 우리는 난이도 구간(difficulty bin) 분석을 수행합니다.

✔ 문장 5.3-5-2

Specifically, we compare beam-search ($M = 4$) against best-of-N. In Figure 3 (right), we find that, despite performing similarly in aggregate, the two methods exhibit very different behavior across difficulty levels.

🔍 단위 해석

구체적으로, 우리는 빔 서치($M = 4$)와 Best-of-N을 비교합니다. Figure 3(오른쪽)에서, 두 방법이 전체적으로는 비슷한 성능을 보임에도 불구하고 난이도 레벨에 따라 매우 다른 거동을 보인다는 것을 발견했습니다.

✔ 문장 5.3-5-3

For example, on easy questions (levels 1/2), the stronger optimizer of the two, beam search, degrades in performance as the budget increases, suggesting possible exploitation of the PRM signal.

🔍 단위 해석

예를 들어, 쉬운 문제(레벨 1/2)에서는 둘 중 더 강력한 최적화 도구인 빔 서치가 예산이 늘어날수록 성능이 저하되는데, 이는 PRM 신호가 악용(exploitation)되고 있을 가능성을 시사합니다.

✔ 문장 5.3-5-4

In contrast, on the harder questions (levels 3/4), beam search outperforms best-of-N.

🔍 단위 해석

반대로, 더 어려운 문제(레벨 3/4)에서는 빔 서치가 Best-of-N보다 뛰어난 성능을 보입니다.

📖 단락 5.3-6

Compute-optimal search. Given the above, it is clear that question difficulty is a useful statistic for predicting the best search strategy at each budget. Additionally, the selected best search strategy varies as a function of difficulty. We visualize this “compute-optimal” scaling trend, as represented by the best performing search strategy, between best-of-N and beam search ($M = 4$), at each difficulty level in Figure 4. Interestingly, we see that with low budgets, using both the oracle and predicted difficulty, compute-optimal scaling can nearly outperform best-of-N using up to 4x less test-time compute (e.g. 16 versus 64 generations). While at higher budgets, some of these benefits diminish with the use of predicted difficulty, but the oracle bins still see improvements from optimal scaling. This result demonstrates that there are clear performance gains to be obtained by adaptively allocating test-time compute during search using predicted difficulty as an input statistic.

✔ 문장 5.3-6-1

Compute-optimal search. Given the above, it is clear that question difficulty is a useful statistic for predicting the best search strategy at each budget.

단위 해석
연산 최적화 탐색. 위 내용을 종합해 볼 때, 문제 난이도가 각 예산 수준에서 최고의 탐색 전략을 예측하는 데 유용한 통계량임이 분명합니다.

문장 5.3-6-2

We visualize this “compute-optimal” scaling trend, as represented by the best performing search strategy, between best-of-N and beam search ($M = 4$), at each difficulty level in Figure 4.

단위 해석
우리는 Figure 4에서 각 난이도 레벨별로 Best-of-N과 빔 서치($M = 4$) 중 가장 성능이 좋은 전략을 선택했을 때의 “연산 최적화” 스케일링 추세를 시각화했습니다.

시각 자료 [Figure 4]

(원문 캡션: **Figure 4:** Comparing compute-optimal test-time scaling against baselines with PRM search. By scaling test-time compute optimally, we nearly outperform PRM best-of-N using up to 4x less test-time compute (e.g. 32 versus 128 generations). “Compute-optimal oracle” refers to using oracle difficulty bins derived from the groundtruth correctness, and “compute-optimal predicted” refers to using the PRM’s predictions to generate difficulty bins.)

캡션 해석
PRM 탐색에서 연산 최적화 테스트 시점 스케일링과 베이스라인의 비교. 테스트 시점 연산을 최적으로 확장함으로써, 우리는 최대 4배 적은 연산(예: 32회 대 128회 생성)만으로도 PRM Best-of-N 성능을 거의 능가할 수 있습니다. "Compute-optimal oracle"은 실제 정답에서 도출된 오라클 난이도 구간을 사용한 것이고, "compute-optimal predicted"는 PRM의 예측을 사용하여 난이도 구간을 생성한 것을 의미합니다.

PDF 시각 분석

PDF 7페이지의 **Figure 4** 분석:

- 그래프 추세: 파란색 선(Oracle Optimal)과 주황색 선(Predicted Optimal)이 초록색 선(PRM Best-of-N)보다 왼쪽에 위치합니다.
- 해석: 같은 정확도(Y축)에 도달하기 위해 필요한 예산(X축)이 더 적다는 뜻입니다. 예를 들어, 정확도 30% 선을 보면 Optimal 전략들은 2^3 (8회) 부근에서 도달하지만, 일반 Best-of-N은 2^5 (32회) 부근에서 도달합니다. 즉, 적응형 전략이 훨씬 효율적입니다.

문장 5.3-6-3

Interestingly, we see that with low budgets, using both the oracle and predicted difficulty, compute-optimal scaling can nearly outperform best-of-N using up to 4x less test-time compute (e.g. 16 versus 64 generations).

단위 해석
흥미롭게도 낮은 예산에서 오라클 및 예측 난이도를 모두 사용했을 때, 연산 최적화 스케일링은 최대 4배 적은 테스트 시점 연산(예: 16회 대 64회)을 사용하고도 Best-of-N을 거의 능가하는 성능을 보였습니다.

문장 5.3-6-4

This result demonstrates that there are clear performance gains to be obtained by adaptively allocating test-time compute during search using predicted difficulty as an input statistic.

단위 해석
이 결과는 예측된 난이도를 입력 통계량으로 사용하여 탐색 중에 테스트 시점 연산을 적응적으로 할당함으로써 분명한 성능 이득을 얻을 수 있음을 입증합니다.

6 REFINING THE PROPOSAL DISTRIBUTION

단락 6-1

Now we study how the proposal distribution can be used for test-time scaling (Section 2). Concretely, we enable to improve its own distribution at test-time, by revising answers iteratively. Simply prompting existing LLMs to correct themselves tends to be largely ineffective on reasoning. Therefore, we finetune LLMs to iteratively revise their answers.

문장 6-1-1

Now we study how the proposal distribution can be used for test-time scaling (Section 2).

단위 해석
이제 우리는 제안 분포(proposal distribution)가 테스트 시점 스케일링에 어떻게 사용될 수 있는지 연구합니다(섹션 2 참조).

문장 6-1-2

Concretely, we enable to improve its own distribution at test-time, by revising answers iteratively.

단위 해석

구체적으로, 우리는 모델이 답안을 반복적으로 수정(revising)하게 함으로써 테스트 시점에 자신의 분포를 개선할 수 있도록 만듭니다.

문장 6-1-3

Simply prompting existing LLMs to correct themselves tends to be largely ineffective on reasoning.

단위 해석

기존 LLM에게 단순히 스스로를 교정하라고 프롬프팅하는 것은 추론 작업에서 대체로 효과가 없는 경향이 있습니다.

문장 6-1-4

Therefore, we finetune LLMs to iteratively revise their answers.

단위 해석

따라서 우리는 LLM이 자신의 답을 반복적으로 수정하도록 미세 조정(finetune)합니다.

6.1 TRAINING AND USING REVISION MODELS

단락 6.1-1

Our procedure for finetuning revision models is similar to Qu et al. (2024b), though we introduce some crucial differences. For finetuning, we need trajectories consisting of a sequence of incorrect answers followed by a correct answer, that we can then run SFT on. To do this, we sampled 64 responses in parallel and post-hoc constructed multi-turn rollouts from these independent samples. These rollouts consist of up to four incorrect attempts in context followed by a correct revision. We include more details on our revision model finetuning procedure in Appendix L.

문장 6.1-1-1

Our procedure for finetuning revision models is similar to Qu et al. (2024b), though we introduce some crucial differences.

단위 해석

우리의 수정 모델 미세 조정 절차는 Qu et al. (2024b)와 유사하지만, 몇 가지 중요한 차이점을 도입했습니다.

문장 6.1-1-2

For finetuning, we need trajectories consisting of a sequence of incorrect answers followed by a correct answer, that we can then run SFT on.

단위 해석

미세 조정을 위해서는 ‘오답들의 연속’ 뒤에 ‘정답’이 이어지는 궤적(trajec­tory)이 필요하며, 우리는 이에 대해 SFT(Supervised Fine-Tuning)를 수행할 수 있습니다.

문장 6.1-1-3

To do this, we sampled 64 responses in parallel and post-hoc constructed multi-turn rollouts from these independent samples.

단위 해석

이를 위해 우리는 64개의 응답을 병렬로 샘플링한 뒤, 이 독립적인 샘플들로부터 사후적으로(post-hoc) 멀티턴 롤아웃(multi-turn rollouts)을 구성했습니다.

보조 설명

- [Post-hoc construction]:** 처음부터 모델이 대화하듯 생성한 게 아니라, 따로따로 생성된 답들을 모아서 마치 대화(수정 과정)였던 것처럼 데이터를 조립했다는 뜻입니다.

문장 6.1-1-4

These rollouts consist of up to four incorrect attempts in context followed by a correct revision.

단위 해석

이 롤아웃들은 문맥(context)에 포함된 최대 4번의 오답 시도와 그 뒤에 이어지는 정답 수정으로 구성됩니다.

단락 6.1-2

Using revisions at inference-time. Given a finetuned model, we can then sample a sequence of revisions from the model at test time. While our revision model is only trained with up to four previous answers in-context, we can sample longer chains by truncating the context to the most recent four revisions. In Figure 9(left), we see longer chains gradually improve pass@k demonstrating that we are able to effectively

teach the model to learn from mistakes in previous answers. That said, there is a distribution shift at inference time: the model was trained on only sequences with incorrect answers in context, but at test-time the model may sample correct answers. Thus, the model may turn a correct answer into an incorrect one. Similar to Qu et al. (2024b), around 38% of correct answers get converted to incorrect with our model. Thus, we employ either sequential majority voting or verifier-guided selection to select the correct answer from the sequence of revisions (see Figure 5). Querying the verifier adds a 2x compute overhead, and we account for this in our analysis.

✅ 문장 6.1-2-1

Using revisions at inference-time. Given a finetuned model, we can then sample a sequence of revisions from the model at test time.

🔍 단위 해석

추론 시점의 수정(**Revision**) 활용. 미세 조정된 모델이 주어지면, 테스트 시점에 모델로부터 수정 시퀀스를 샘플링할 수 있습니다.

✅ 문장 6.1-2-2

While our revision model is only trained with up to four previous answers in-context, we can sample longer chains by truncating the context to the most recent four revisions.

🔍 단위 해석

우리 수정 모델은 문맥에 최대 4개의 이전 답안만 포함하여 훈련되었지만, 문맥을 가장 최근 4개의 수정으로 잘라냄(truncating)으로써 더 긴 체인을 샘플링할 수 있습니다.

✅ 문장 6.1-2-3

In Figure 9(left), we see longer chains gradually improve pass@k demonstrating that we are able to effectively teach the model to learn from mistakes in previous answers.

🔍 단위 해석

Figure 9(왼쪽)에서 더 긴 체인이 pass@k를 점진적으로 향상시키는 것을 볼 수 있는데, 이는 우리가 모델에게 이전 답안의 실수로부터 배우도록 효과적으로 가르쳤음을 입증합니다.

✅ 문장 6.1-2-4

That said, there is a distribution shift at inference time: the model was trained on only sequences with incorrect answers in context, but at test-time the model may sample correct answers.

🔍 단위 해석

그렇긴 하지만 추론 시점에는 분포 변화(distribution shift)가 발생합니다: 모델은 문맥에 오답만 있는 시퀀스로 훈련되었으나, 테스트 시점에는 모델이 정답을 샘플링할 수도 있기 때문입니다. (즉, 정답을 보고도 오답인 줄 알고 고치려 할 수 있음)

✅ 문장 6.1-2-5

Similar to Qu et al. (2024b), around 38% of correct answers get converted to incorrect with our model.

🔍 단위 해석

Qu et al. (2024b)와 유사하게, 우리 모델에서도 약 38%의 정답이 오답으로 변환되었습니다. (멀쩡한 정답을 고쳐서 틀리는 경우)

✅ 문장 6.1-2-6

Thus, we employ either sequential majority voting or verifier-guided selection to select the correct answer from the sequence of revisions (see Figure 5).

🔍 단위 해석

따라서 우리는 수정 시퀀스에서 정답을 선택하기 위해 순차적 다수결 투표(sequential majority voting)나 검증기 기반 선택(verifier-guided selection)을 사용합니다(Figure 5 참조).

🖼️ 시각 자료 [Figure 5]

(원문 캡션: **Figure 5:** Parallel sampling (e.g., Best-of-N) versus sequential revisions. Left: Parallel sampling generates N answers independently, whereas sequential revisions generate each one in sequence conditioned on previous attempts. Right: In both the sequential and parallel cases, we can use the verifier to determine the best-of-N answers. We can also split our budget between parallel and sequential sampling. In this case, we first use the verifier to select the best answer within each sequential chain and then select the best answer across chains.)

🔍 캡션 해석

병렬 샘플링(예: Best-of-N) 대 순차적 수정. 왼쪽: 병렬 샘플링은 N개의 답안을 독립적으로 생성하는 반면, 순차적 수정은 이전 시도들에 조건부로 의존하여 각 답안을 순차적으로 생성합니다. 오른쪽: 순차적 및 병렬 케이스 모두에서 우리는 Best-of-N 답안을 결정하기 위해 검증기를 사용할 수 있습니다. 또한 예산을 병렬 샘플링과 순차적 샘플링 사이에 분할할 수도 있습니다. 이 경우, 먼저 각 순차적 체인 내에서 검증기를 사용해 최고의 답을 선택한 다음, 체인들 사이에서 최고의 답을 선택합니다.

1. Left Diagram:
 - Parallel: 하나의 질문(Q)에서 화살표가 여러 갈래로 뻗어나가 독립적인 답($A_1, A_2, \dots$)을 만듭니다. 서로 영향을 주지 않습니다.
 - Sequential: 질문(Q)에서 첫 답(A_1)이 나오고, A_1 을 보고 A_2 가 나오고, A_2 를 보고 A_3 가 나오는 체인 구조입니다. 이전 실수를 반영할 수 있습니다.
2. Right Diagram:
 - 이 두 가지를 섞은 하이브리드 구조를 보여줍니다. 여러 개의 순차적 체인(Sequential Chain)을 병렬로 돌리고, 각 체인의 승자(Winner)들끼리 다시 경쟁하여 최종 답(Final Answer)을 뽑는 계층적 구조입니다.

단락 6.1-3

Comparisons. To test the efficacy of modifying the proposal distribution via revisions, we set up a comparison between the performance of sampling N revisions in sequence and sampling N attempts at a question in parallel. We see in Figure 9 (right), that with both the verifier-based and majority-based selection mechanisms, sequential sampling outperforms parallel sampling.

문장 6.1-3-1

Comparisons. To test the efficacy of modifying the proposal distribution via revisions, we set up a comparison between the performance of sampling N revisions in sequence and sampling N attempts at a question in parallel.

단위 해석

비교. 수정을 통해 제안 분포를 수정하는 것의 효용성을 테스트하기 위해, 우리는 N개의 수정을 순차적으로 샘플링하는 것과 N개의 시도를 병렬로 샘플링하는 것 사이의 성능 비교를 설정했습니다.

문장 6.1-3-2

We see in Figure 9 (right), that with both the verifier-based and majority-based selection mechanisms, sequential sampling outperforms parallel sampling.

단위 해석

Figure 9(오른쪽)에서, 검증기 기반 선택 메커니즘과 다수결 기반 선택 메커니즘 모두에서 순차적 샘플링이 병렬 샘플링보다 우수함을 알 수 있습니다.

6.2 ANALYSIS RESULTS: TEST-TIME SCALING WITH REVISIONS

단락 6.2-1

We see that sampling sequentially outperforms in parallel. We might expect however, that these approaches have different properties. Intuitively, sampling in parallel acts as a global search process that could, in principle, provide coverage over many different approaches for solving a problem. Sequential sampling, on the other hand, may work more as a local refinement process. This motivates striking a balance between these two approaches by allocating some of our budget to parallel sampling (e.g. $\sqrt{N}$) and the rest to sequential (e.g. $\sqrt{N}$). We will now show the existence of a compute-optimal ratio between sequential and parallel sampling, and understand their pros and cons based on the difficulty of a prompt.

문장 6.2-1-1

We see that sampling sequentially outperforms in parallel.

단위 해석

우리는 순차적 샘플링이 병렬 샘플링보다 성능이 우수함을 확인했습니다.

문장 6.2-1-2

We might expect however, that these approaches have different properties.

단위 해석

하지만 우리는 이 접근법들이 서로 다른 속성을 가질 것이라 예상할 수 있습니다.

문장 6.2-1-3

Intuitively, sampling in parallel acts as a global search process that could, in principle, provide coverage over many different approaches for solving a problem.

단위 해석
직관적으로, 병렬 샘플링은 전역 탐색(global search) 과정으로 작용하여, 원칙적으로 문제 해결을 위한 다양한 접근 방식들을 포괄(coverage)할 수 있습니다. (다양한 시도)

문장 6.2-1-4

Sequential sampling, on the other hand, may work more as a local refinement process.

단위 해석
반면, 순차적 샘플링은 국소적 정제(local refinement) 과정으로 더 많이 작용할 수 있습니다. (하나를 깊게 파고들기)

문장 6.2-1-5

This motivates striking a balance between these two approaches by allocating some of our budget to parallel sampling (e.g. $\sqrt{N}$) and the rest to sequential (e.g. $\sqrt{N}$).

단위 해석
이는 예산의 일부(예: $\sqrt{N}$)를 병렬 샘플링에 할당하고 나머지를 순차적 샘플링(예: $\sqrt{N}$)에 할당함으로써 두 접근법 사이의 균형을 맞추도록 동기를 부여합니다.

단락 6.2-2

Trading off sequential and parallel compute. To understand how to allocate sequential and parallel compute, we perform a sweep over different configurations. We see, in Figure 7 (left), that indeed, at a given budget, there exists an ideal sequential to parallel ratio. We also see in Figure 7 (right) that this ideal ratio varies depending on question difficulty. Easy questions benefit more from revisions, whereas on difficult questions it is optimal to strike a balance between sequential and parallel computation. This finding supports the hypothesis that sequential revisions (i.e., varying the proposal distribution) and parallel sampling (i.e., search with verifiers) are complementary axes for scaling test-time compute, which may be more effective on a per-prompt basis. We include examples of our model’s generations in Appendix Q. Additional results are in Appendix D.

문장 6.2-2-1

Trading off sequential and parallel compute. To understand how to allocate sequential and parallel compute, we perform a sweep over different configurations.

단위 해석
순차적 연산과 병렬 연산의 트레이드오프. 순차적 연산과 병렬 연산을 어떻게 할당할지 이해하기 위해, 우리는 다양한 설정에 대해 스윙(실험)을 수행합니다.

문장 6.2-2-2

We see, in Figure 7 (left), that indeed, at a given budget, there exists an ideal sequential to parallel ratio.

단위 해석
Figure 7(왼쪽)에서 볼 수 있듯이, 실제로 주어진 예산하에서 이상적인 '순차 대 병렬 비율'이 존재합니다.

문장 6.2-2-3

We also see in Figure 7 (right) that this ideal ratio varies depending on question difficulty.

단위 해석
또한 Figure 7(오른쪽)에서 이 이상적인 비율이 문제 난이도에 따라 달라짐을 알 수 있습니다.

문장 6.2-2-4

Easy questions benefit more from revisions, whereas on difficult questions it is optimal to strike a balance between sequential and parallel computation.

단위 해석
쉬운 문제들은 수정(revision)으로부터 더 많은 이득을 얻는 반면, 어려운 문제들에서는 순차적 연산과 병렬 연산 사이의 균형을 맞추는 것이 최적입니다.

문장 6.2-2-5

This finding supports the hypothesis that sequential revisions (i.e., varying the proposal distribution) and parallel sampling (i.e., search with verifiers) are complementary axes for scaling test-time compute, which may be more effective on a per-prompt basis.

단위 해석
이 발견은 순차적 수정(제안 분포 변화)과 병렬 샘플링(검증기 탐색)이 테스트 시점 연산을 확장하는 상호 보완적인 축이며, 프롬프트별로 적용할 때 더 효과적일 수 있다는 가설을 뒷받침합니다.

시각 자료 [Figure 7]

(원문 캡션: **Figure 7:** Left: Varying the ratio of the generation budget allocated sequential revisions to versus parallel samples. Each line represents a fixed generation budget as the ratio is changed. We use the verifier for answer selection. We see that while increased sequential revisions tends to outperform more parallel compute, at higher generation budgets there is an ideal ratio that strikes a balance between the two extremes. Right: Varying the sequential to parallel ratio for a generation budget of 128 across difficulty bins. Using verifier-based selection, we see that the easier questions attain the best performance with full sequential compute. On the harder questions, there is an ideal ratio of sequential to parallel test-time compute.)

캡션 해석

왼쪽: 생성 예산 내에서 순차적 수정 대 병렬 샘플에 할당된 비율을 변화시킴. 각 선은 비율이 변할 때의 고정된 생성 예산을 나타냅니다. 답안 선택에는 검증기를 사용합니다. 순차적 수정을 늘리는 것이 더 많은 병렬 연산보다 성능이 좋은 경향이 있지만, 높은 생성 예산에서는 두 극단 사이의 균형을 맞추는 이상적인 비율이 존재함을 볼 수 있습니다. 오른쪽: 생성 예산 128에서 난이도 구간별로 순차 대 병렬 비율을 변화시킴. 검증기 기반 선택을 사용할 때, 쉬운 문제들은 완전한 순차적 연산(full sequential)으로 최고의 성능을 얻음을 알 수 있습니다. 더 어려운 문제들에서는 순차 및 병렬 테스트 시점 연산의 이상적인 비율이 존재합니다.

PDF 시각 분석

PDF 9페이지의 **Figure 7** 분석:

- Left Graph:** X축은 순차적 샘플의 수(N_{seq}), Y축은 정확도입니다. 같은 색깔의 선(동일 예산)을 따라가면, 오른쪽 끝(순차 몰빵)이나 왼쪽 끝(병렬 몰빵)보다 중간 지점에서 피크를 찍는 경우가 있습니다(특히 보라색 선, Budget 256). 이는 '적절한 믹스'가 중요함을 시사합니다.
- Right Graph (난이도별):**
 - 난이도 1 (파란색):** 그래프가 오른쪽으로 갈수록 높아집니다. 즉, 순차적 수정(고치기)을 많이 할수록 좋습니다. (쉬운 문제는 고치면 맞는다)
 - 난이도 4 (빨간색):** 그래프가 중간에서 가장 높습니다. 무작정 고치기만 하는 것보다, 여러 개를 병렬로 시도해보는 것(탐색)을 섞는 게 좋습니다.

단락 6.2-3

Compute-optimal revisions. Given our finding that the efficacy of sequential and parallel sampling depends on difficulty, we can select the ideal ratio of sequential to parallel compute per difficulty bin (we describe the specific ratios in Appendix O). In Figure 6, we plot results using our compute-optimal scaling when employing both oracle and predicted difficulty. In both cases, we substantially improve test-time compute scaling by optimally scaling the proposal distribution. In particular, we see that at higher generation budgets, parallel sampling plateaus, whereas compute-optimal scaling continues to improve. For both oracle and predicted difficulty, we see that compute-optimal scaling can outperform best-of-N using up to 4x less test-time compute (e.g. 64 samples versus 256). Overall, these results demonstrate the potential for improved test-time compute scaling by adjusting the proposal distribution on a per-prompt basis.

문장 6.2-3-1

Compute-optimal revisions. Given our finding that the efficacy of sequential and parallel sampling depends on difficulty, we can select the ideal ratio of sequential to parallel compute per difficulty bin (we describe the specific ratios in Appendix O).

단위 해석

연산 최적화 수정. 순차 및 병렬 샘플링의 효율성이 난이도에 의존한다는 우리의 발견을 바탕으로, 우리는 각 난이도 구간별로 순차 대 병렬 연산의 이상적인 비율을 선택할 수 있습니다 (구체적인 비율은 부록 O에 기술되어 있음).

문장 6.2-3-2

In Figure 6, we plot results using our compute-optimal scaling when employing both oracle and predicted difficulty.

단위 해석

Figure 6에서, 우리는 오라클 난이도와 예측 난이도를 모두 적용했을 때의 연산 최적화 스케일링 결과를 도표로 나타냅니다.

시각 자료 [Figure 6]

(원문 캡션: **Figure 6:** Compute-optimal scaling with our revision model. By optimally scaling test-time compute, we outperform best-of-N with 4x less compute (i.e., 128 samples versus 512). “Compute Optimal Oracle” refers to difficulty derived from ground truth correctness and “Compute Optimal Predicted” refers to using the PRM to estimate difficulty.)

캡션 해석

우리 수정 모델을 사용한 연산 최적화 스케일링. 테스트 시점 연산을 최적으로 확장함으로써, 우리는 4배 더 적은 연산(예: 128개 샘플 대 512개)으로 Best-of-N을 능가합니다.

PDF 시각 분석

PDF 9페이지의 **Figure 6** 분석:

- 그래프 해석:** 초록색 선(Parallel Best-of-N)은 예산이 늘어나면(2^6 이후) 평평해지는데(plateau) 경향을 보입니다. 반면 파란색(Oracle)과 주황색(Predicted) 선은 계속 우상향하며 초록색 선보다 훨씬 위에 위치합니다. 이는 단순 물량 공세(Parallel)보다 전략적 접근(Compute Optimal)이 훨씬 효율적이고 고점도 높음을 시각적으로 증명합니다.

✔ 문장 6.2-3-3

In both cases, we substantially improve test-time compute scaling by optimally scaling the proposal distribution.

🔍 단위 해석

두 경우 모두, 우리는 제안 분포를 최적으로 스케일링함으로써 테스트 시점 연산 스케일링을 상당히 개선합니다.

✔ 문장 6.2-3-4

In particular, we see that at higher generation budgets, parallel sampling plateaus, whereas compute-optimal scaling continues to improve.

🔍 단위 해석

특히, 높은 생성 예산에서 병렬 샘플링은 정체기(plateau)에 접어드는 반면, 연산 최적화 스케일링은 계속해서 성능이 향상되는 것을 볼 수 있습니다.

✔ 문장 6.2-3-5

For both oracle and predicted difficulty, we see that compute-optimal scaling can outperform best-of-N using up to 4x less test-time compute (e.g. 64 samples versus 256).

🔍 단위 해석

오라클 및 예측 난이도 모두에서, 연산 최적화 스케일링은 최대 4배 적은 테스트 시점 연산(예: 64개 샘플 대 256개)을 사용하더라도 Best-of-N을 능가할 수 있음을 확인했습니다.

7 EXCHANGING PRETRAINING AND TEST-TIME COMPUTE

📖 단락 7-1

We saw that utilizing additional test-time compute can enable us to represent more complex distributions than the one predicted by the base LLM, thereby increasing performance. We now posit that this increased flexibility of representing distributions means that we can expect additional test-time compute to make up for the lack of a higher-capacity model or training for more FLOPs during pretraining. In this section, we study to what extent this is possible. We pose the following question:

✔ 문장 7-1-1

We saw that utilizing additional test-time compute can enable us to represent more complex distributions than the one predicted by the base LLM, thereby increasing performance.

🔍 단위 해석

우리는 추가적인 테스트 시점 연산을 활용하는 것이 기본 LLM이 예측하는 것보다 더 복잡한 분포를 표현할 수 있게 하여 성능을 높인다는 것을 보았습니다.

✔ 문장 7-1-2

We now posit that this increased flexibility of representing distributions means that we can expect additional test-time compute to make up for the lack of a higher-capacity model or training for more FLOPs during pretraining.

🔍 단위 해석

이제 우리는 이러한 분포 표현의 유연성 증가가, 더 큰 용량의 모델이나 사전 학습 시의 더 많은 연산량(FLOPs) 부재를 추가적인 테스트 시점 연산으로 메꿀 수 있음을 의미한다고 가정합니다.

✔ 문장 7-1-3

In this section, we study to what extent this is possible.

🔍 단위 해석

이 섹션에서는 이것이 어느 정도까지 가능한지 연구합니다.

✔ 문장 7-1-4

We pose the following question:

🔍 단위 해석

우리는 다음과 같은 질문을 던집니다:

📖 단락 7-2

Question: Exchanging pretraining and test-time compute

Suppose a model was pre-trained with X FLOPs. Assume that we plan to run Y FLOPs of inference with this model. If we want to improve

performance by increasing the total FLOPs budget by a factor of M (i.e., M(X+Y) total FLOPs across both pretraining and inference), should we spend our FLOPS on increased pretraining compute or on additional test-time compute?

Increasing pretraining FLOPS introduces the additional design decision of whether to allocate compute to training with more data or more parameters. We focus on the setting in which model parameters are scaled up and training data amount is fixed, matching the canonical approach from the LLaMA series of models.

✅ 문장 7-2-1

Question: Exchanging pretraining and test-time compute

Suppose a model was pre-trained with X FLOPs. Assume that we plan to run Y FLOPs of inference with this model.

🔍 단위 해석

질문: 사전 학습과 테스트 시점 연산의 교환

어떤 모델이 X FLOPs로 사전 학습되었다고 가정합니다. 그리고 이 모델로 Y FLOPs만큼의 추론을 실행할 계획이라고 가정합니다.

✅ 문장 7-2-2

If we want to improve performance by increasing the total FLOPs budget by a factor of M (i.e., M(X+Y) total FLOPs across both pretraining and inference), should we spend our FLOPS on increased pretraining compute or on additional test-time compute?

🔍 단위 해석

만약 우리가 총 FLOPs 예산을 M배로 늘려 성능을 향상시키고 싶다면(즉, 사전 학습과 추론을 합쳐 총 M(X+Y) FLOPs), 이 늘어난 FLOPs를 사전 학습 연산을 늘리는 데 써야 할까요, 아니면 추가적인 테스트 시점 연산에 써야 할까요?

✅ 문장 7-2-3

Increasing pretraining FLOPS introduces the additional design decision of whether to allocate compute to training with more data or more parameters.

🔍 단위 해석

사전 학습 FLOPs를 늘리는 것은 더 많은 데이터로 훈련할지 아니면 더 많은 파라미터(모델 크기)로 훈련할지에 대한 추가적인 설계 결정을 도입합니다.

✅ 문장 7-2-4

We focus on the setting in which model parameters are scaled up and training data amount is fixed, matching the canonical approach from the LLaMA series of models.

🔍 단위 해석

우리는 LLaMA 시리즈 모델들의 전형적인 접근 방식을 따라, 훈련 데이터 양은 고정된 채 모델 파라미터를 확장하는 설정에 초점을 맞춥니다.

📖 단락 7-3

Exchanging FLOPs. We use the common formula for pretraining FLOPS $X = 6ND_{pretrain}$ (Hoffmann et al., 2022), and for inference FLOPs, we use $Y = 4ND_{inference}$ (Sardana & Frankle, 2023), which multiplies the standard $2ND_{inference}$ by two to account for the overhead of calling the verifier. Here N represents model parameters, $D_{pretrain}$ is the total tokens used for pretraining, and $D_{inference}$ the total tokens generated at inference. If we multiply N by a factor of M, then both the pretraining and inference FLOPs (due to the cost of greedy decoding with the larger model) increase by a factor of M, giving a total of M(X+Y) FLOPs.

✅ 문장 7-3-1

Exchanging FLOPs. We use the common formula for pretraining FLOPS $X = 6ND_{pretrain}$ (Hoffmann et al., 2022), and for inference FLOPs, we use $Y = 4ND_{inference}$ (Sardana & Frankle, 2023), which multiplies the standard $2ND_{inference}$ by two to account for the overhead of calling the verifier.

🔍 단위 해석

FLOPs 교환. 우리는 사전 학습 FLOPs에 대해 일반적인 공식 $X = 6ND_{pretrain}$ 을 사용하고, 추론 FLOPs에 대해서는 $Y = 4ND_{inference}$ 를 사용하는데, 이는 검증기 호출 오버헤드를 고려하여 표준 $2ND_{inference}$ 에 2를 곱한 것입니다.

✅ 문장 7-3-2

Here N represents model parameters, $D_{pretrain}$ is the total tokens used for pretraining, and $D_{inference}$ the total tokens generated at inference.

🔍 단위 해석

여기서 N은 모델 파라미터 수, $D_{pretrain}$ 은 사전 학습에 사용된 총 토큰 수, $D_{inference}$ 는 추론 시 생성된 총 토큰 수를 나타냅니다.

✔ 문장 7-3-3

If we multiply N by a factor of M, then both the pretraining and inference FLOPs (due to the cost of greedy decoding with the larger model) increase by a factor of M, giving a total of M(X+Y) FLOPs.

🔍 단위 해석

만약 N(파라미터 수)을 M배로 늘리면, 사전 학습 FLOPs와 추론 FLOPs(더 큰 모델로 그리디 디코딩을 하는 비용 때문) 모두 M배로 증가하여, 총 M(X+Y) FLOPs가 됩니다.

📖 단락 7-4

To match the FLOPs between scaling parameters and scaling test-time compute, we multiply the smaller model’s inference compute by $M + \frac{3}{2}(D_{pre}/D_{inf})(M - 1)$. Notably, this multiplier depends on the ratio D_{pre}/D_{inf} . We refer to the inverse of this ratio as $R = D_{inf}/D_{pre}$. Depending on the specific production setting, we should expect very different values of R. In particular, in large scale production settings, we may expect more inference tokens than pretraining tokens, in which case we have $R \gg 1$. On the other hand, in many self-improvement setups, we would likely generate fewer inference tokens than pretraining tokens, giving $R \ll 1$. Therefore, since the scale of test-time compute depends on this ratio, we expect differing conclusions depending on the specific setting.

✔ 문장 7-4-1

To match the FLOPs between scaling parameters and scaling test-time compute, we multiply the smaller model’s inference compute by $M + \frac{3}{2}(D_{pre}/D_{inf})(M - 1)$.

🔍 단위 해석

파라미터 스케일링과 테스트 시점 연산 스케일링 간의 FLOPs를 일치시키기 위해, 우리는 작은 모델의 추론 연산량에 $M + \frac{3}{2}(D_{pre}/D_{inf})(M - 1)$ 를 곱합니다.

💡 보조 설명

- 이 수식은 "큰 모델을 만들고 굴리는 비용"과 "작은 모델을 만들고 엄청나게 많이 굴리는 비용"을 같게 만드는 등식에서 유도된 것입니다.

✔ 문장 7-4-2

Notably, this multiplier depends on the ratio D_{pre}/D_{inf} . We refer to the inverse of this ratio as $R = D_{inf}/D_{pre}$.

🔍 단위 해석

주목할 점은, 이 승수가 D_{pre}/D_{inf} 비율에 의존한다는 것입니다. 우리는 이 비율의 역수를 $R = D_{inf}/D_{pre}$ 라고 부릅니다.

✔ 문장 7-4-3

Depending on the specific production setting, we should expect very different values of R.

🔍 단위 해석

구체적인 프로덕션 환경에 따라 매우 다른 R 값을 예상해야 합니다.

✔ 문장 7-4-4

In particular, in large scale production settings, we may expect more inference tokens than pretraining tokens, in which case we have $R \gg 1$.

🔍 단위 해석

특히 대규모 프로덕션 환경에서는 사전 학습 토큰보다 추론 토큰이 더 많을 것으로 예상되며, 이 경우 $R \gg 1$ 이 됩니다. (추론 부하가 큰 상황)

✔ 문장 7-4-5

On the other hand, in many self-improvement setups, we would likely generate fewer inference tokens than pretraining tokens, giving $R \ll 1$.

🔍 단위 해석

반면, 많은 자가 발전(self-improvement) 설정에서는 사전 학습 토큰보다 적은 추론 토큰을 생성할 가능성이 높으므로 $R \ll 1$ 이 됩니다. (추론 부하가 적은 상황)

✔ 문장 7-4-6

Therefore, since the scale of test-time compute depends on this ratio, we expect differing conclusions depending on the specific setting.

🔍 단위 해석

따라서 테스트 시점 연산의 규모는 이 비율에 달려 있으므로, 구체적인 설정에 따라 서로 다른 결론이 나올 것으로 예상합니다.

단락 7-5

In Figure 8, we use this approach to exchanging test-time and pretraining compute to compare our compute-optimal scaling against scaling up model parameters by a factor of ~ 14 . We conduct comparisons for 3 values of R: 0.08 ($R \ll 1$), 0.40 ($R \sim 1$), and 11 ($R \gg 1$), with each ratio corresponding to an inference budget. Observe that if we only expect to see difficult questions (e.g. bins 4/5 or have a larger $D_{inference}$ (i.e., larger R value), then it is often more effective to allocate compute towards pretraining (e.g. the star is above the line). If instead, we expect mostly easy or intermediate difficulty questions (e.g. bins 1-3 and sometimes 4) or have lower inference requirements (as is the case in self-improvement pipelines), then scaling test-time compute is preferred.

문장 7-5-1

In Figure 8, we use this approach to exchanging test-time and pretraining compute to compare our compute-optimal scaling against scaling up model parameters by a factor of ~ 14 .

단위 해석

Figure 8에서, 우리는 이러한 ‘테스트 시점 연산과 사전 학습 연산 교환’ 접근법을 사용하여, 우리의 연산 최적화 스케일링을 모델 파라미터를 약 14배 키운 경우와 비교합니다.

문장 7-5-2

We conduct comparisons for 3 values of R: 0.08 ($R \ll 1$), 0.40 ($R \sim 1$), and 11 ($R \gg 1$), with each ratio corresponding to an inference budget.

단위 해석

우리는 R의 3가지 값인 0.08 ($R \ll 1$), 0.40 ($R \sim 1$), 11 ($R \gg 1$)에 대해 비교를 수행하며, 각 비율은 특정 추론 예산에 해당합니다.

문장 7-5-3

Observe that if we only expect to see difficult questions (e.g. bins 4/5 or have a larger $D_{inference}$ (i.e., larger R value), then it is often more effective to allocate compute towards pretraining (e.g. the star is above the line).

단위 해석

만약 우리가 어려운 문제(예: 구간 4/5)만 마주할 것으로 예상하거나 더 큰 $D_{inference}$ (즉, 더 큰 R 값)를 가진다면, 연산을 사전 학습 쪽에 할당하는 것이 종종 더 효과적임(예: 별표가 선 위에 위치함)을 관찰할 수 있습니다.

보조 설명

- **[Star is above the line]:** 별표(큰 모델의 성능)가 선(작은 모델 + 추가 연산의 성능)보다 위에 있다는 것은, 큰 모델을 쓰는 게 더 낫다는 뜻입니다.

문장 7-5-4

If instead, we expect mostly easy or intermediate difficulty questions (e.g. bins 1-3 and sometimes 4) or have lower inference requirements (as is the case in self-improvement pipelines), then scaling test-time compute is preferred.

단위 해석

반대로, 우리가 주로 쉽거나 중간 난이도의 문제(예: 구간 1-3 및 때때로 4)를 예상하거나 더 낮은 추론 요구량(자가 발전 파이프라인의 경우처럼)을 가진다면, 테스트 시점 연산을 확장하는 것이 더 선호됩니다.

시각 자료 [Figure 8]

(원문 캡션: **Figure 8:** The tradeoff between pretraining and test-time compute in a FLOPs-matched evaluation. Each line represents the performance of scaling test-time compute with our compute-optimal policy in each oracle difficulty bin for revisions (left) and search (right). The stars represent the greedy pass@1 performance of a base model pretrained with ~ 14 times more parameters. We plot test-time compute budget on the x-axis and stars at three different locations along the x-axis, each corresponding to the FLOPs equivalent point of comparison between scaling parameters and scaling test-time compute for three different inference compute loads (e.g. $R = \frac{D_{inference}}{D_{pretrain}}$). If the star is below the line, this implies that it is more effective to use test-time compute than to scale model parameters, and if the star is above the line this implies that scaling parameters is more effective. We see that on the easy questions or in settings with a lower inference load (e.g. $R \ll 1$, test-time compute can generally outperform scaling model parameters. However, on the harder questions or in settings with a higher inference load (e.g. $R \gg 1$), pretraining is a more effective way to improve performance.)

캡션 해석

FLOPs 매칭 평가에서의 사전 학습과 테스트 시점 연산 간의 트레이드오프. 각 선은 각 오라클 난이도 구간에서 우리의 연산 최적화 정책(왼쪽: 수정, 오른쪽: 탐색)을 사용해 테스트 시점 연산을 확장했을 때의 성능을 나타냅니다. 별표(Star)는 약 14배 더 많은 파라미터로 사전 학습된 베이스 모델의 greedy pass@1 성능을 나타냅니다. 우리는 X축에 테스트 시점 연산 예산을 표시하고, 세 가지 다른 추론 연산 부하(R)에 대해 파라미터 스케일링과 테스트 시점 연산 스케일링 간의 FLOPs 등가 비교 지점에 해당하는 세 위치에 별표를 찍었습니다. 별표가 선 아래에 있으면 모델 파라미터를 키우는 것보다 테스트 시점 연산을 쓰는 게 더 효과적이라는 뜻이고, 별표가 선 위에 있으면 파라미터를 키우는 게 더 낫다는 뜻입니다. 쉬운 문제나 추론 부하가 낮은 설정($R \ll 1$)에서는 일반적으로 테스트 시점 연산이 모델 파라미터 확장을 능가할 수 있음을 볼 수 있습니다. 그러나 어려운 문제나 추론 부하가 높은 설정($R \gg 1$)에서는 사전 학습이 성능을 높이는 더 효과적인 방법입니다.

- 그래프 구조: X축은 추가 연산 예산(Budget), Y축은 정확도입니다.
- 별표(Stars)의 위치: 왼쪽(R=0.08), 중간(R=0.40), 오른쪽(R=11)에 각각 배치되어 있습니다. 오른쪽으로 갈수록 "큰 모델을 굴리는 비용"과 맞먹으려면 "작은 모델로 엄청나게 많이(Budget 높게) 굴려야 함"을 의미합니다.
- 결과 해석:
 - Easy/Medium(파란/초록 선): 별표보다 선이 훨씬 위에 있습니다. → 작은 모델로 여러 번 푸는 게(Test-time compute) 큰 모델 한 번 쓰는 것보다 압도적으로 좋습니다.
 - Hard(주황/빨간 선): 별표가 선 위에 있거나 비슷합니다. → 어려운 문제는 아무리 작은 모델로 여러 번 풀어봤자, 그냥 똑똑한 큰 모델(Pretrained larger model) 한 번 쓰는 게 낫습니다.

8 REPRODUCIBILITY STATEMENT

단락 8-1

Our work does not propose any new method and instead conducts analysis using methods proposed in prior works on the popular MATH benchmark using the PaLM 2 — S^* model. We include extensive details about differences between our work and these prior works that we build on in Sections 5 6 and Appendices C, L, F, G, M, N, J, O including all relevant fine-tuning hyper-parameters used. We also conduct numerous ablations in Sections 5, 6 and Appendix D, E, G, H, N, P, K, and include a handful of examples outputs from our models in Appendix R and Q. We believe that all of these details included in the paper contribute to our work’s reproducibility. The base LLM that we used for our analysis, PaLM 2 — S^* , can be accessed for both finetuning and inference on the Google Cloud Vertex API (it is referred to as Codey on the API).

문장 8-1-1

Our work does not propose any new method and instead conducts analysis using methods proposed in prior works on the popular MATH benchmark using the PaLM 2 — S^* model.

단위 해석

본 연구는 새로운 방법론을 제안하지 않으며, 대신 대중적인 MATH 벤치마크와 PaLM 2 — S^* 모델을 사용하여 선행 연구들에서 제안된 방법들을 활용해 분석을 수행합니다.

문장 8-1-2

We include extensive details about differences between our work and these prior works that we build on in Sections 5 6 and Appendices C, L, F, G, M, N, J, O including all relevant fine-tuning hyper-parameters used.

단위 해석

우리는 섹션 5, 6 및 부록 C, L, F, G, M, N, J, O에 사용된 모든 관련 미세 조정 하이퍼파라미터를 포함하여, 우리가 기반으로 한 선행 연구들과 본 연구 간의 차이점에 대한 광범위한 세부 정보를 포함합니다.

문장 8-1-3

We also conduct numerous ablations in Sections 5, 6 and Appendix D, E, G, H, N, P, K, and include a handful of examples outputs from our models in Appendix R and Q.

단위 해석

또한 섹션 5, 6 및 부록 D, E, G, H, N, P, K에서 수많은 소거 분석(ablations)을 수행하고, 부록 R과 Q에 우리 모델의 출력 예시들을 다수 포함합니다.

문장 8-1-4

We believe that all of these details included in the paper contribute to our work’s reproducibility.

단위 해석

우리는 논문에 포함된 이 모든 세부 사항들이 본 연구의 재현성(reproducibility)에 기여한다고 믿습니다.

문장 8-1-5

The base LLM that we used for our analysis, PaLM 2 — S^* , can be accessed for both finetuning and inference on the Google Cloud Vertex API (it is referred to as Codey on the API).

단위 해석

분석에 사용된 기본 LLM인 PaLM 2 — S^* 는 Google Cloud Vertex API(API 상에서는 Codey로 지칭됨)에서 미세 조정 및 추론 용도로 모두 접근 가능합니다.

📖 단락 8-2

Reproductions and subsequent build-ups. Since the paper submission deadline, two subsequent works from Beeching et al. and Liu et al. (2025) have independently reproduced our main findings using open LLMs (e.g. LLaMA and Qwen models) and using other math benchmarks (e.g. AIME-2024). These results provide additional evidence to support the title and main clam made in this paper that scaling test-time compute can outperform scaling model parameters.

✅ 문장 8-2-1

Reproductions and subsequent build-ups. Since the paper submission deadline, two subsequent works from Beeching et al. and Liu et al. (2025) have independently reproduced our main findings using open LLMs (e.g. LLaMA and Qwen models) and using other math benchmarks (e.g. AIME-2024).

🔍 단위 해석

재현 및 후속 연구. 논문 제출 마감 이후, Beeching et al.과 Liu et al. (2025)의 두 후속 연구가 오픈 LLM(예: LLaMA 및 Qwen 모델)과 다른 수학 벤치마크(예: AIME-2024)를 사용하여 우리의 주요 발견을 독립적으로 재현했습니다.

✅ 문장 8-2-2

These results provide additional evidence to support the title and main clam made in this paper that scaling test-time compute can outperform scaling model parameters.

🔍 단위 해석

이러한 결과들은 테스트 시점 연산을 확장하는 것이 모델 파라미터를 확장하는 것보다 뛰어난 성능을 낼 수 있다는 본 논문의 제목과 주요 주장을 뒷받침하는 추가적 인 증거를 제공합니다.

📖 단락 8-3

We also remark that more recently, OpenAI o1/o3 models (OpenAI, 2024b) and DeepSeek R1 (DeepSeek-AI, 2025) models have demonstrated that training models to output extended chains of thought (similar in spirit to the iterative revision approach we study), can be a highly effective way to enable test time scaling. While we do not analyze this exact approach to test-time scaling in this work, since this approach came out after this paper, these results only further strengthen our main claim about the efficacy of test-time scaling.

✅ 문장 8-3-1

We also remark that more recently, OpenAI o1/o3 models (OpenAI, 2024b) and DeepSeek R1 (DeepSeek-AI, 2025) models have demonstrated that training models to output extended chains of thought (similar in spirit to the iterative revision approach we study), can be a highly effective way to enable test time scaling.

🔍 단위 해석

우리는 또한 최근 OpenAI o1/o3 모델과 DeepSeek R1 모델들이 확장된 사고의 사슬(extended chains of thought, 우리가 연구한 반복적 수정 접근법과 맥락 이 유사함)을 출력하도록 모델을 훈련시키는 것이 테스트 시점 스케일링을 가능하게 하는 매우 효과적인 방법임을 입증했다는 점을 언급합니다.

💡 보조 설명

- **[OpenAI o1/o3, DeepSeek R1]:** 논문 작성 시점 이후에 등장한 최신 모델들로, 추론 과정(Thinking Process)을 길게 생성하여 성능을 높이는 방식을 채택 하고 있습니다. 저자들은 이것이 자신들의 연구 방향(테스트 시점 연산 확장)과 일치함을 강조하고 있습니다.

✅ 문장 8-3-2

While we do not analyze this exact approach to test-time scaling in this work, since this approach came out after this paper, these results only further strengthen our main claim about the efficacy of test-time scaling.

🔍 단위 해석

비록 이 접근법은 본 논문 이후에 나왔기에 우리가 이 정확한 방식의 테스트 시점 스케일링을 분석하지는 않았지만, 이러한 결과들은 테스트 시점 스케일링의 효용성 에 대한 우리의 주요 주장을 더욱 강화해 줄 뿐입니다.

Appendix A. DISCUSSION AND FUTURE WORK

📖 단락 A-1

In this work, we conducted a thorough analysis of the efficacy of different techniques that aim to either improve search against a verifier or to refine an LLM’s proposal distribution, for scaling test-time compute for math reasoning. In general, we found that the efficacy of a given approach heavily correlates with the difficulty of the problem from the perspective of the base LLM’s capabilities. This motivated us to

introduce the notion of “compute-optimal” scaling of test-time computation, which prescribes an adaptive, prompt-dependent strategy to improve performance under a given test-time compute budget. By applying such a compute-optimal scaling strategy, we find that we can improve the efficiency of test-time compute scaling by a factor of $2 - 4x$. When comparing benefits obtained from additional test-time compute against benefits from additional pre-training compute in a FLOPs-matched setting, we show for the first time that using test-time computation with seemingly simple methods (i.e., revisions and search) can already scale well on certain types of prompts, providing gains over spending those FLOPs in pretraining.

✔ 문장 A-1-1

In this work, we conducted a thorough analysis of the efficacy of different techniques that aim to either improve search against a verifier or to refine an LLM’s proposal distribution, for scaling test-time compute for math reasoning.

🔍 단위 해석
본 연구에서 우리는 수학 추론을 위한 테스트 시점 연산 확장을 위해, 검증기를 활용한 탐색을 개선하거나 LLM의 제안 분포를 정제하는 것을 목표로 하는 다양한 기법들의 효용성에 대해 철저한 분석을 수행했습니다.

✔ 문장 A-1-2

In general, we found that the efficacy of a given approach heavily correlates with the difficulty of the problem from the perspective of the base LLM’s capabilities.

🔍 단위 해석
일반적으로, 우리는 특정 접근법의 효용성이 기본 LLM의 능력 관점에서 본 문제의 난이도와 밀접하게 상관관계가 있음을 발견했습니다.

✔ 문장 A-1-3

This motivated us to introduce the notion of “compute-optimal” scaling of test-time computation, which prescribes an adaptive, prompt-dependent strategy to improve performance under a given test-time compute budget.

🔍 단위 해석
이는 우리로 하여금 테스트 시점 연산의 “연산 최적화(compute-optimal)” 스케일링 개념을 도입하도록 동기를 부여했으며, 이는 주어진 테스트 시점 연산 예산하에서 성능을 개선하기 위해 적응적이고 프롬프트 의존적인 전략을 처방합니다.

✔ 문장 A-1-4

By applying such a compute-optimal scaling strategy, we find that we can improve the efficiency of test-time compute scaling by a factor of $2 - 4x$.

🔍 단위 해석
이러한 연산 최적화 스케일링 전략을 적용함으로써, 우리는 테스트 시점 연산 스케일링의 효율성을 2~4배까지 향상시킬 수 있음을 발견했습니다.

✔ 문장 A-1-5

When comparing benefits obtained from additional test-time compute against benefits from additional pre-training compute in a FLOPs-matched setting, we show for the first time that using test-time computation with seemingly simple methods (i.e., revisions and search) can already scale well on certain types of prompts, providing gains over spending those FLOPs in pretraining.

🔍 단위 해석
FLOPs(연산량)를 일치시킨 환경에서 추가적인 테스트 시점 연산으로 얻는 이득과 추가적인 사전 학습 연산으로 얻는 이득을 비교할 때, 우리는 겉보기에 단순한 방법(즉, 수정 및 탐색)을 사용한 테스트 시점 연산이 특정 유형의 프롬프트에서 이미 잘 확장(scale)될 수 있으며, 해당 FLOPs를 사전 학습에 쓰는 것보다 더 큰 이득을 제공한다는 것을 최초로 보여주었습니다.

📖 단락 A-2

Limitations. That said, there are also limitations associated with our study that future work can aim to address. Firstly, while we obtained strong results with beam-search, our lookahead search generally underperformed. It is possible that our lookahead search algorithm could be further optimized by training a PRM verifier with online MCTS training. Future work should explore the degree much PRM search can be improved in this way. Secondly, computing our notion of difficulty requires applying a non-trivial amount of test-time compute. Future work should consider alternative ways of efficiently estimating prompt difficulty. Finally, it is unclear to what extent our findings generalize beyond math and easily verifiable domains more broadly. Furthermore, many limitations of test-time compute are largely unknown; it is possible that in some domains test-time compute may be limited in ways that pretraining is not. While answering this question is out of scope for the present work, we believe it is an important topic for future research.

✔ 문장 A-2-1

Limitations. That said, there are also limitations associated with our study that future work can aim to address.

단위 해석

한계점. 그렇긴 하지만, 우리 연구에는 후속 연구가 해결해야 할 한계점들도 존재합니다.

문장 A-2-2

Firstly, while we obtained strong results with beam-search, our lookahead search generally underperformed.

단위 해석

첫째, 빔 서치(beam-search)로는 강력한 결과를 얻었지만, 룩어헤드 서치(lookahead search)는 전반적으로 성능이 저조했습니다.

문장 A-2-3

It is possible that our lookahead search algorithm could be further optimized by training a PRM verifier with online MCTS training.

단위 해석

우리의 룩어헤드 서치 알고리즘은 온라인 MCTS(Monte Carlo Tree Search) 훈련을 통해 PRM 검증기를 학습시킴으로써 추가적으로 최적화될 가능성이 있습니다.

문장 A-2-4

Future work should explore the degree much PRM search can be improved in this way.

단위 해석

미래의 연구는 이러한 방식으로 PRM 탐색이 얼마나 개선될 수 있는지 탐구해야 합니다.

문장 A-2-5

Secondly, computing our notion of difficulty requires applying a non-trivial amount of test-time compute.

단위 해석

둘째, 우리가 정의한 난이도 개념을 계산하려면 적지 않은 양의 테스트 시점 연산을 적용해야 합니다. (난이도를 알기 위해 문제를 풀어봐야 하는 모순)

문장 A-2-6

Future work should consider alternative ways of efficiently estimating prompt difficulty.

단위 해석

미래의 연구는 프롬프트 난이도를 효율적으로 추정할 수 있는 대안적인 방법을 고려해야 합니다.

문장 A-2-7

Finally, it is unclear to what extent our findings generalize beyond math and easily verifiable domains more broadly.

단위 해석

마지막으로, 우리의 발견이 수학 및 쉽게 검증 가능한 영역을 넘어 얼마나 광범위하게 일반화될지는 불분명합니다.

문장 A-2-8

Furthermore, many limitations of test-time compute are largely unknown; it is possible that in some domains test-time compute may be limited in ways that pretraining is not.

단위 해석

게다가, 테스트 시점 연산의 많은 한계점들은 여전히 미지수입니다; 어떤 도메인에서는 테스트 시점 연산이 사전 학습과는 다른 방식으로 제한될 가능성도 있습니다.

문장 A-2-9

While answering this question is out of scope for the present work, we believe it is an important topic for future research.

단위 해석

이 질문에 답하는 것은 본 연구의 범위를 벗어나지만, 우리는 이것이 미래 연구를 위한 중요한 주제라고 믿습니다.

단락 A-3

Future-work. We believe there are many future research directions that can build on our findings. We describe a few of these directions here. While we focused on improving the test-time compute scaling of two primary mechanisms independently (the verifier and the proposal distribution), future work should investigate how test-time compute scaling can be further improved by combining these approaches or via fundamentally different mechanisms than those explored in this paper. Moreover, our work focused purely on test-time compute scaling. In the future, we envision that the outputs of applying additional test-time compute can be distilled back into the base LLM, enabling an iterative self-improvement loop that operates on open-ended natural language, which we believe is an exciting direction for future work to explore. Moreover, future work should conduct additional scaling-law analysis with respect to test-time compute: 1) how does the scaling of test-time compute improve as pretraining is scaled; and 2) if we were to finetune models on much larger datasets (e.g. of millions of

question answer pairs) how would test-time compute scaling improve? In this setting, how would the cost of fine-tuning affect the balance between scaling up model size vs test-time inference? These are questions that future work can study building on framework built in our work.

✔문장 A-3-1

Future-work. We believe there are many future research directions that can build on our findings.

🔍단위 해석

향후 연구. 우리는 우리의 발견을 기반으로 할 수 있는 많은 미래 연구 방향이 있다고 믿습니다.

✔문장 A-3-2

We describe a few of these directions here.

🔍단위 해석

여기서 그 방향 중 몇 가지를 설명합니다.

✔문장 A-3-3

While we focused on improving the test-time compute scaling of two primary mechanisms independently (the verifier and the proposal distribution), future work should investigate how test-time compute scaling can be further improved by combining these approaches or via fundamentally different mechanisms than those explored in this paper.

🔍단위 해석

우리는 두 가지 주요 메커니즘(검증기와 제안 분포)의 테스트 시점 연산 스케일링을 독립적으로 개선하는 데 집중했으나, 미래의 연구는 이러한 접근법들을 결합하거나 본 논문에서 탐구한 것과는 근본적으로 다른 메커니즘을 통해 어떻게 스케일링을 더욱 개선할 수 있을지 조사해야 합니다.

✔문장 A-3-4

Moreover, our work focused purely on test-time compute scaling.

🔍단위 해석

또한, 우리 연구는 순수하게 테스트 시점 연산 스케일링에만 집중했습니다.

✔문장 A-3-5

In the future, we envision that the outputs of applying additional test-time compute can be distilled back into the base LLM, enabling an iterative self-improvement loop that operates on open-ended natural language, which we believe is an exciting direction for future work to explore.

🔍단위 해석

미래에는 추가적인 테스트 시점 연산을 적용한 결과물(Outputs)이 다시 기본 LLM으로 증류(distilled, 지식 전수)되어, 개방형 자연어에서 작동하는 반복적인 자가 발전 루프(iterative self-improvement loop)를 가능하게 할 것으로 그리며, 이는 미래 연구가 탐구할 흥미로운 방향이라고 믿습니다.

✔문장 A-3-6

Moreover, future work should conduct additional scaling-law analysis with respect to test-time compute: 1) how does the scaling of test-time compute improve as pretraining is scaled; and 2) if we were to finetune models on much larger datasets (e.g. of millions of question answer pairs) how would test-time compute scaling improve?

🔍단위 해석

나아가 미래 연구는 테스트 시점 연산에 관한 추가적인 스케일링 법칙(scaling-law) 분석을 수행해야 합니다: 1) 사전 학습 규모가 커질수록 테스트 시점 연산의 스케일링은 어떻게 개선되는가; 2) 만약 수백만 개의 질의응답 쌍과 같은 훨씬 더 큰 데이터셋으로 모델을 미세 조정한다면 테스트 시점 연산 스케일링은 어떻게 개선될 것인가?

✔문장 A-3-7

In this setting, how would the cost of fine-tuning affect the balance between scaling up model size vs test-time inference?

🔍단위 해석

이러한 설정에서, 미세 조정 비용은 ‘모델 크기 확장’ 대 ‘테스트 시점 추론’ 사이의 균형에 어떤 영향을 미칠 것인가?

✔문장 A-3-8

These are questions that future work can study building on framework built in our work.

🔍단위 해석

이것들은 본 연구에서 구축된 프레임워크를 기반으로 미래 연구가 공부할 수 있는 질문들입니다.

Appendix B. RELATED WORK

📖 단락 B-1

Language model reasoning. Language model performance on challenging mathematical reasoning tasks has rapidly improved in recent years. These improvements can be attributed to three primary factors: 1) running continued pretraining on large corpora of math focused data (Lewkowycz et al., 2022; Azerbayev et al., 2023); 2) improving the LLM proposal distribution by either applying targeted optimization on specific reasoning tasks by finetuning with RL (Zelikman et al., 2022; Singh et al., 2024; Kumar et al., 2024) enabling models to critique and revise their answers iteratively (Qu et al., 2024b; Madaan et al., 2023); 3) enabling LLMs to benefit from additional test-time computation by finetuning verifiers (Cobbe et al., 2021; Lightman et al., 2023; Wang et al., 2023). Our work builds on these second and third lines of research by analyzing the extent to which test-time compute scaling can be improved by 1) refining an LLM’s proposal distribution and 2) conducting search against verifiers.

✅ 문장 B-1-1

Language model reasoning. Language model performance on challenging mathematical reasoning tasks has rapidly improved in recent years.

🔍 단위 해석

언어 모델 추론. 최근 몇 년간 고난도 수학 추론 과제에서 언어 모델의 성능이 급격히 향상되었습니다.

✅ 문장 B-1-2

These improvements can be attributed to three primary factors: 1) running continued pretraining on large corpora of math focused data; 2) improving the LLM proposal distribution by either applying targeted optimization on specific reasoning tasks by finetuning with RL enabling models to critique and revise their answers iteratively; 3) enabling LLMs to benefit from additional test-time computation by finetuning verifiers.

🔍 단위 해석

이러한 향상은 세 가지 주요 요인에 기인합니다: 1) 수학 중심의 대규모 데이터 말뭉치에 대해 지속적인 사전 학습(continued pretraining)을 수행하는 것; 2) RL(강화학습)을 통한 미세 조정으로 특정 추론 작업에 대한 목표 최적화를 적용하거나 모델이 답을 반복적으로 비평 및 수정할 수 있게 함으로써 LLM의 제안 분포를 개선하는 것; 3) 검증기(verifier)를 미세 조정하여 LLM이 추가적인 테스트 시점 연산의 이점을 누리게 하는 것.

✅ 문장 B-1-3

Our work builds on these second and third lines of research by analyzing the extent to which test-time compute scaling can be improved by 1) refining an LLM’s proposal distribution and 2) conducting search against verifiers.

🔍 단위 해석

우리의 연구는 1) LLM의 제안 분포를 정제하고 2) 검증기를 활용한 탐색을 수행함으로써 테스트 시점 연산 스케일링이 어느 정도까지 개선될 수 있는지 분석함으로써, 이 두 번째와 세 번째 연구 흐름을 기반으로 구축되었습니다.

📖 단락 B-2

Analyzing test-time compute scaling. The tradeoff between train-time and test-time compute using Monte-Carlo tree search applied to the board game Hex was previously studied by Jones (2021). We instead focus our analysis on full-scale language model math reasoning problems. A survey work by Villalobos & Atkinson (2023) analyzed the tradeoff between training and inference across a number of domains. Similarly, work by Hassid et al. (2024) explores the trade-off between scaling test-time compute with smaller models and using larger models without additional test-time compute on code completion tasks. However, in each of these prior works, much of the analysis is focused on test-time scaling in settings where the ground-truth answer is known. In contrast, our analysis focuses on the setting when the ground-truth answer is not known. Additionally, a number of works in the RL literature have proposed methods, such as MCTS, which aim to navigate the tradeoff between test-time and training-time compute so as to enable a form of iterative self-play. The findings in our work can be used to help develop similar algorithms that can operate on open-ended natural language.

✅ 문장 B-2-1

Analyzing test-time compute scaling. The tradeoff between train-time and test-time compute using Monte-Carlo tree search applied to the board game Hex was previously studied by Jones (2021).

🔍 단위 해석

테스트 시점 연산 스케일링 분석. 보드게임 Hex에 적용된 몬테카를로 트리 탐색(MCTS)을 사용하여 훈련 시간과 테스트 시간 연산 간의 트레이드오프를 Jones (2021)가 연구한 바 있습니다.

✔ 문장 B-2-2

We instead focus our analysis on full-scale language model math reasoning problems.

🔍 단위 해석

우리는 대신 풀스케일 언어 모델의 수학 추론 문제에 분석의 초점을 맞춥니다.

✔ 문장 B-2-3

A survey work by Villalobos & Atkinson (2023) analyzed the tradeoff between training and inference across a number of domains.

🔍 단위 해석

Villalobos & Atkinson (2023)의 설문 연구는 여러 도메인에 걸쳐 훈련과 추론 간의 트레이드오프를 분석했습니다.

✔ 문장 B-2-4

Similarly, work by Hassid et al. (2024) explores the trade-off between scaling test-time compute with smaller models and using larger models without additional test-time compute on code completion tasks.

🔍 단위 해석

유사하게, Hassid et al. (2024)의 연구는 코드 완성 작업에서 작은 모델로 테스트 시점 연산을 확장하는 것과 추가 테스트 시점 연산 없이 더 큰 모델을 사용하는 것 사이의 트레이드오프를 탐구했습니다.

✔ 문장 B-2-5

However, in each of these prior works, much of the analysis is focused on test-time scaling in settings where the ground-truth answer is known.

🔍 단위 해석

그러나 이들 선행 연구 각각에서, 분석의 대부분은 정답(ground-truth answer)이 알려진 환경에서의 테스트 시점 스케일링에 집중되어 있습니다.

✔ 문장 B-2-6

In contrast, our analysis focuses on the setting when the ground-truth answer is not known.

🔍 단위 해석

이와 대조적으로, 우리의 분석은 정답을 알 수 없는(실제 배포 상황과 같은) 환경에 초점을 맞춥니다.

✔ 문장 B-2-7

Additionally, a number of works in the RL literature have proposed methods, such as MCTS, which aim to navigate the tradeoff between test-time and training-time compute so as to enable a form of iterative self-play.

🔍 단위 해석

또한, 강화학습(RL) 문헌의 많은 연구들이 MCTS와 같은 방법들을 제안했는데, 이는 일종의 반복적 자가 플레이(iterative self-play)를 가능하게 하기 위해 테스트 시점 연산과 훈련 시점 연산 간의 트레이드오프를 조절하는 것을 목표로 합니다.

✔ 문장 B-2-8

The findings in our work can be used to help develop similar algorithms that can operate on open-ended natural language.

🔍 단위 해석

우리 연구의 발견들은 개방형 자연어에서 작동할 수 있는 유사한 알고리즘을 개발하는 데 도움을 줄 수 있습니다.

📖 단락 B-3

Augmenting LLMs with test-time compute. Beyond verifiers and revisions, a number of additional works have proposed alternative methods for enabling LMs to use test-time compute for reasoning. Namely, Wang et al. (2024b) conducts a hierarchical hypothesis search to enable inductive reasoning capabilities. A number of related works have proposed augmenting language models with tools at test-time (Schick et al., 2024; Gou et al., 2023), which can greatly improve their performance on downstream tasks. Several works have proposed methods for learning thought tokens in an unsupervised manner (Zelikkman et al., 2022; Goyal et al., 2024), enabling models to more effectively utilize the additional test-time compute that comes with sampling longer sequences. Finally, Saad-Falcon et al. (2024) explore applying architecture-search based techniques to effectively compose several different test-time scaling techniques. While we focus our analysis on two primary mechanisms by which test-time compute can be scaled in this work (e.g. verifiers and revisions), many of the methods by which we conduct our analysis (e.g. compute optimal scaling according to question difficulty) could, in principle, also be applied to any of these other methods of scaling test-time compute, and we believe that this is an interesting direction for future research.

✔ 문장 B-3-1

Augmenting LLMs with test-time compute. Beyond verifiers and revisions, a number of additional works have proposed alternative methods for enabling LMs to use test-time compute for reasoning.

🔍 단위 해석

테스트 시점 연산으로 **LLM 증강하기**. 검증기와 수정 외에도, 다수의 추가 연구들이 LLM이 추론을 위해 테스트 시점 연산을 사용할 수 있게 하는 대안적인 방법들을 제안했습니다.

✔ 문장 B-3-2

Namely, Wang et al. (2024b) conducts a hierarchical hypothesis search to enable inductive reasoning capabilities.

🔍 단위 해석

구체적으로, Wang et al. (2024b)는 귀납적 추론 능력을 활성화하기 위해 계층적 가설 탐색을 수행합니다.

✔ 문장 B-3-3

A number of related works have proposed augmenting language models with tools at test-time (Schick et al., 2024; Gou et al., 2023), which can greatly improve their performance on downstream tasks.

🔍 단위 해석

다수의 관련 연구들이 테스트 시점에 도구(tools)로 언어 모델을 증강하는 것을 제안했으며, 이는 다운스트림 작업에서의 성능을 크게 향상시킬 수 있습니다.

✔ 문장 B-3-4

Several works have proposed methods for learning thought tokens in an unsupervised manner (Zelikkman et al., 2022; Goyal et al., 2024), enabling models to more effectively utilize the additional test-time compute that comes with sampling longer sequences.

🔍 단위 해석

몇몇 연구들은 비지도 방식으로 '생각 토큰(thought tokens)'을 학습하는 방법을 제안하여, 모델이 더 긴 시퀀스를 샘플링함으로써 발생하는 추가적인 테스트 시점 연산을 더 효과적으로 활용할 수 있게 했습니다.

✔ 문장 B-3-5

Finally, Saad-Falcon et al. (2024) explore applying architecture-search based techniques to effectively compose several different test-time scaling techniques.

🔍 단위 해석

마지막으로, Saad-Falcon et al. (2024)는 여러 다른 테스트 시점 스케일링 기법들을 효과적으로 조합하기 위해 아키텍처 탐색(architecture-search) 기반 기술을 적용하는 것을 탐구합니다.

✔ 문장 B-3-6

While we focus our analysis on two primary mechanisms by which test-time compute can be scaled in this work (e.g. verifiers and revisions), many of the methods by which we conduct our analysis (e.g. compute optimal scaling according to question difficulty) could, in principle, also be applied to any of these other methods of scaling test-time compute, and we believe that this is an interesting direction for future research.

🔍 단위 해석

우리는 본 연구에서 테스트 시점 연산을 확장할 수 있는 두 가지 주요 메커니즘(검증기와 수정)에 분석을 집중했지만, 우리의 분석 방법들(예: 문제 난이도에 따른 연산 최적화 스케일링) 중 다수는 원칙적으로 이러한 다른 테스트 시점 스케일링 방법들에도 적용될 수 있으며, 우리는 이것이 미래 연구를 위한 흥미로운 방향이라고 믿습니다.

Appendix C. SEARCH ALGORITHM DETAILS

📖 단락 C-1

Below we include additional details for each of our search algorithms in Section 5.

Best-of-N weighted. We sample N answers independently from the base LLM and then select the best answer according to the PRM’s final answer judgment.

✔ 문장 C-1-1

Below we include additional details for each of our search algorithms in Section 5.

🔍 단위 해석

아래에 섹션 5에 나온 각 탐색 알고리즘에 대한 추가 세부 사항을 포함합니다.

✔ 문장 C-1-2

Best-of-N weighted. We sample N answers independently from the base LLM and then select the best answer according to the PRM’s final answer judgment.

🔍 단위 해석

가중 **Best-of-N.** 기본 LLM에서 N개의 답안을 독립적으로 샘플링한 다음, PRM의 최종 답안 판단에 따라 가장 좋은 답을 선택합니다.

📖 단락 C-2

Beam search. Beam search optimizes the PRM by searching over its per-step predictions. Our implementation is similar to BFS-V (Yao et al., 2023; Feng et al., 2024). Concretely, we consider a fixed number of beams N and a beam width M. We then run the following steps:

1. sample N initial predictions for the first step in the solution
2. score the generated steps according to the PRM’s predicted step-wise reward-to-go estimate (which also corresponds to the total reward from the prefix since the reward is sparse in this setting)
3. filter for only the top N/M highest scoring steps
4. now from each candidate, sample M proposals from the next step, resulting in a total of $N/M \times M$ candidate prefixes again. Then repeat steps 2-4 again.

We run this algorithm until the end of a solution or the maximum number of rounds of beam expansion are attained (40 in our case). We conclude the search with N final answer candidates, to which we apply best-of-N weighted selection described above to make our final answer prediction.

✔ 문장 C-2-1

Beam search. Beam search optimizes the PRM by searching over its per-step predictions.

🔍 단위 해석

빔 서치. 빔 서치는 PRM의 단계별 예측값을 탐색하여 PRM 활용을 최적화합니다.

✔ 문장 C-2-2

Our implementation is similar to BFS-V (Yao et al., 2023; Feng et al., 2024).

🔍 단위 해석

우리의 구현은 BFS-V와 유사합니다.

✔ 문장 C-2-3

Concretely, we consider a fixed number of beams N and a beam width M. We then run the following steps:

🔍 단위 해석

구체적으로, 우리는 고정된 빔 개수 N과 빔 폭 M을 고려합니다. 그런 다음 다음 단계들을 실행합니다:

✔ 문장 C-2-4

1. sample N initial predictions for the first step in the solution

🔍 단위 해석

1. 해답의 첫 번째 단계에 대해 N개의 초기 예측을 샘플링합니다.

✔ 문장 C-2-5

2. score the generated steps according to the PRM’s predicted step-wise reward-to-go estimate (which also corresponds to the total reward from the prefix since the reward is sparse in this setting)

🔍 단위 해석

2. PRM이 예측한 단계별 ‘보상 잔여값(reward-to-go)’ 추정치에 따라 생성된 단계들의 점수를 매깁니다. (이 설정에서는 보상이 희소(sparse)하므로, 이는 접두어(prefix)로부터 얻은 총보상과도 일치합니다.)

✔ 문장 C-2-6

3. filter for only the top N/M highest scoring steps

🔍 단위 해석

3. 점수가 가장 높은 상위 N/M개의 단계만 남기고 필터링합니다.

✔ 문장 C-2-7

4. now from each candidate, sample M proposals from the next step, resulting in a total of $N/M \times M$ candidate prefixes again. Then repeat steps 2-4 again.

🔍 단위 해석

4. 이제 각 후보로부터 다음 단계에 대한 M 개의 제안을 샘플링하여, 다시 총 $N/M \times M (=N)$ 개의 후보 접두어를 만듭니다. 그런 다음 2-4 단계를 반복합니다.

✔ 문장 C-2-8

We run this algorithm until the end of a solution or the maximum number of rounds of beam expansion are attained (40 in our case).

🔍 단위 해석

우리는 해답이 끝나거나 최대 빔 확장 라운드 수(우리 경우에는 40회)에 도달할 때까지 이 알고리즘을 실행합니다.

✔ 문장 C-2-9

We conclude the search with N final answer candidates, to which we apply best-of- N weighted selection described above to make our final answer prediction.

🔍 단위 해석

우리는 N 개의 최종 답안 후보로 탐색을 마치고, 여기에 앞서 설명한 가중 Best-of- N 선택을 적용하여 최종 정답 예측을 수행합니다.

📖 단락 C-3

Lookahead search. Lookahead search modifies how beam search evaluates individual steps. It uses lookahead rollouts to improve the accuracy of the PRM's value estimation in each step of the search process. Specifically, at each step in the beam search, rather than using the PRM score at the current step to select the top candidates, lookahead search performs a simulation, rolling out up to k steps further while stopping early if the end of solution is reached. To minimize variance in the simulation rollout, we perform rollouts using temperature 0. The PRM's prediction at the end of this rollout is then used to score the current step in the beam search. That is, in other words, we can view beam search as a special case of lookahead search with $k=0$. Given an accurate PRM, increasing k should improve the accuracy of the per-step value estimates at the cost of additional compute. Also note that this version of lookahead search is a special case of MCTS, wherein the stochastic elements of MCTS, designed to facilitate exploration, are removed since the PRM is already trained and is frozen. These stochastic elements are largely useful for learning the value function (which we've already learned with our PRM), but less useful at test-time when we want to exploit rather than explore. Therefore, lookahead search is largely representative of how MCTS-style methods would be applied at test-time.

✔ 문장 C-3-1

Lookahead search. Lookahead search modifies how beam search evaluates individual steps.

🔍 단위 해석

룩어헤드 서치. 룩어헤드 서치는 빔 서치가 개별 단계를 평가하는 방식을 수정합니다.

✔ 문장 C-3-2

It uses lookahead rollouts to improve the accuracy of the PRM's value estimation in each step of the search process.

🔍 단위 해석

이는 탐색 과정의 각 단계에서 PRM 가치 추정의 정확도를 높이기 위해 룩어헤드 롤아웃(미리보기 시뮬레이션)을 사용합니다.

✔ 문장 C-3-3

Specifically, at each step in the beam search, rather than using the PRM score at the current step to select the top candidates, lookahead search performs a simulation, rolling out up to k steps further while stopping early if the end of solution is reached.

🔍 단위 해석

구체적으로, 빔 서치의 각 단계에서 상위 후보를 선택하기 위해 현재 단계의 PRM 점수를 사용하는 대신, 룩어헤드 서치는 최대 k 단계까지 더 진행해 보는 시뮬레이션을 수행하며, 해답이 끝나면 조기에 중단합니다.

✔ 문장 C-3-4

To minimize variance in the simulation rollout, we perform rollouts using temperature 0.

🔍 단위 해석

시뮬레이션 롤아웃의 분산(변동성)을 최소화하기 위해, 우리는 온도(temperature) 0을 사용하여 롤아웃을 수행합니다. (가장 확률 높은 토큰만 선택)

✔ 문장 C-3-5

The PRM's prediction at the end of this rollout is then used to score the current step in the beam search.

단위 해석
이 룰아웃 끝에서의 PRM 예측값이 빔 서치상의 현재 단계 점수를 매기는 데 사용됩니다.

문장 C-3-6

That is, in other words, we can view beam search as a special case of lookahead search with $k=0$.

단위 해석
즉 다시 말해, 우리는 빔 서치를 $k=0$ 인 룰어헤드 서치의 특수 사례로 볼 수 있습니다.

문장 C-3-7

Given an accurate PRM, increasing k should improve the accuracy of the per-step value estimates at the cost of additional compute.

단위 해석
정확한 PRM이 주어졌다면, k 를 늘리는 것은 추가적인 연산 비용을 대가로 단계별 가치 추정의 정확도를 향상시킬 것입니다.

문장 C-3-8

Also note that this version of lookahead search is a special case of MCTS, wherein the stochastic elements of MCTS, designed to facilitate exploration, are removed since the PRM is already trained and is frozen.

단위 해석
또한 이 버전의 룰어헤드 서치는 MCTS의 특수한 사례로 볼 수 있는데, PRM이 이미 훈련되어 고정(frozen)되었기 때문에 탐험(exploration)을 촉진하기 위해 설계된 MCTS의 확률적 요소들이 제거된 형태입니다.

문장 C-3-9

These stochastic elements are largely useful for learning the value function (which we’ve already learned with our PRM), but less useful at test-time when we want to exploit rather than explore.

단위 해석
이러한 확률적 요소들은 가치 함수를 학습할 때는 유용하지만(우리는 PRM으로 이미 학습했음), 탐험보다는 착취(exploit, 최적 경로 활용)를 원하는 테스트 시점에는 덜 유용합니다.

문장 C-3-10

Therefore, lookahead search is largely representative of how MCTS-style methods would be applied at test-time.

단위 해석
따라서 룰어헤드 서치는 MCTS 스타일의 방법론이 테스트 시점에 어떻게 적용될지를 대변하는 대표적인 방식입니다.

Appendix D. ADDITIONAL REVISION RESULTS

단락 D-1

In Figure 9 left, we plot the pass@1 for our revision model at each revision step. We see that pass@1 gradually improves after each step. In Figure 9 right, we compare performance when generating N initial answers in parallel from our revision model, versus generating N revisions sequentially, with the model. When using both the verifier and majority voting to select the answer, we see that generating answers sequentially with the revision model narrowly outperforms generating them in parallel.

문장 D-1-1

In Figure 9 left, we plot the pass@1 for our revision model at each revision step.

단위 해석
Figure 9 왼쪽에서, 우리는 각 수정 단계(revision step)별로 수정 모델의 pass@1(한 번에 맞출 확률)을 도표로 나타냅니다.

문장 D-1-2

We see that pass@1 gradually improves after each step.

단위 해석
우리는 각 단계가 지날 때마다 pass@1이 점진적으로 향상되는 것을 볼 수 있습니다.

✔ 문장 D-1-3

In Figure 9 right, we compare performance when generating N initial answers in parallel from our revision model, versus generating N revisions sequentially, with the model.

🔍 단위 해석

Figure 9 오른쪽에서, 우리는 수정 모델로부터 N개의 초기 답안을 병렬로 생성할 때와 N개의 수정을 순차적으로 생성할 때의 성능을 비교합니다.

✔ 문장 D-1-4

When using both the verifier and majority voting to select the answer, we see that generating answers sequentially with the revision model narrowly outperforms generating them in parallel.

🔍 단위 해석

답안을 선택하기 위해 검증기와 다수결 투표를 모두 사용했을 때, 수정 모델로 답안을 순차적으로 생성하는 것이 병렬로 생성하는 것보다 근소하게(narrowly) 앞서는 것을 볼 수 있습니다.

🖼️ 시각 자료 [Figure 9]

(원문 캡션: **Figure 9:** Left: Our revision model’s pass@1 at each revision step. Pass@1 gradually improves after each revision step, even improving beyond the 4 revision steps that it was trained for. We estimate pass@1 at each step by averaging over the performance of 4 revision trajectories of length 64 for each question in the test-set. Right: Sequential vs parallel sampling from the revision model. Comparing performance when generating N initial answers in parallel from our revision model, versus generating N revisions sequentially, with the model. To account for the cost of querying the verifier relative to majority voting, we shift the curves which involve a verifier over by one point. When using both the verifier and majority voting to select the answer, we see that generating answers sequentially with the revision model narrowly outperforms generating them in parallel.)

🔍 캡션 해석

왼쪽: 각 수정 단계에서의 우리 수정 모델의 pass@1. Pass@1은 각 수정 단계 후 점진적으로 향상되며, 심지어 훈련된 4단계를 넘어서도 향상됩니다... 오른쪽: 수정 모델에서의 순차적 대 병렬 샘플링. 수정 모델에서 N개의 초기 답안을 병렬로 생성할 때와 N개의 수정을 순차적으로 생성할 때의 성능 비교... 답안 선택에 검증기와 다수결 투표를 모두 사용할 때, 순차적 생성이 병렬 생성보다 근소하게 앞서는 것을 볼 수 있습니다.

📄 PDF 시각 분석

PDF 12페이지의 **Figure 9** 분석:

- 1. **Left Graph:** X축은 수정 단계(Revision Step), Y축은 정확도(Pass@1)입니다. 1에서 8까지 단계가 진행될수록 그래프가 꾸준히 우상향합니다. 이는 모델이 반복적으로 고치면서 정답률을 스스로 높여간다는 것을 의미합니다.
- 2. **Right Graph:** X축은 샘플 수(지수 스케일), Y축은 정확도입니다. 같은 색상 계열끼리 비교했을 때 실선(Sequential)이 점선(Parallel)보다 항상 위에 위치합니다. 이는 같은 개수라면 병렬로 여러 개 뽑는 것보다 꼬리에 꼬리를 무는 수정 방식이 더 낫다는 것을 시각적으로 보여줍니다.

📖 단락 D-2

We plot additional results for majority voting selection using our PaLM 2 — S^* revision model in Figure 10. With majority selection, we see largely similar trends to those found in Figure 7 for verifier selection.

✔ 문장 D-2-1

We plot additional results for majority voting selection using our PaLM 2 — S^* revision model in Figure 10.

🔍 단위 해석

우리는 PaLM 2 — S^* 수정 모델을 사용한 다수결 투표 선택에 대한 추가 결과를 Figure 10에 도식화합니다.

✔ 문장 D-2-2

With majority selection, we see largely similar trends to those found in Figure 7 for verifier selection.

🔍 단위 해석

다수결 선택에서도 검증기 선택을 사용했던 Figure 7에서 발견된 것과 대체로 유사한 추세를 볼 수 있습니다.

🖼️ 시각 자료 [Figure 10]

(원문 캡션: **Figure 10:** Varying the ratio of generation budget allocated to sequential versus parallel samples, using majority voting to select the answer, rather than the verifier. Left: Each line represents a fixed generation budget as the ratio is changed. We see that similar to the verifier case, in the majority case, there exists an ideal ratio of sequential to parallel test-time compute at a given budget. Right: Analyzing performance across difficulty bins, we see that the easier questions are mostly invariant to the ratio of sequential to parallel, whereas on the harder questions there is an ideal ratio of sequential to parallel test-time compute.)

🔍 **캡션 해석**

검증기 대신 다수결 투표를 사용하여 답을 선택할 때, 순차적 대 병렬 샘플에 할당된 생성 예산 비율을 변화시킴. 왼쪽: 각 선은 비율이 변할 때의 고정된 생성 예산을 나타냅니다. 검증기 케이스와 유사하게 다수결 케이스에서도 주어진 예산에서 이상적인 순차 대 병렬 비율이 존재함을 알 수 있습니다. 오른쪽: 난이도 구간별 성능 분석 결과, 쉬운 문제들은 순차 대 병렬 비율에 거의 영향을 받지 않는(invariant) 반면, 더 어려운 문제들에서는 이상적인 비율이 존재함을 알 수 있습니다.

 PDF 시각 분석

PDF 12페이지의 **Figure 10** 분석:

- **Left Graph:** Figure 7과 마찬가지로, 예산이 클수록(보라색 선) 그래프가 중간에서 봉긋 솟아오르는 형태를 보입니다. 즉, 검증기 없이 다수결만 쓰더라도 ‘적절한 비율 섞기’ 전략이 유효합니다.
- **Right Graph:** 난이도별 분석에서 어려운 문제(Level 4, 빨간색)는 중간 지점에서 피크를 찍는 반면, 쉬운 문제(Level 1, 파란색)는 평평하거나 오른쪽(순차 위주)이 약간 더 높습니다. 어려운 문제일수록 탐색(Parallel)과 수정(Sequential)의 균형이 중요합니다.

Appendix E. DIFFICULTY BINS

단락 E-1

Oracle difficulty bins. We compute our oracle difficulty bins by obtaining the ground-truth pass@1 correctness rate for each question, and then using this statistic to bin questions into quantiles representing 5 distinct difficulty bins. There are in total 500 questions in the test-set. Difficulty levels 1, 2, and 3 all have 100 questions in them. Difficulty levels 4 and 5 have 105 and 95 questions respectively. This imbalance in the last two bins is merely due to a boundary condition/ties in the quantile computation, causing one bin to inherit slightly more questions than the other.

✅ **문장 E-1-1**

Oracle difficulty bins. We compute our oracle difficulty bins by obtaining the ground-truth pass@1 correctness rate for each question, and then using this statistic to bin questions into quantiles representing 5 distinct difficulty bins.

🔍 **단위 해석**

오라클 난이도 구간. 우리는 각 문제에 대한 실제 정답 기반의 pass@1 정답률을 구한 다음, 이 통계량을 사용하여 문제들을 5개의 서로 다른 난이도 구간을 나타내는 분위수(quantiles)로 나누어 오라클 난이도 구간을 계산합니다.

✅ **문장 E-1-2**

There are in total 500 questions in the test-set.

🔍 **단위 해석**

테스트 세트에는 총 500개의 문제가 있습니다.

✅ **문장 E-1-3**

Difficulty levels 1, 2, and 3 all have 100 questions in them. Difficulty levels 4 and 5 have 105 and 95 questions respectively.

🔍 **단위 해석**

난이도 레벨 1, 2, 3에는 모두 100개의 문제가 포함되어 있습니다. 난이도 레벨 4와 5에는 각각 105개와 95개의 문제가 있습니다.

✅ **문장 E-1-4**

This imbalance in the last two bins is merely due to a boundary condition/ties in the quantile computation, causing one bin to inherit slightly more questions than the other.

🔍 **단위 해석**

마지막 두 구간의 이러한 불균형은 단지 분위수 계산 시의 경계 조건이나 동점(ties) 처리 때문이며, 이로 인해 한 구간이 다른 구간보다 약간 더 많은 문제를 가져가게 된 것뿐입니다.

단락 E-2

Predicted difficulty bins. We compute difficulty bins without oracle ground-truth correctness information by averaging the PRM final-answer score over 2048 samples on each question, so as to obtain a value estimate corresponding to the question. Similar to the oracle case, we then bin the value for each question in the test-set into five quintiles (using the same procedure as the oracle difficulty bins). We refer to this as “predicted difficulty”. Each of the bins has 100 questions in this case. Technically this procedure is extremely costly because it requires generating many samples. While we do not account for this cost in our analysis, in a practical production setting, this cost would be problematic. A more efficient approach would be to finetune a model to predict correctness directly, given the question. We do not explore this in our work, but leave such exploration of cheaper methods of estimating difficulty to future work.

✔ 문장 E-2-1

Predicted difficulty bins. We compute difficulty bins without oracle ground-truth correctness information by averaging the PRM final-answer score over 2048 samples on each question, so as to obtain a value estimate corresponding to the question.

🔍 단위 해석

예측 난이도 구간. 우리는 오라클 정답 정보 없이, 각 문제에 대해 2048개 샘플의 PRM 최종 답안 점수를 평균 내어 해당 문제에 상응하는 가치 추정치를 얻음으로써 난이도 구간을 계산합니다.

✔ 문장 E-2-2

Similar to the oracle case, we then bin the value for each question in the test-set into five quintiles (using the same procedure as the oracle difficulty bins).

🔍 단위 해석

오라클 케이스와 유사하게, 우리는 테스트 세트의 각 문제에 대한 이 값을 5개의 5분위수(quintiles)로 구간화합니다 (오라클 난이도 구간과 동일한 절차 사용).

✔ 문장 E-2-3

We refer to this as “predicted difficulty”. Each of the bins has 100 questions in this case.

🔍 단위 해석

우리는 이를 "예측 난이도"라고 부릅니다. 이 경우 각 구간은 100개의 문제를 포함합니다.

✔ 문장 E-2-4

Technically this procedure is extremely costly because it requires generating many samples.

🔍 단위 해석

기술적으로 이 절차는 많은 샘플 생성을 요구하기 때문에 비용이 매우 많이 듭니다.

✔ 문장 E-2-5

While we do not account for this cost in our analysis, in a practical production setting, this cost would be problematic.

🔍 단위 해석

우리는 분석에서 이 비용을 고려하지 않았지만, 실제 프로덕션 환경에서는 이 비용이 문제가 될 것입니다.

✔ 문장 E-2-6

A more efficient approach would be to finetune a model to predict correctness directly, given the question.

🔍 단위 해석

더 효율적인 접근 방식은 질문이 주어졌을 때 정답 여부를 직접 예측하도록 모델을 미세 조정하는 것일 겁니다.

✔ 문장 E-2-7

We do not explore this in our work, but leave such exploration of cheaper methods of estimating difficulty to future work.

🔍 단위 해석

우리는 본 연구에서 이를 탐구하지 않지만, 난이도를 추정하는 더 저렴한 방법에 대한 탐구는 미래의 과제로 남겨둡니다.

📖 단락 E-3

In Figure 12 we plot PRM-search results using our predicted (non-oracle) difficulty bins, and in Figure 11 we plot the corresponding revision results. We see that in both settings these predicted bins demonstrate similar trends to the oracle bins.

✔ 문장 E-3-1

In Figure 12 we plot PRM-search results using our predicted (non-oracle) difficulty bins, and in Figure 11 we plot the corresponding revision results.

🔍 단위 해석

Figure 12에서 우리는 예측된(비 오라클) 난이도 구간을 사용한 PRM 탐색 결과를 도표로 나타내고, Figure 11에서는 그에 상응하는 수정 결과를 도표로 나타냅니다.

✔ 문장 E-3-2

We see that in both settings these predicted bins demonstrate similar trends to the oracle bins.

단위 해석

우리는 두 설정 모두에서 이 예측된 구간들이 오라클 구간과 유사한 추세를 보여준다는 것을 확인했습니다.

시각 자료 [Figure 11 & 12]

(원문 캡션: **Figure 11:** Using our PaLM 2 — S^* PRM to compute difficulty bins without ground truth correctness information for revisions...
Figure 12: Using our PaLM 2 — S^* PRM to compute difficulty bins without ground truth correctness information for PRM search...)

캡션 해석

Figure 11: 정답 정보 없이 PaLM 2 — S^* PRM을 사용하여 수정(revisions)을 위한 난이도 구간을 계산한 결과...

Figure 12: 정답 정보 없이 PaLM 2 — S^* PRM을 사용하여 PRM 탐색(search)을 위한 난이도 구간을 계산한 결과...

PDF 시각 분석

PDF 13페이지의 **Figure 11 & 12** 분석:

- Figure 11 (Revisions with Predicted Difficulty):** 오라클 난이도를 사용했던 Figure 7(Right)이나 Figure 10(Right)과 패턴이 매우 흡사합니다. 쉬운 문제(Level 1)는 오른쪽(순차)이 높고, 어려운 문제(Level 4)는 중간이 높습니다. 이는 PRM 예측만으로도 문제 난이도를 꽤 정확히 분류하여 최적 전략을 짤 수 있음을 의미합니다.
- Figure 12 (Search with Predicted Difficulty):** 오라클 난이도를 쓴 Figure 3(Right)과 비교했을 때 거의 동일한 양상을 보입니다. 쉬운 문제에서는 빔 서치의 성능 저하가 관찰되고, 중간/어려운 문제에서는 빔 서치가 우세합니다.

Appendix F. PRM TRAINING DETAILS

단락 F-1

Originally PRM training (Uesato et al., 2022; Lightman et al., 2023) used human crowd-worker labels. While Lightman et al. (2023) released their PRM training data (i.e., the PRM800k dataset), we found this data to be largely ineffective for us. We found that it was easy to exploit a PRM trained on this dataset via even naïve strategies such as best-of-N sampling. We hypothesize that this is likely a result of the distribution shift between the GPT-4 generated samples in their dataset and our PaLM 2 models. Rather than proceeding with the expensive process of collecting crowd-worker PRM labels for our PaLM 2 models, we instead apply the approach of Wang et al. (2023) to supervise PRMs without human labels, using estimates of per-step correctness obtained from running Monte Carlo rollouts from each step in the solution. Our PRM’s per-step predictions therefore correspond to value estimates of reward-to-go for the base model’s sampling policy, similar to recent work. We also compare to an ORM baseline (Appendix H) but found that our PRM consistently outperforms the ORM. Hence, all of the search experiments in this section use a PRM model.

문장 F-1-1

Originally PRM training (Uesato et al., 2022; Lightman et al., 2023) used human crowd-worker labels.

단위 해석

원래 PRM 훈련(Uesato et al., 2022; Lightman et al., 2023)은 인간 크라우드 워커의 라벨을 사용했습니다.

문장 F-1-2

While Lightman et al. (2023) released their PRM training data (i.e., the PRM800k dataset), we found this data to be largely ineffective for us.

단위 해석

Lightman et al. (2023)이 그들의 PRM 훈련 데이터(즉, PRM800k 데이터셋)를 공개했지만, 우리는 이 데이터가 우리에게는 대체로 효과적이지 않다는 것을 발견했습니다.

문장 F-1-3

We found that it was easy to exploit a PRM trained on this dataset via even naïve strategies such as best-of-N sampling.

단위 해석

우리는 이 데이터셋으로 훈련된 PRM이 Best-of-N 샘플링과 같은 단순한 전략을 통해서도 쉽게 악용(exploit)될 수 있음을 발견했습니다. (점수만 높고 실제 정답은 틀리는 현상 등)

문장 F-1-4

We hypothesize that this is likely a result of the distribution shift between the GPT-4 generated samples in their dataset and our PaLM 2 models.

단위 해석

우리는 이것이 그들 데이터셋의 GPT-4 생성 샘플과 우리의 PaLM 2 모델 사이의 분포 변화(distribution shift) 때문일 가능성이 높다고 가설을 세웁니다.

✅ 문장 F-1-5

Rather than proceeding with the expensive process of collecting crowd-worker PRM labels for our PaLM 2 models, we instead apply the approach of Wang et al. (2023) to supervise PRMs without human labels, using estimates of per-step correctness obtained from running Monte Carlo rollouts from each step in the solution.

🔍 단위 해석

우리 PaLM 2 모델을 위해 비싼 클라우드 워커 PRM 라벨 수집 과정을 진행하는 대신, 우리는 Wang et al. (2023)의 접근 방식을 적용하여 인간 라벨 없이 PRM을 지도 학습시켰습니다. 이는 해답의 각 단계에서 몬테카를로 롤아웃을 실행하여 얻은 단계별 정답 여부 추정치를 사용하는 방식입니다.

✅ 문장 F-1-6

Our PRM’s per-step predictions therefore correspond to value estimates of reward-to-go for the base model’s sampling policy, similar to recent work.

🔍 단위 해석

따라서 우리 PRM의 단계별 예측값은 최근 연구와 유사하게 기본 모델의 샘플링 정책에 대한 '보상 잔여값(reward-to-go)'의 가치 추정치에 해당합니다.

✅ 문장 F-1-7

We also compare to an ORM baseline (Appendix H) but found that our PRM consistently outperforms the ORM. Hence, all of the search experiments in this section use a PRM model.

🔍 단위 해석

우리는 또한 ORM 베이스라인과도 비교했지만(부록 H), PRM이 일관되게 ORM보다 성능이 우수함을 확인했습니다. 따라서 이 섹션의 모든 탐색 실험은 PRM 모델을 사용합니다.

📖 단락 F-2

We finetune our PRM as a binary classifier, where the model predicts a value between 0 and 1 at each step in the solution. We train the model with soft values obtained from the monte-carlo rollouts, using a binary cross entropy loss function (e.g. $-(y \log(\hat{y}) + (1 - y) \log(1 - \hat{y}))$ where y corresponds to the soft ground-truth value and $\hat{y}$ the model’s predicted value). We finetune the model base model using the AdamW optimizer, with lr 3e-5, batch size 128, dropout 0.05, and Adam betas (0.9, 0.95). We conduct early stopping, selecting the checkpoint with the lowest validation loss on a random held-out validation set, consisting of 10% of the questions in the original PRM800k training split.

✅ 문장 F-2-1

We finetune our PRM as a binary classifier, where the model predicts a value between 0 and 1 at each step in the solution.

🔍 단위 해석

우리는 PRM을 이진 분류기(binary classifier)로 미세 조정하며, 모델은 해답의 각 단계에서 0과 1 사이의 값을 예측합니다.

✅ 문장 F-2-2

We train the model with soft values obtained from the monte-carlo rollouts, using a binary cross entropy loss function (e.g. $-(y \log(\hat{y}) + (1 - y) \log(1 - \hat{y}))$ where y corresponds to the soft ground-truth value and $\hat{y}$ the model’s predicted value).

🔍 단위 해석

우리는 몬테카를로 롤아웃에서 얻은 소프트 값(soft values, 0과 1 사이의 실수)을 사용하여 모델을 훈련시키며, 이때 이진 교차 엔트로피 손실 함수를 사용합니다. (여기서 y 는 소프트 정답 값에 해당하고 $\hat{y}$ 는 모델의 예측값입니다.)

✅ 문장 F-2-3

We finetune the model base model using the AdamW optimizer, with lr 3e-5, batch size 128, dropout 0.05, and Adam betas (0.9, 0.95).

🔍 단위 해석

우리는 학습률 3e-5, 배치 크기 128, 드롭아웃 0.05, Adam 베타 (0.9, 0.95)로 설정된 AdamW 옵티마이저를 사용하여 기본 모델을 미세 조정합니다.

✅ 문장 F-2-4

We conduct early stopping, selecting the checkpoint with the lowest validation loss on a random held-out validation set, consisting of 10% of the questions in the original PRM800k training split.

🔍 단위 해석

우리는 조기 종료(early stopping)를 수행하며, 원래 PRM800k 훈련 분할의 질문 중 10%로 구성된 무작위 보류(held-out) 검증 세트에서 가장 낮은 검증 손실을 보인 체크포인트를 선택합니다.

단락 F-3

We finetune the PRM on 16 samples per question from the corresponding few-shot prompted base model. At each step, we use 16 monte-carlo rollouts, using the same base model and prompt, to estimate the step-level value. We filter out all samples which fail to output a valid, parsable final answer from the training data, as we found these to hurt PRM performance in initial experiments. When generating the samples, the base model is prompted to output answers in newline separated step-by-step format, as done in Lightman et al. (2023). We then separate each of the answers into steps using a simple newline splitting procedure. We include details about our prompt in Appendix J.

문장 F-3-1

We finetune the PRM on 16 samples per question from the corresponding few-shot prompted base model.

단위 해석

우리는 퓨샷 프롬프팅된 기본 모델에서 문제당 16개의 샘플을 추출하여 PRM을 미세 조정합니다.

문장 F-3-2

At each step, we use 16 monte-carlo rollouts, using the same base model and prompt, to estimate the step-level value.

단위 해석

각 단계에서, 우리는 단계별 가치를 추정하기 위해 동일한 기본 모델과 프롬프트를 사용하여 16번의 몬테카를로 롤아웃을 사용합니다.

문장 F-3-3

We filter out all samples which fail to output a valid, parsable final answer from the training data, as we found these to hurt PRM performance in initial experiments.

단위 해석

우리는 유효하고 파싱 가능한 최종 답안을 출력하지 못한 모든 샘플을 훈련 데이터에서 필터링(제외)했는데, 초기 실험에서 이러한 샘플들이 PRM 성능을 저하시키는 것을 확인했기 때문입니다.

문장 F-3-4

When generating the samples, the base model is prompted to output answers in newline separated step-by-step format, as done in Lightman et al. (2023).

단위 해석

샘플을 생성할 때, 기본 모델은 Lightman et al. (2023)에서와 같이 줄바꿈으로 구분된 단계별(step-by-step) 형식으로 답안을 출력하도록 프롬프트됩니다.

문장 F-3-5

We then separate each of the answers into steps using a simple newline splitting procedure.

단위 해석

그런 다음 우리는 간단한 줄바꿈 분할 절차를 사용하여 각 답안을 단계별로 분리합니다.

Appendix G. PRM AGGREGATION

단락 G-1

At test time, process-based verifiers can be used to score each individual step in a set of solutions sampled from the base model. In order to select the best-of-N answers with the PRM, we need a function that can aggregate across all the per-step scores for each answer to determine the best candidate for the correct answer. To do this, we first aggregate each individual answer’s per-step scores to obtain a final score for the full solution (step-wise aggregation). We then aggregate across answers to determine the best answer (inter-answer aggregation). Concretely, we handle step-wise and inter-answer aggregation as follows:

문장 G-1-1

At test time, process-based verifiers can be used to score each individual step in a set of solutions sampled from the base model.

단위 해석

테스트 시점에, 과정 기반 검증기(PRM)는 기본 모델에서 샘플링된 해답 세트 내의 각 개별 단계에 점수를 매기는 데 사용될 수 있습니다.

✔ 문장 G-1-2

In order to select the best-of-N answers with the PRM, we need a function that can aggregate across all the per-step scores for each answer to determine the best candidate for the correct answer.

🔍 단위 해석

PRM으로 Best-of-N 답안을 선택하기 위해서는, 정답이 될 최고의 후보를 결정하기 위해 각 답안의 모든 단계별 점수를 통합(aggregate)할 수 있는 함수가 필요합니다.

✔ 문장 G-1-3

To do this, we first aggregate each individual answer’s per-step scores to obtain a final score for the full solution (step-wise aggregation).

🔍 단위 해석

이를 위해, 우리는 먼저 각 개별 답안의 단계별 점수를 통합하여 전체 해답에 대한 최종 점수를 얻습니다 (단계별 통합).

✔ 문장 G-1-4

We then aggregate across answers to determine the best answer (inter-answer aggregation).

🔍 단위 해석

그런 다음 우리는 답안들 간에 통합을 수행하여 최고의 답안을 결정합니다 (답안 간 통합).

📖 단락 G-2

- **Step-wise aggregation.** Rather than aggregating the per-step scores by taking the product or minimum, we instead use the PRM’s prediction at the last step as the full-answer score. We found this to perform the best out of all aggregation methods we studied (see below).
- **Inter-answer aggregation.** We follow Li et al. (2023) and apply “best-of-N weighted” selection rather than standard best-of-N. Best-of-N weighted selection marginalizes the verifier’s correctness scores across all solutions with the same final answer, selecting final answer with the greatest total sum.

✔ 문장 G-2-1

- **Step-wise aggregation.** Rather than aggregating the per-step scores by taking the product or minimum, we instead use the PRM’s prediction at the last step as the full-answer score.

🔍 단위 해석

- 단계별 통합. 단계별 점수를 곱하거나 최솟값을 취하여 통합하는 대신, 우리는 PRM의 마지막 단계 예측값을 전체 답안 점수로 사용합니다.

✔ 문장 G-2-2

We found this to perform the best out of all aggregation methods we studied (see below).

🔍 단위 해석

우리는 이것이 우리가 연구한 모든 통합 방법 중 가장 성능이 좋다는 것을 발견했습니다(아래 참조).

✔ 문장 G-2-3

- **Inter-answer aggregation.** We follow Li et al. (2023) and apply “best-of-N weighted” selection rather than standard best-of-N.

🔍 단위 해석

- 답안 간 통합. 우리는 Li et al. (2023)을 따라 표준 Best-of-N 대신 “가중(weighted) Best-of-N” 선택을 적용합니다.

✔ 문장 G-2-4

Best-of-N weighted selection marginalizes the verifier’s correctness scores across all solutions with the same final answer, selecting final answer with the greatest total sum.

🔍 단위 해석

가중 Best-of-N 선택은 동일한 최종 정답을 가진 모든 해답들에 대해 검증기의 정답 점수를 합산(marginalizes)하여, 총합이 가장 큰 최종 정답을 선택하는 방식입니다.

G.1 COMPARING STEP-WISE AGGREGATION STRATEGIES

단락 G.1-1

We compare different methods of aggregating per-step PRM scores to produce a final score for the full solution. Specifically we compare: 1) taking the minimum score across all steps as done in Lightman et al. (2023) (e.g. “min”); 2) taking the product of all step correctness probabilities (e.g. “prod”); and 3) taking just the last step prediction (e.g. “last”). We see in Figure 14 that taking the last step outperforms the other two approaches. Prior works found min to be the best aggregator. We believe that the discrepancy is due to the fact that our verifier was trained with soft MC return labels, which surface very differently from binary correctness labels, and therefore other aggregation strategies may not have the same effect. Interestingly, when using the last step aggregation, we are effectively using the PRM like an ORM. However, we see that the PRM outperforms the ORM, suggesting that in our case the per-step PRM training may be largely useful as a form of representation learning, rather than purely as a tool at inference time. Future work should further explore this line of reasoning.

문장 G.1-1-1

We compare different methods of aggregating per-step PRM scores to produce a final score for the full solution.

단위 해석

우리는 전체 해답에 대한 최종 점수를 산출하기 위해 단계별 PRM 점수를 통합하는 다양한 방법들을 비교합니다.

문장 G.1-1-2

Specifically we compare: 1) taking the minimum score across all steps as done in Lightman et al. (2023) (e.g. “min”); 2) taking the product of all step correctness probabilities (e.g. “prod”); and 3) taking just the last step prediction (e.g. “last”).

단위 해석

구체적으로 다음을 비교합니다: 1) Lightman et al. (2023)에서와 같이 모든 단계 중 최솟값을 취하는 것(“min”); 2) 모든 단계의 정답 확률을 곱하는 것(“prod”); 3) 마지막 단계의 예측값만 취하는 것(“last”).

문장 G.1-1-3

We see in Figure 14 that taking the last step outperforms the other two approaches.

단위 해석

Figure 14에서 우리는 마지막 단계를 취하는 것이 다른 두 접근법보다 성능이 우수함을 볼 수 있습니다.

문장 G.1-1-4

Prior works found min to be the best aggregator.

단위 해석

선행 연구들은 'min'이 최고의 통합자임을 발견했었습니다.

문장 G.1-1-5

We believe that the discrepancy is due to the fact that our verifier was trained with soft MC return labels, which surface very differently from binary correctness labels, and therefore other aggregation strategies may not have the same effect.

단위 해석

우리는 이러한 불일치가 우리 검증기가 이진 정답 라벨과는 매우 다르게 나타나는 소프트 MC 반환 라벨(soft MC return labels)로 훈련되었기 때문이며, 따라서 다른 통합 전략들이 동일한 효과를 내지 못했을 것이라고 믿습니다.

문장 G.1-1-6

Interestingly, when using the last step aggregation, we are effectively using the PRM like an ORM.

단위 해석

흥미롭게도, 마지막 단계 통합을 사용할 때 우리는 사실상 PRM을 ORM처럼 사용하고 있는 셈입니다. (중간 과정 점수 무시하고 마지막만 보니까)

문장 G.1-1-7

However, we see that the PRM outperforms the ORM, suggesting that in our case the per-step PRM training may be largely useful as a form of representation learning, rather than purely as a tool at inference time.

단위 해석

하지만 우리는 PRM이 ORM보다 성능이 우수한 것을 볼 수 있는데, 이는 우리 경우에 단계별 PRM 훈련이 단순히 추론 도구로서라기보다는 표현 학습 (representation learning)의 한 형태로서 주로 유용했을 수 있음을 시사합니다.

시각 자료 [Figure 14]

(원문 캡션: **Figure 14:** We compare different methods of aggregating per-step PRM scores to produce a final score for the full solution: “min” refers to taking the minimum score accross all steps, “prod” takes the product of all step correctness probabilities, and “last” just uses the last step score. To account for the cost of querying the verifier relative to majority voting, we shift the curves which involve a verifier over by one point. We see that “PRM last” performs the best across all aggregation strategies.)

캡션 해석

전체 해답의 최종 점수를 산출하기 위한 단계별 PRM 점수 통합 방법 비교: "min"은 모든 단계 중 최솟값을 취하는 것, "prod"는 모든 단계의 정답 확률을 곱하는 것, "last"는 마지막 단계 점수만 사용하는 것을 의미합니다. 검증기 쿼리 비용을 고려해 그래프를 이동시켰습니다. "PRM last"가 모든 통합 전략 중 가장 좋은 성능을 보임을 알 수 있습니다.

PDF 시각 분석

PDF 14페이지의 **Figure 14** 분석:

- 그래프 해석:** 주황색 실선(PRM last)이 파란색 점선(PRM min)이나 초록색 점선(PRM prod)보다 지속적으로 상위에 위치합니다. 특히 샘플 수(2^7 이상)가 늘어날수록 격차가 유지되거나 벌어집니다. 이는 중간 과정에서 하나라도 실수했다고 판정(min)하거나 확률을 모두 곱해버리는(prod) 것보다, "그래서 결국 마지막에 정답에 얼마나 가까워졌나"를 보는(last) 방식이 소프트 라벨 훈련 환경에서는 더 강건함을 시사합니다.

G.2 COMPARING INTER-ANSWER AGGREGATION STRATEGIES

단락 G.2-1

In Figure 13 we compare best-of-N against best-of-N weighted for both our ORM and PRM verifiers. We find that while best-of-N weighted shows superior performance in both settings, the best-of-N performance with the PRM is still very competitive. On the other hand, in the ORM best-of-N setting, we observe Goodharting at higher budgets.

문장 G.2-1-1

In Figure 13 we compare best-of-N against best-of-N weighted for both our ORM and PRM verifiers.

단위 해석

Figure 13에서 우리는 ORM 및 PRM 검증기 모두에 대해 Best-of-N과 가중 Best-of-N을 비교합니다.

문장 G.2-1-2

We find that while best-of-N weighted shows superior performance in both settings, the best-of-N performance with the PRM is still very competitive.

단위 해석

우리는 가중 Best-of-N이 두 설정 모두에서 더 우수한 성능을 보이지만, PRM을 사용한 일반 Best-of-N 성능도 여전히 매우 경쟁력이 있음을 발견했습니다.

문장 G.2-1-3

On the other hand, in the ORM best-of-N setting, we observe Goodharting at higher budgets.

단위 해석

반면, ORM Best-of-N 설정에서는 높은 예산에서 굿하트의 법칙(Goodharting, 지표가 목표가 되면 더 이상 좋은 지표가 아닌 현상, 여기서는 과적합으로 인한 성능 하락) 현상이 관찰되었습니다.

시각 자료 [Figure 13]

(원문 캡션: **Figure 13:** We compare best-of-N and best-of-N weighted for our ORM and PRM verifiers finetuned from PaLM 2 — S^* . We use the PaLM 2 — S^* base LM to sample outputs, using a few-shot prompt. To account for the cost of querying the verifier relative to majority voting, we shift the curves which involve a verifier over by one point. We see that while best-of-N weighted shows superior performance in both settings, the best-of-N performance with the PRM is still very competitive. On the other hand, in the ORM best-of-N setting, we observe Goodharting at higher budgets.)

캡션 해석

PaLM 2 — S^* 에서 미세 조정된 ORM 및 PRM 검증기에 대해 Best-of-N과 가중 Best-of-N을 비교합니다... 가중 Best-of-N이 두 설정 모두에서 우수하지만, PRM의 Best-of-N 성능도 경쟁력이 있습니다. 반면 ORM Best-of-N 설정에서는 높은 예산에서 굿하트 현상(Goodharting)이 관찰됩니다.

PDF 시각 분석

PDF 14페이지의 **Figure 13** 분석:

- PRM (주황/빨강 계열):** 실선(PRM weighted)과 점선(PRM) 모두 우상향하며 높은 정확도에 도달합니다. 둘 사이의 차이가 크지 않습니다.

- **ORM (파랑/초록 계열):** 실선(ORM weighted)은 잘 올라가지만, 점선(ORM, 일반 Best-of-N)은 2^6 을 넘어가면서 급격히 꺾여 내려갑니다(Goodharting). 이는 단순 ORM 점수만 믿고 뽑으면, 모델이 점수만 높고 틀린 답을 내는 경향이 심해짐을 보여줍니다. 가중치(weighted) 방식이 이를 보정해 줍니다.

Appendix H. COMPARING PRM AND ORM

단락 H-1

We trained a PRM and ORM model using the PaLM 2 — S^* base LM. We see in Figure 15, that the PRM outperforms the ORM, and the gap between the PRM and ORM grows with the number of samples used. We use the last step prediction from the PRM to score the answers as described in Appendix G.

문장 H-1-1

We trained a PRM and ORM model using the PaLM 2 — S^* base LM.

단위 해석

우리는 PaLM 2 — S^* 기본 언어 모델을 사용하여 PRM(과정 보상 모델)과 ORM(결과 보상 모델)을 훈련시켰습니다.

문장 H-1-2

We see in Figure 15, that the PRM outperforms the ORM, and the gap between the PRM and ORM grows with the number of samples used.

단위 해석

Figure 15에서 우리는 PRM이 ORM보다 성능이 우수하며, 사용된 샘플 수가 증가할수록 PRM과 ORM 간의 격차가 벌어지는 것을 볼 수 있습니다.

문장 H-1-3

We use the last step prediction from the PRM to score the answers as described in Appendix G.

단위 해석

우리는 부록 G에서 설명한 대로 PRM의 마지막 단계 예측값을 사용하여 답안의 점수를 매깁니다.

시각 자료 [Figure 15]

(원문 캡션: **Figure 15:** We compare PRM and ORM models finetuned from PaLM 2 — S^* in a best-of-N evaluation. We use the PaLM 2 — S^* base LM to sample outputs, using a few-shot prompt. To account for the cost of querying the verifier relative to majority voting, we shift the curves which involve a verifier over by one point. We see that the PRM greatly outperforms the ORM at a large number of samples.)

캡션 해석

Best-of-N 평가에서 PaLM 2 — S^* 로부터 미세 조정된 PRM과 ORM 모델을 비교합니다. 퓨샷 프롬프트를 사용하여 PaLM 2 — S^* 기본 모델로 출력을 샘플링 합니다. 다수결 투표 대비 검증기 호출 비용을 고려하여 검증기 곡선을 한 포인트 이동시켰습니다. 많은 샘플 수에서 PRM이 ORM을 크게 능가함을 알 수 있습니다.

PDF 시각 분석

PDF 15페이지의 **Figure 15** 분석:

- **그래프 해석:** 파란색 실선(PRM)이 주황색 실선(ORM)보다 항상 위에 있습니다. 특히 X 축(샘플 수)이 오른쪽으로 갈수록 파란색 선은 계속 올라가지만 주황색 선은 기울기가 완만해집니다. 이는 과정(Process)을 검증하는 것이 결과(Outcome)만 보는 것보다 훨씬 더 강력한 신호(Signal)를 제공함을 의미합니다.

Appendix I. SEARCH USING PALM 2-S AND M

단락 I-1

In Figure 16, we plot the performance of beam-search and best-of-N binned by difficulty levels using PaLM 2-S and PaLM 2-M as the base models. We observe broadly similar trends to those in Figure 3 on PaLM 2- S^* , demonstrating that our findings likely transfer to other base LLMs.

문장 I-1-1

In Figure 16, we plot the performance of beam-search and best-of-N binned by difficulty levels using PaLM 2-S and PaLM 2-M as the base models.

단위 해석

Figure 16에서 우리는 PaLM 2-S와 PaLM 2-M을 기본 모델로 사용하여, 난이도 구간별로 빔 서치와 Best-of-N의 성능을 도표로 나타냅니다.

✔ 문장 I-1-2

We observe broadly similar trends to those in Figure 3 on PaLM 2-S*, demonstrating that our findings likely transfer to other base LLMs.

🔍 단위 해석

우리는 PaLM 2-S*에 대한 Figure 3의 결과와 대체로 유사한 추세를 관찰했으며, 이는 우리의 발견이 다른 기본 LLM들에도 전이(적용)될 가능성이 높음을 입증합니다.

🖼️ 시각 자료 [Figure 16]

(원문 캡션: **Figure 16:** Comparing beam search and best-of-N binned by difficulty level with PaLM 2-S (left) and PaLM 2-M (right). The four bars in each difficulty bin correspond to increasing test-time compute budgets (4, 16, 64, and 256 generations). We observe broadly similar trends to those in Figure 3 on PaLM 2-S*, demonstrating that our findings likely transfer to other base LLMs.)

🔍 캡션 해석

PaLM 2-S(왼쪽)와 PaLM 2-M(오른쪽)에 대해 난이도 구간별로 빔 서치와 Best-of-N 비교. 각 난이도 구간의 4개 막대는 증가하는 테스트 시점 연산 예산(4, 16, 64, 256회 생성)에 해당합니다. PaLM 2-S*에 대한 Figure 3과 대체로 유사한 추세를 관찰할 수 있습니다.

📄 PDF 시각 분석

PDF 15페이지의 **Figure 16** 분석:

- 트렌드 재확인: 왼쪽(PaLM 2-S)과 오른쪽(PaLM 2-M) 그래프 모두에서, 쉬운 문제(Bin 1~2)는 빔 서치(파란 막대)가 예산을 늘려도 성능이 잘 안 오르거나 떨어지는 반면, Best-of-N(주황 막대)은 꾸준합니다. 반대로 어려운 문제(Bin 3~4)에서는 빔 서치가 Best-of-N을 앞서는 구간이 명확히 보입니다. 모델 크기가 달라도 "어려운 문제엔 탐색(Search)이 유리하다"는 법칙이 유지됩니다.

Appendix J. PROMPTING DETAILS

📖 단락 J-1

In order to enable the base model to output answers in a step-by-step format to which a PRM can be applied, we use a 4-shot prompt consisting of randomly selected correct answer examples from the PRM800k data released by Lightman et al. (2023). Specifically we use answers from the phase 1 training split. These answers correspond to GPT-4 generated correct answer examples, which include the correct step-by-step format. In initial experiments, we found that this prompting procedure produces similar results to the prompt used in Lewkowycz et al. (2022). We use this prompt for generating training data for the PRM and the revision model. We also use this prompt when conducting search against the PRM on the test-set. To grade the final answer predicted by this prompt, we use the grading function released by Lightman et al. (2023).

✔ 문장 J-1-1

In order to enable the base model to output answers in a step-by-step format to which a PRM can be applied, we use a 4-shot prompt consisting of randomly selected correct answer examples from the PRM800k data released by Lightman et al. (2023).

🔍 단위 해석

기본 모델이 PRM을 적용할 수 있는 단계별(step-by-step) 형식으로 답안을 출력하게 하기 위해, 우리는 Lightman et al. (2023)이 공개한 PRM800k 데이터에서 무작위로 선택된 4개의 정답 예시로 구성된 4-shot 프롬프트를 사용합니다.

✔ 문장 J-1-2

Specifically we use answers from the phase 1 training split.

🔍 단위 해석

구체적으로 우리는 1단계(phase 1) 훈련 분할의 답안들을 사용합니다.

✔ 문장 J-1-3

These answers correspond to GPT-4 generated correct answer examples, which include the correct step-by-step format.

🔍 단위 해석

이 답안들은 GPT-4가 생성한 정답 예시들에 해당하며, 올바른 단계별 형식을 포함하고 있습니다.

✔ 문장 J-1-4

In initial experiments, we found that this prompting procedure produces similar results to the prompt used in Lewkowycz et al. (2022).

🔍 단위 해석

초기 실험에서 우리는 이 프롬프팅 절차가 Lewkowycz et al. (2022)에서 사용된 프롬프트와 유사한 결과를 산출함을 확인했습니다.

✔ 문장 J-1-5

We use this prompt for generating training data for the PRM and the revision model.

🔍 단위 해석

우리는 이 프롬프트를 PRM과 수정 모델을 위한 훈련 데이터를 생성하는 데 사용합니다.

✔ 문장 J-1-6

We also use this prompt when conducting search against the PRM on the test-set.

🔍 단위 해석

우리는 또한 테스트 세트에서 PRM을 상대로 탐색을 수행할 때 이 프롬프트를 사용합니다.

Appendix K. THE EFFECT OF VERIFIER QUALITY ON OUR CONCLUSIONS

📖 단락 K-1

We would like to understand whether our conclusions in Section 7, regarding whether scaling test-time compute is favorable to scaling model parameters, are robust to the quality of the specific finetuned PRM verifier that we used in our experiments. In particular, we want to determine whether we would observe similar trends using less capable verifiers. To do this, we conduct a similar analysis to that in Section 7, using four different settings that are specifically designed to emulate the effect of having a lower quality verifier:

A) A “uniform random” verifier. Since weighted BoN with a random verifier converges to majority voting, we simulate a random verifier by using majority voting.

B) Our PRM verifier but with 20% i.i.d. label flip noise. In particular, if the verifier predicts a correctness probability of p , we flip its prediction to $(1-p)$ 20% of the time.

C) Our standard PRM verifier. This is identical to the standard PRM used in Section 7.

D) Majority voting using parallel samples from the revisions model with no PRM. This setting is equivalent to using a “uniform random” verifier on top of of our finetuned revisions model.

✔ 문장 K-1-1

We would like to understand whether our conclusions in Section 7, regarding whether scaling test-time compute is favorable to scaling model parameters, are robust to the quality of the specific finetuned PRM verifier that we used in our experiments.

🔍 단위 해석

우리는 섹션 7의 결론(테스트 시점 연산 확장이 모델 파라미터 확장보다 유리한지 여부)이 우리가 실험에 사용한 특정 미세 조정 PRM 검증기의 품질에 대해 강건(robust)한지 이해하고자 합니다. (즉, 검증기가 좀 멍청해도 결과가 같을까?)

✔ 문장 K-1-2

In particular, we want to determine whether we would observe similar trends using less capable verifiers.

🔍 단위 해석

특히, 성능이 떨어지는 검증기를 사용해도 유사한 추세를 관찰할 수 있는지 확인하고 싶습니다.

✔ 문장 K-1-3

To do this, we conduct a similar analysis to that in Section 7, using four different settings that are specifically designed to emulate the effect of having a lower quality verifier:

🔍 단위 해석

이를 위해 우리는 낮은 품질의 검증기 효과를 모방하도록 특별히 설계된 네 가지 다른 설정을 사용하여 섹션 7과 유사한 분석을 수행합니다:

✔ 문장 K-1-4

A) A “uniform random” verifier. Since weighted BoN with a random verifier converges to majority voting, we simulate a random verifier by using majority voting.

🔍 단위 해석

A) “균등 무작위(uniform random)” 검증기. 무작위 검증기를 사용한 가중 Best-of-N은 다수결 투표로 수렴하기 때문에, 우리는 다수결 투표를 사용하여 무작위 검증기를 시뮬레이션합니다.

✔ 문장 K-1-5

B) Our PRM verifier but with 20% i.i.d. label flip noise. In particular, if the verifier predicts a correctness probability of p , we flip its prediction to $(1-p)$ 20% of the time.

🔍 단위 해석

B) 20%의 i.i.d. 라벨 반전 노이즈(label flip noise)가 섞인 우리 PRM 검증기. 구체적으로, 검증기가 정답 확률을 p 로 예측하면 20%의 확률로 그 예측을 $(1-p)$ 로 뒤집습니다.

✔ 문장 K-1-6

C) Our standard PRM verifier. This is identical to the standard PRM used in Section 7.

🔍 단위 해석

C) 우리의 표준 PRM 검증기. 이는 섹션 7에서 사용된 표준 PRM과 동일합니다.

✔ 문장 K-1-7

D) Majority voting using parallel samples from the revisions model with no PRM. This setting is equivalent to using a “uniform random” verifier on top of of our finetuned revisions model.

🔍 단위 해석

D) PRM 없이 수정 모델의 병렬 샘플을 사용한 다수결 투표. 이 설정은 우리의 미세 조정된 수정 모델 위에 “균등 무작위” 검증기를 사용하는 것과 동등합니다.

📖 단락 K-2

Observe in Figure 17 that while majority voting (A) generally under-performs the larger pretrained model, the noised verifier (B) can outperform pretraining on easy/medium questions in settings with low inference requirements. We also note that majority voting with a revisions model (D) can outperform scaling model parameters on easy and medium questions, without using a verifier suggesting that it is still possible to get decent test-time scaling that outperforms pretraining with a less capable verifier (or no verifier, if the base model is finetuned in a certain way). These findings demonstrate that even with a somewhat weaker verifier model, the main conclusions from our FLOPs analysis in Section 7 can still hold. Moreover, we see that higher quality verifiers tend to show better test-time scaling, suggesting that it may be possible for us to substantially improve the test-time scaling of verifiers by finetuning the verifier on a much larger dataset of mathematical question answers pairs than the 12k that we used in this work. We believe this is an exciting direction for future work to explore.

✔ 문장 K-2-1

Observe in Figure 17 that while majority voting (A) generally under-performs the larger pretrained model, the noised verifier (B) can outperform pretraining on easy/medium questions in settings with low inference requirements.

🔍 단위 해석

Figure 17에서 다수결 투표(A)는 일반적으로 더 큰 사전 학습 모델보다 성능이 떨어지지만, 노이즈가 섞인 검증기(B)는 추론 요구량이 낮은 환경의 쉬운/중간 난이도 문제에서 사전 학습을 능가할 수 있음을 관찰하세요.

✔ 문장 K-2-2

We also note that majority voting with a revisions model (D) can outperform scaling model parameters on easy and medium questions, without using a verifier suggesting that it is still possible to get decent test-time scaling that outperforms pretraining with a less capable verifier (or no verifier, if the base model is finetuned in a certain way).

🔍 단위 해석

또한 수정 모델을 사용한 다수결 투표(D)가 검증기 없이도 쉬운/중간 난이도 문제에서 모델 파라미터 확장을 능가할 수 있다는 점에 주목합니다. 이는 성능이 낮은 검증기(혹은 기본 모델이 특정 방식으로 미세 조정된 경우 검증기가 없더라도)를 가지고도 사전 학습을 능가하는 괄찬은 테스트 시점 스케일링을 얻는 것이 여전히 가능함을 시사합니다.

✔ 문장 K-2-3

These findings demonstrate that even with a somewhat weaker verifier model, the main conclusions from our FLOPs analysis in Section 7 can still hold.

🔍 단위 해석

이러한 발견들은 다소 약한 검증기 모델을 사용하더라도 섹션 7의 FLOPs 분석에서 도출된 주요 결론이 여전히 유효할 수 있음을 입증합니다.

✔ 문장 K-2-4

Moreover, we see that higher quality verifiers tend to show better test-time scaling, suggesting that it may be possible for us to substantially improve the test-time scaling of verifiers by finetuning the verifier on a much larger dataset of mathematical question answers pairs than the 12k that we used in this work.

단위 해석
더 나아가, 우리는 더 높은 품질의 검증기가 더 나은 테스트 시점 스케일링을 보여주는 경향이 있음을 확인했습니다. 이는 본 연구에서 사용한 12,000개보다 훨씬 더 큰 수학 질의응답 쌍 데이터셋으로 검증기를 미세 조정함으로써 검증기의 테스트 시점 스케일링을 상당히 개선할 수 있는 가능성을 시사합니다.

시각 자료 [Figure 17]

(원문 캡션: **Figure 17:** We extend the analysis in Section 7 by comparing the test-time scaling of weaker verifiers against the performance of a 14x larger pretrained model. We see that while majority voting (A) generally under-performs the larger pretrained model, the noised verifier (B) can outperform pertaining on easy/medium questions in settings with low inference requirements. Additionally, majority voting with a revisions model (D) can outperform scaling model parameters on easy and medium questions, without using a verifier.)

캡션 해석
약한 검증기들의 테스트 시점 스케일링과 14배 더 큰 사전 학습 모델의 성능을 비교하여 섹션 7의 분석을 확장합니다. 다수결 투표(A)는 일반적으로 큰 모델보다 성능이 떨어지지만, 노이즈가 섞인 검증기(B)는 낮은 추론 요구량 설정에서 쉬운/중간 문제에 대해 사전 학습 모델을 능가할 수 있습니다. 또한, 수정 모델을 이용한 다수결 투표(D)는 검증기 없이도 쉬운/중간 문제에서 모델 파라미터 확장을 능가할 수 있습니다.

PDF 시각 분석
PDF 16페이지의 **Figure 17** 분석:

- 그래프 해석:
 - (A) Majority Voting:** 별표(큰 모델 성능)보다 대부분 아래에 있습니다. 즉, 단순 다수결만으로는 큰 모델을 이기기 어렵습니다.
 - (B) Noised Verifier:** 검증기를 일부러 망가뜨려도(노이즈 추가), 쉬운 문제(왼쪽 그래프)에서는 별표보다 선이 위에 있는 구간이 존재합니다. 즉, 완벽하지 않은 검증기도 쓸모가 있습니다.
 - (D) Revision Majority:** 검증기 없이 모델이 스스로 수정(Revision)한 뒤 다수결을 하면, 별표를 능가하는 경우가 꽤 많이 보입니다. 이는 ‘수정 능력’ 자체가 강력한 스케일링 도구임을 증명합니다.

Appendix L. REVISION MODEL FINETUNING DETAILS

단락 L-1

Our procedure for finetuning revision models is similar to (Qu et al., 2024b), though we introduce some crucial differences. For finetuning, we need trajectories consisting of a sequence of incorrect answers followed by a correct answer, that we can then run SFT on. Ideally, we want the correct answer to be correlated with the incorrect answers provided in context, so as to effectively teach the model to implicitly identify mistakes in examples provided in-context, followed by correcting those mistakes by making edits as opposed to ignoring the in-context examples altogether, and trying again from scratch.

문장 L-1-1

Our procedure for finetuning revision models is similar to (Qu et al., 2024b), though we introduce some crucial differences.

단위 해석
우리의 수정 모델 미세 조정 절차는 (Qu et al., 2024b)와 유사하지만, 몇 가지 중요한 차이점을 도입합니다.

문장 L-1-2

For finetuning, we need trajectories consisting of a sequence of incorrect answers followed by a correct answer, that we can then run SFT on.

단위 해석
미세 조정을 위해서는 ‘일련의 오답들’ 뒤에 ‘정답’이 오는 궤적(trajec-tory)이 필요하며, 우리는 이에 대해 SFT(지도 미세 조정)를 수행할 수 있습니다.

문장 L-1-3

Ideally, we want the correct answer to be correlated with the incorrect answers provided in context, so as to effectively teach the model to implicitly identify mistakes in examples provided in-context, followed by correcting those mistakes by making edits as opposed to ignoring the in-context examples altogether, and trying again from scratch.

단위 해석
이상적으로는 정답이 문맥에 제공된 오답들과 상관관계가 있기를 원합니다. 이는 모델이 문맥 속 예시들의 실수를 암묵적으로 식별한 뒤 이를 수정(edits)하여 고치도록 효과적으로 가르치기 위함이며, 문맥 속 예시를 완전히 무시하고 처음부터 다시 시도하는 것을 방지하기 위함입니다.

단락 L-2

Generating revision data. The on-policy approach of Qu et al. (2024b) for obtaining several multi-turn rollouts was shown to be effective, but it was not entirely feasible in our infrastructure due to compute costs associated with running multi-turn rollouts. Therefore, we sampled

64 responses in parallel at a higher temperature and post-hoc constructed multi-turn rollouts from these independent samples. Specifically, following the recipe of (Kumar et al., 2024), we pair up each correct answer with a sequence of incorrect answers from this set as context to construct multi-turn finetuning data. We include up to four incorrect answers in context, where the specific number of solutions in context is sampled randomly from a uniform distribution over categories 0 to 4. The correct answer is used as the last answer in the trajectory (which we train the model to produce) and the incorrect answers are included in context. If the sampled number is greater than 0, we then find the closest incorrect answer according to a character-level edit distance metric to include as the last incorrect answer in the trajectory. The goal here is to select an incorrect answer which is somewhat correlated with the correct answer, to improve learning. Note that token edit distance is not a perfect measure of correlation, but we found this heuristic to be sufficient to correlate incorrect in-context answers with correct target answers to facilitate training a meaningful revision model, as opposed to randomly pairing incorrect and correct responses with uncorrelated responses. Finally, in the case where there are fewer than 4 incorrect answers sampled, we truncate the uniform distribution’s max to match the number of incorrect samples. We use this procedure to generate trajectories for all questions in the training data.

✔

문장 L-2-1

Generating revision data. The on-policy approach of Qu et al. (2024b) for obtaining several multi-turn rollouts was shown to be effective, but it was not entirely feasible in our infrastructure due to compute costs associated with running multi-turn rollouts.

🔍 단위 해석

수정 데이터 생성. 여러 멀티턴 롤아웃을 얻기 위한 Qu et al. (2024b)의 온폴리시(on-policy) 접근법은 효과적인 것으로 나타났으나, 멀티턴 롤아웃 실행과 관련된 연산 비용 문제로 인해 우리 인프라에서는 완전히 실행 가능하지 않았습니다.

✔

문장 L-2-2

Therefore, we sampled 64 responses in parallel at a higher temperature and post-hoc constructed multi-turn rollouts from these independent samples.

🔍 단위 해석

따라서 우리는 더 높은 온도(temperature)에서 64개의 응답을 병렬로 샘플링한 뒤, 이 독립적인 샘플들로부터 사후적으로(post-hoc) 멀티턴 롤아웃을 구성했습니다.

✔

문장 L-2-3

Specifically, following the recipe of (Kumar et al., 2024), we pair up each correct answer with a sequence of incorrect answers from this set as context to construct multi-turn finetuning data.

🔍 단위 해석

구체적으로 (Kumar et al., 2024)의 레시피를 따라, 우리는 각 정답을 이 세트에서 추출한 일련의 오답들과 짝지어 문맥으로 삼음으로써 멀티턴 미세 조정 데이터를 구성했습니다.

✔

문장 L-2-4

We include up to four incorrect answers in context, where the specific number of solutions in context is sampled randomly from a uniform distribution over categories 0 to 4.

🔍 단위 해석

우리는 문맥에 최대 4개의 오답을 포함시키며, 이때 문맥에 들어갈 구체적인 해답 수는 0에서 4까지의 범주에 대한 균등 분포에서 무작위로 샘플링됩니다.

✔

문장 L-2-5

The correct answer is used as the last answer in the trajectory (which we train the model to produce) and the incorrect answers are included in context.

🔍 단위 해석

정답은 궤적의 마지막 답안(모델이 생성하도록 훈련시키는 대상)으로 사용되며, 오답들은 문맥에 포함됩니다.

✔

문장 L-2-6

If the sampled number is greater than 0, we then find the closest incorrect answer according to a character-level edit distance metric to include as the last incorrect answer in the trajectory.

🔍 단위 해석

만약 샘플링된 수가 0보다 크다면, 우리는 문자 수준 편집 거리(character-level edit distance) 지표에 따라 가장 가까운 오답을 찾아 궤적의 마지막 오답으로 포함시킵니다.

✔

문장 L-2-7

The goal here is to select an incorrect answer which is somewhat correlated with the correct answer, to improve learning.

🔍 단위 해석

여기서의 목표는 학습을 개선하기 위해 정답과 어느 정도 상관관계가 있는 오답을 선택하는 것입니다.

✔ 문장 L-2-8

Note that token edit distance is not a perfect measure of correlation, but we found this heuristic to be sufficient to correlate incorrect in-context answers with correct target answers to facilitate training a meaningful revision model, as opposed to randomly pairing incorrect and correct responses with uncorrelated responses.

🔍 단위 해석

토큰 편집 거리가 상관관계의 완벽한 척도는 아니지만, 우리는 이 휴리스틱이 오답과 정답을 무작위로 짝짓는 것과 달리, 문맥 속 오답과 타겟 정답을 연관시켜 의미 있는 수정 모델 훈련을 촉진하기에 충분하다는 것을 발견했습니다.

✔ 문장 L-2-9

Finally, in the case where there are fewer than 4 incorrect answers sampled, we truncate the uniform distribution’s max to match the number of incorrect samples.

🔍 단위 해석

마지막으로, 샘플링된 오답이 4개 미만인 경우에는 균등 분포의 최댓값을 오답 샘플 수에 맞춰 잘라냅니다(truncate).

📖 단락 L-3

We then finetune the base language model on the correct answer solutions in these generated trajectories. We use the AdamW optimizer with lr 1e-5, batch size 128, dropout 0.0, and Adam betas (0.9, 0.95). We find that generally evaluating loss on an evaluation set consisting of trajectories generated as described above, does not provide a good signal for early stopping. Rather, we find that checkpoints much after the evaluation loss begins increasing are much more capable of revisions. This is likely because after finetuning the revision model, the evaluation set represents off-policy data, which will naturally be out-of-distribution compared to the trajectories that the model itself would generate on-policy. We therefore select our revision model checkpoint slightly after the point where we observe overfitting on the validation set.

✔ 문장 L-3-1

We then finetune the base language model on the correct answer solutions in these generated trajectories.

🔍 단위 해석

그런 다음 우리는 생성된 궤적들의 정답 해법에 대해 기본 언어 모델을 미세 조정합니다.

✔ 문장 L-3-2

We use the AdamW optimizer with lr 1e-5, batch size 128, dropout 0.0, and Adam betas (0.9, 0.95).

🔍 단위 해석

우리는 학습률 1e-5, 배치 크기 128, 드롭아웃 0.0, Adam 베타 (0.9, 0.95)로 설정된 AdamW 옵티마이저를 사용합니다.

✔ 문장 L-3-3

We find that generally evaluating loss on an evaluation set consisting of trajectories generated as described above, does not provide a good signal for early stopping.

🔍 단위 해석

우리는 위와 같이 생성된 궤적들로 구성된 평가 세트에서 손실(loss)을 평가하는 것이 일반적으로 조기 종료(early stopping)를 위한 좋은 신호를 제공하지 못한다는 것을 발견했습니다.

✔ 문장 L-3-4

Rather, we find that checkpoints much after the evaluation loss begins increasing are much more capable of revisions.

🔍 단위 해석

오히려, 평가 손실이 증가하기 시작한 시점보다 훨씬 뒤의 체크포인트들이 수정 능력 면에서 훨씬 더 뛰어나다는 것을 발견했습니다.

✔ 문장 L-3-5

This is likely because after finetuning the revision model, the evaluation set represents off-policy data, which will naturally be out-of-distribution compared to the trajectories that the model itself would generate on-policy.

🔍 단위 해석

이는 수정 모델을 미세 조정한 후에는 평가 세트가 오프폴리시(off-policy) 데이터를 나타내게 되며, 이는 모델이 온폴리시(on-policy)로 직접 생성할 궤적들과 비교했을 때 자연스럽게 분포를 벗어난(out-of-distribution) 데이터가 되기 때문일 것입니다.

✔ 문장 L-3-6

We therefore select our revision model checkpoint slightly after the point where we observe overfitting on the validation set.

단위 해석

따라서 우리는 검증 세트에서 과적합(overfitting)이 관찰되는 시점보다 약간 뒤의 수정 모델 체크포인트를 선택합니다.

Appendix M. REVISION MODEL SELECTION CRITERIA

단락 M-1

As described in Section 6.1, in order to effectively use our revision model we need to deploy a criteria for selecting the best answer both within a revision trajectory and between multiple parallel trajectories. We use two approaches: 1) ORM verifier; and 2) majority voting.

문장 M-1-1

As described in Section 6.1, in order to effectively use our revision model we need to deploy a criteria for selecting the best answer both within a revision trajectory and between multiple parallel trajectories.

단위 해석

섹션 6.1에서 설명한 바와 같이, 수정 모델을 효과적으로 사용하기 위해서는 하나의 수정 궤적 내에서뿐만 아니라 여러 병렬 궤적들 사이에서도 최고의 답을 선택하기 위한 기준을 배포해야 합니다.

문장 M-1-2

We use two approaches: 1) ORM verifier; and 2) majority voting.

단위 해석

우리는 두 가지 접근법을 사용합니다: 1) ORM 검증기; 2) 다수결 투표.

단락 M-2

For the ORM verifier, we train an ORM on the revision model’s outputs according to the procedure in Appendix N. At inference time we then use this verifier to select the best answer. Since we have two axes across which to aggregate (within each revision trajectories and between multiple trajectories), we deploy a hierarchical strategy, first selecting the best answer within each revision trajectory and then aggregating these selected answers across trajectories. To select the best answer within each trajectory, we perform best-of-N weighted aggregation and then choose the highest scoring solution with the maximum best-of-N weighted answer. Then, to select the final answer across all revision chains, we perform another round of best-of-N weighted selection using the best answer from each revision chain. The answer after this second round of best-of-N weighted represents our final answer prediction.

문장 M-2-1

For the ORM verifier, we train an ORM on the revision model’s outputs according to the procedure in Appendix N.

단위 해석

ORM 검증기의 경우, 우리는 부록 N의 절차에 따라 수정 모델의 출력에 대해 ORM을 훈련시킵니다.

문장 M-2-2

At inference time we then use this verifier to select the best answer.

단위 해석

추론 시점에 우리는 이 검증기를 사용하여 최고의 답을 선택합니다.

문장 M-2-3

Since we have two axes across which to aggregate (within each revision trajectories and between multiple trajectories), we deploy a hierarchical strategy, first selecting the best answer within each revision trajectory and then aggregating these selected answers across trajectories.

단위 해석

우리는 통합해야 할 두 개의 축(각 수정 궤적 내부, 그리고 여러 궤적들 사이)을 가지고 있으므로, 먼저 각 수정 궤적 내에서 최고의 답을 선택한 다음 이 선택된 답들을 궤적 간에 통합하는 계층적 전략(hierarchical strategy)을 사용합니다.

문장 M-2-4

To select the best answer within each trajectory, we perform best-of-N weighted aggregation and then choose the highest scoring solution with the maximum best-of-N weighted answer.

단위 해석

각 궤적 내에서 최고의 답을 선택하기 위해, 우리는 가중 Best-of-N 통합을 수행한 뒤 최대 가중 Best-of-N 점수를 가진 가장 높은 점수의 해법을 선택합니다.

문장 M-2-5

Then, to select the final answer across all revision chains, we perform another round of best-of-N weighted selection using the best answer from each revision chain.

단위 해석

그런 다음 모든 수정 체인(chain)에 걸쳐 최종 답을 선택하기 위해, 각 수정 체인에서 뽑힌 최고의 답들을 사용하여 또 한 번의 가중 Best-of-N 선택을 수행합니다.

문장 M-2-6

The answer after this second round of best-of-N weighted represents our final answer prediction.

단위 해석

이 두 번째 가중 Best-of-N 라운드 후의 답이 우리의 최종 정답 예측이 됩니다.

단락 M-3

For majority voting we found hierarchical aggregation to create problems when the length of the trajectory or the number of trajectories was too small. The problem being that without enough samples, majority voting is unable to effectively select the best option. Therefore, for majority voting, we simply take all answers, across all trajectories, at once and take their majority as the final-answer. We found this to produce much smoother scaling behavior than the hierarchical approach.

문장 M-3-1

For majority voting we found hierarchical aggregation to create problems when the length of the trajectory or the number of trajectories was too small.

단위 해석

다수결 투표의 경우, 궤적의 길이나 궤적의 수가 너무 적을 때 계층적 통합이 문제를 일으킨다는 것을 발견했습니다.

문장 M-3-2

The problem being that without enough samples, majority voting is unable to effectively select the best option.

단위 해석

문제는 샘플이 충분하지 않으면 다수결 투표가 최선의 옵션을 효과적으로 선택할 수 없다는 것입니다.

문장 M-3-3

Therefore, for majority voting, we simply take all answers, across all trajectories, at once and take their majority as the final-answer.

단위 해석

따라서 다수결 투표의 경우, 우리는 단순히 모든 궤적에 걸친 모든 답안을 한꺼번에 가져와서 그들의 다수결(majority)을 최종 정답으로 삼습니다.

문장 M-3-4

We found this to produce much smoother scaling behavior than the hierarchical approach.

단위 해석

우리는 이것이 계층적 접근 방식보다 훨씬 더 부드러운 스케일링 거동을 생성한다는 것을 발견했습니다.

Appendix N. REVISION MODEL VERIFIER TRAINING

단락 N-1

We found that the PRM we finetuned on the PaLM 2 — S^* base model outputs was not as effective when applied to the PaLM 2 — S^* revision model’s outputs (see Figure 18(a)), likely due to distribution shift with the revision model. We therefore, trained a separate ORM verifier to use with our PaLM 2 — S^* revision model. We could have trained a PRM as well, but opted for an ORM due to the high cost of generating per-step PRM labels.

✔ 문장 N-1-1

We found that the PRM we finetuned on the PaLM 2 — S^* base model outputs was not as effective when applied to the PaLM 2 — S^* revision model's outputs (see Figure 18(a)), likely due to distribution shift with the revision model.

🔍 단위 해석

우리는 PaLM 2 — S^* 기본 모델 출력에 대해 미세 조정한 PRM이 PaLM 2 — S^* 수정 모델의 출력에 적용되었을 때 그리 효과적이지 않다는 것을 발견했으며 (Figure 18(a) 참조), 이는 아마도 수정 모델과의 분포 변화 때문일 것입니다.

✔ 문장 N-1-2

We therefore, trained a separate ORM verifier to use with our PaLM 2 — S^* revision model.

🔍 단위 해석

따라서 우리는 PaLM 2 — S^* 수정 모델과 함께 사용할 별도의 ORM 검증기를 훈련시켰습니다.

✔ 문장 N-1-3

We could have trained a PRM as well, but opted for an ORM due to the high cost of generating per-step PRM labels.

🔍 단위 해석

PRM을 훈련시킬 수도 있었지만, 단계별 PRM 라벨을 생성하는 높은 비용 때문에 ORM을 선택했습니다.

📖 단락 N-2

We modified the standard ORM slightly for the revision setting, by finetuning the ORM with previous revision in context, such that the verifier has access to the same context as the revision model, allowing the verifier see the revision model's previous answer attempts when scoring the current answer. All other experiment details are identical to those used for training the PRM. Empirically, we find that including the revision history in context improves performance slightly (see Figure 18(b)). Additionally, even without the revisions in context, we see that sequential revisions still slightly outperforms parallel, demonstrating improvements from sequential sampling are not just due to the verifier's context.

✔ 문장 N-2-1

We modified the standard ORM slightly for the revision setting, by finetuning the ORM with previous revision in context, such that the verifier has access to the same context as the revision model, allowing the verifier see the revision model's previous answer attempts when scoring the current answer.

🔍 단위 해석

우리는 수정 설정을 위해 표준 ORM을 약간 수정했는데, 검증기가 수정 모델과 동일한 문맥에 접근할 수 있도록 이전 수정 내용을 문맥에 포함하여 ORM을 미세 조정했습니다. 이를 통해 검증기는 현재 답안의 점수를 매길 때 수정 모델의 이전 시도들을 볼 수 있게 됩니다.

✔ 문장 N-2-2

All other experiment details are identical to those used for training the PRM.

🔍 단위 해석

다른 모든 실험 세부 사항은 PRM 훈련에 사용된 것과 동일합니다.

✔ 문장 N-2-3

Empirically, we find that including the revision history in context improves performance slightly (see Figure 18(b)).

🔍 단위 해석

경험적으로, 우리는 문맥에 수정 이력을 포함하는 것이 성능을 약간 향상시킨다는 것을 발견했습니다 (Figure 18(b) 참조).

✔ 문장 N-2-4

Additionally, even without the revisions in context, we see that sequential revisions still slightly outperforms parallel, demonstrating improvements from sequential sampling are not just due to the verifier's context.

🔍 단위 해석

또한, 문맥에 수정 이력이 없는 경우에도 여전히 순차적 수정이 병렬 수정보다 약간 더 우수한 성능을 보이는데, 이는 순차적 샘플링의 개선 효과가 단지 검증기의 문맥 때문만은 아님을 입증합니다.

시각 자료 [Figure 18]

(원문 캡션: **Figure 18:** Left: we compare the ORM we trained on the revision model’s outputs against the PRM we trained on the PaLM 2 — S^* base model’s outputs. We see that when applied to outputs from the revision model, the ORM adapted to the revision model outperforms the PRM, likely due to distribution shift with the revision model. Right: we ablate the effect of including previous revisions in the revision model verifier’s context. We see that including revisions in-context helps the verifier slightly, but both settings still outperform the parallel baseline.)

캡션 해석

왼쪽: 수정 모델의 출력에 대해 훈련된 ORM과 PaLM 2 — S^* 기본 모델의 출력에 대해 훈련된 PRM을 비교합니다. 수정 모델의 출력에 적용했을 때, 수정 모델에 적응된 ORM이 PRM보다 성능이 우수한데, 이는 수정 모델과의 분포 변화 때문일 가능성이 큼니다. 오른쪽: 수정 모델 검증기의 문맥에 이전 수정 내용을 포함하는 효과를 소거 분석(ablate)합니다. 문맥에 수정을 포함하는 것이 검증기에게 약간 도움이 되지만, 두 설정 모두 여전히 병렬 베이스라인보다 성능이 우수함을 알 수 있습니다.

PDF 시각 분석

PDF 16페이지의 **Figure 18** 분석:

- Left (a):** 파란색 선(Revision ORM)이 주황색 선(Base PRM)보다 위에 있습니다. 일반적으로 PRM이 ORM보다 좋지만(Figure 15 참조), 여기서는 ‘모델 불일치(Distribution Shift)’ 때문에 자기 짝인 ORM이 더 낫다는 것을 보여줍니다.
- Right (b):** 파란색 실선(With Context)이 점선(No Context)보다 아주 미세하게 높습니다. 즉, 이전 오답 기록을 보여주면 검증기가 채점을 더 잘하지만, 안 보여줘도(점선) 초록색 선(Parallel)보다는 높습니다. 즉, 순차적 수정(Sequential) 자체의 효용성이 검증기 성능 빨(Context 유무)보다 더 본질적이라는 뜻입니다.

Appendix O. COMPUTE OPTIMAL REVISIONS HYPERPARAMETERS

단락 O-1

In Table 1 we list the sequential/parallel ratios that we select between at each generation budget when estimating compute optimal scaling in Figure 6.

문장 O-1-1

In Table 1 we list the sequential/parallel ratios that we select between at each generation budget when estimating compute optimal scaling in Figure 6.

단위 해석

Table 1에 우리는 Figure 6에서 연산 최적화 스케일링을 추정할 때 각 생성 예산별로 선택 가능한 ‘순차/병렬 비율’ 목록을 나열했습니다.

시각 자료 [Table 1]

(원문 표: **Table 1:** We list the set of sequential to parallel ratios that we consider at each generation budget N...)

캡션 해석

각 생성 예산 N에서 우리가 고려하는 순차 대 병렬 비율 세트.

PDF 시각 분석

PDF 16페이지의 **Table 1** 분석:

- 예산(N)이 커질수록 선택할 수 있는 비율의 조합이 다양해집니다.
 - N=1일 때는 1:1 하나뿐입니다.
 - N=256일 때는 1:256 (순차 1번, 병렬 256개)부터 256:1 (순차 256번)까지 폭넓은 스펙트럼(1:16, 1:1, 16:1 등)을 가집니다.
- 이 표는 Figure 6의 “Compute Optimal” 곡선을 그릴 때, 각 점(예산)마다 이 표에 있는 비율 중 가장 성능이 좋은 것을 골라 찍었다는 근거 자료입니다.

Appendix P. RESTEM REVISION MODEL EXPERIMENTS

단락 P-1

We experimented with further optimizing our PaLM 2-S* revision model by training the model with a simplified RL algorithm: ReSTEM (Singh et al., 2024). Specifically, we generated 64 revision trajectories of maximum length 5 for each question on the MATH training set. We stopped the revision model at the first correct answer in each trajectory. Using this generated data, we then finetuned the base LM on the correct answer data. To help the model learn the task, we explicitly balanced the distribution of trajectory lengths.

✔ 문장 P-1-1

We experimented with further optimizing our PaLM 2-S* revision model by training the model with a simplified RL algorithm: ReSTEM (Singh et al., 2024).

🔍 단위 해석

우리는 단순화된 강화학습 알고리즘인 ReSTEM(Singh et al., 2024)으로 모델을 훈련시켜 PaLM 2-S* 수정 모델을 추가로 최적화하는 실험을 했습니다.

✔ 문장 P-1-2

Specifically, we generated 64 revision trajectories of maximum length 5 for each question on the MATH training set.

🔍 단위 해석

구체적으로, 우리는 MATH 훈련 세트의 각 질문에 대해 최대 길이 5인 수정 궤적을 64개씩 생성했습니다.

✔ 문장 P-1-3

We stopped the revision model at the first correct answer in each trajectory.

🔍 단위 해석

각 궤적에서 첫 번째 정답이 나오면 수정 모델을 중단시켰습니다.

✔ 문장 P-1-4

Using this generated data, we then finetuned the base LM on the correct answer data.

🔍 단위 해석

이렇게 생성된 데이터를 사용하여, 우리는 정답 데이터에 대해 기본 언어 모델(base LM)을 미세 조정했습니다.

✔ 문장 P-1-5

To help the model learn the task, we explicitly balanced the distribution of trajectory lengths.

🔍 단위 해석

모델이 작업을 학습하는 것을 돕기 위해, 우리는 궤적 길이의 분포를 명시적으로 균형 맞췄습니다.

📖 단락 P-2

In Figure 19, we plot the performance of this new revision model as we vary the sequential to parallel ratio. We see that additional sequential revisions substantially hurts performance with this new model. We hypothesize that this degradation is due to the fact that the online data obtained from running $ReST^{EM}$ exacerbates spurious correlations in revision data, causing the optimized model to fail to learn the revision task. We believe that using a more offline data collection strategy, as done in Qu et al. (2024b), may be more effective, and leave further exploration to future work.

✔ 문장 P-2-1

In Figure 19, we plot the performance of this new revision model as we vary the sequential to parallel ratio.

🔍 단위 해석

Figure 19에서, 우리는 순차 대 병렬 비율을 변화시키며 이 새로운 수정 모델의 성능을 도표로 나타냅니다.

✔ 문장 P-2-2

We see that additional sequential revisions substantially hurts performance with this new model.

🔍 단위 해석

우리는 이 새로운 모델의 경우 추가적인 순차적 수정이 성능을 상당히 저하시킨다는 것을 확인했습니다.

✔ 문장 P-2-3

We hypothesize that this degradation is due to the fact that the online data obtained from running $ReST^{EM}$ exacerbates spurious correlations in revision data, causing the optimized model to fail to learn the revision task.

🔍 단위 해석

우리는 이러한 성능 저하가 $ReST^{EM}$ 실행을 통해 얻은 온라인 데이터가 수정 데이터 내의 허위 상관관계(spurious correlations)를 악화시켜, 최적화된 모델이 수정 작업 자체를 학습하는 데 실패하게 만들었기 때문이라고 가설을 세웁니다.

✔ 문장 P-2-4

We believe that using a more offline data collection strategy, as done in Qu et al. (2024b), may be more effective, and leave further exploration to future work.

🔍 단위 해석

우리는 Qu et al. (2024b)에서 수행한 것처럼 더 오프라인 방식의 데이터 수집 전략을 사용하는 것이 효과적일 수 있다고 믿으며, 이에 대한 추가 탐구는 미래의 과제로 남겨둡니다.

🖼️ 시각 자료 [Figure 19]

(원문 캡션: **Figure 19:** Performance of our ReSTEM optimized revision model as the sequential to parallel ratio is varied. We use majority voting to select the answer. We see that this optimized revision model demonstrates substantial performance degradations with additional sequential revisions.)

🔍 캡션 해석

순차 대 병렬 비율 변화에 따른 ReSTEM 최적화 수정 모델의 성능. 답안 선택에는 다수결 투표를 사용합니다. 이 최적화된 수정 모델은 순차적 수정이 추가될수록 상당한 성능 저하를 보임을 알 수 있습니다.

📊 PDF 시각 분석

PDF 17페이지의 **Figure 19** 분석:

- **그래프 해석:** X축(순차적 샘플 수)이 오른쪽으로 갈수록 모든 예산(색깔별 선)에서 정확도가 뚝뚝 떨어집니다. 예를 들어, 보라색 선(2^7)은 왼쪽 끝(병렬 위주)에서는 높지만 오른쪽 끝(순차 위주)에서는 바닥을 칩니다. 이는 ReSTEM 방식의 훈련이 '고쳐서 정답을 맞는 능력'을 오히려 망가뜨렸음을 시각적으로 보여줍니다.

Appendix Q. REVISION MODEL EXAMPLE OUTPUTS

📖 단락 Q-1

In Figures 20, 21, 22, 23, 24, 25, and 26, we include select examples of our revision model’s outputs.

✔ 문장 Q-1-1

In Figures 20, 21, 22, 23, 24, 25, and 26, we include select examples of our revision model’s outputs.

🔍 단위 해석

Figure 20부터 26까지, 우리는 수정 모델의 출력 예시들을 선별하여 포함했습니다.

🖼️ 시각 자료 [Figure 20-26]

(원문 캡션 요약: 각 그림은 수정 모델이 1차, 2차 시도에서 실패하다가 최종적으로 정답을 맞히거나 형식을 수정하는 과정을 보여줍니다.)

📊 PDF 시각 분석 (대표 예시)

PDF 18~24페이지의 **Figure 20~26** 분석:

- **Figure 20 (PDF p.18):** 첫 번째 시도(Attempt 1)에서 계산 실수로 틀린 답을 냈지만, 모델이 스스로 "Let’s double check"라며 다시 계산하지는 않고, 새로운 시도(Attempt 3)에서 올바른 합계를 구해 정답 $\frac{25}{12}$ 를 도출하는 과정을 보여줍니다.
- **Figure 23 (PDF p.21):** 기수법 변환(Base 10 to Base 8) 문제에서 초기에는 변환 방식을 헷갈려 오답을 내다가, 마지막 시도에서 올바른 나눗셈 과정을 통해 정답 1001_8 을 찾아냅니다.
- **핵심:** 이 그림들은 모델이 문맥(이전의 오답)을 보고 "아, 이게 아니구나"라고 인지하여 다른 접근법이나 계산을 시도(Sequential Revision)하는 능력이 있음을 보여줍니다.

Appendix R. PRM BEAM SEARCH EXAMPLE OUTPUTS

📖 단락 R-1

In Figures 27, 28, 29, 30, 31, and 32, we include select examples of PRM beam search. We include the PRM score, between 0 and 1, for each step in the examples.

✅ 문장 R-1-1

In Figures 27, 28, 29, 30, 31, and 32, we include select examples of PRM beam search.

🔍 단위 해석

Figure 27부터 32까지, 우리는 PRM 빔 서치의 선별된 예시들을 포함했습니다.

✅ 문장 R-1-2

We include the PRM score, between 0 and 1, for each step in the examples.

🔍 단위 해석

우리는 예시의 각 단계마다 0과 1 사이의 PRM 점수를 함께 표기했습니다.

🖼️ 시각 자료 [Figure 27-32]

(원문 캡션 요약: 각 그림은 빔 서치 과정에서 각 단계(Step)가 어떤 PRM 점수를 받았는지를 보여줍니다.)

📄 PDF 시각 분석 (대표 예시)

PDF 25~30페이지의 **Figure 27~32** 분석:

- **Figure 27 (PDF p.25):** “Step 1 (score 0.77)” -> “Step 2 (score 0.82)” 식으로 단계가 진행될수록 점수가 높아지거나 유지되는 경로를 따라갑니다. 각 단계 끝에 `[score 0.XX]` 형태로 PRM이 매긴 점수가 표시되어 있습니다.
- **Figure 32 (PDF p.30):** 이 예시는 실패 사례(Failure mode)를 보여줍니다. 모델이 "Wait, I made a mistake"와 같은 문장을 반복하며 점수가 낮은 상태 (0.01)에서 헤어 나오지 못하거나, 무의미한 단계를 반복하는 모습을 볼 수 있습니다. 이는 섹션 5.3에서 언급된 "반복적이고 정보가 낮은 단계(repetitive low-information steps)"의 구체적인 시각적 예시입니다.